

LLD Design Problems

16 problems. LLD interview lo మీరు test అవుతున్నది coding వేగం కాదు – మీ **code** ని 6 నెలల తర్వాత **maintain చేయగలమా** అన్నది.

ప్రతి problem కి: interviewer అడిగే విధానం → clarifying questions → nouns→classes విభజన → UML class diagram → runnable JavaScript code → design patterns మరియు అవి ఎందుకు → extensibility test (కొత్త requirement వచ్చినప్పుడు ఏమవుతుంది) → interview lo చెప్పాల్సిన పూర్తి English script → follow-ups.

మొదట ఇది చదవండి

Read time **8 min** · ఇది skip చేస్తే మిగతాదంతా వృథా

LLD interview lo నిజంగా ఏం చూస్తారు

HLD lo scale గురించి అడుగుతారు. **LLD lo ఒక్క ప్రశ్న మాత్రమే ఉంటుంది:**

"ఈ మనిషి రాసిన code ని 6 నెలల తర్వాత, కొత్త requirement వచ్చినప్పుడు, మేము సులభంగా మార్చగలమా?"

అందుకే ఇక్కడ "పని చేసే code" సరిపోదు. Interviewer చూసేది – మీరు **ఏది మారుతుందో ముందే ఊహించి**, ఆ భాగాన్ని వేరు చేశారా అన్నది.

ప్రతి LLD INTERVIEW యొక్క నిజమైన ముగింపు

మీరు design చూపించాక interviewer ఖచ్చితంగా ఒక **కొత్త requirement** విసురుతాడు – "ఇప్పుడు electric vehicles add చెయ్యి", "ఇప్పుడు weekend rate పెట్టు", "ఇప్పుడు undo కూడా కావాలి". మీ design lo ఆ మార్పు కోసం **ఒక్క existing class ని కూడా edit చెయ్యకుండా** కొత్త class add చేయగలిగితే – మీరు గెలిచారు. ఇదే Open/Closed Principle యొక్క నిజమైన పరీక్ష మరియు ఈ book మొత్తం దాని చుట్టూనే తిరుగుతుంది.

Universal Framework – ఏ LLD problem కైనా ఇదే 6 అడుగులు

60-MINUTE LLD INTERVIEW · TIME BUDGET

Clarify

5 min

Use cases

5 min

Nouns → Classes

8 min

Class diagram

10 min

Code

22 min

Extend + Q&A

10 min

వెంటనే code రాయడం మొదలుపెట్టే candidates ఇక్కడ ఓడతారు

ఇక్కడ interview గెలుస్తారు లేదా ఓడతారు

#	అడుగు	ఏం చేయాలి
1	Clarify	Scope ని కుదించండి. "ఇది scope బయట" అని మీరే చెప్పడం maturity signal
2	Use cases	Actors ఎవరు, ప్రతి actor ఏం చేస్తాడు – bullet points గా
3	Nouns → Classes	Problem statement lo nouns underline. కానీ ప్రతి noun ఒక class కాదు – filter చేయడమే నైపుణ్యం
4	Relationships	Association / Aggregation / Composition / Inheritance. UML గీయండి
5	Patterns	ఎక్కడ variation ఉందో అక్కడే pattern. అనవసరంగా వద్దు

#	అడుగు	ఏం చేయాలి
6	Code + Extend	Core classes రాయండి, తర్వాత కొత్త requirement కి సిద్ధంగా ఉండండి

ఏ problem ఏ pattern నేర్పుతుంది

#	Problem	ప్రధాన pattern / పాఠం
01	Parking Lot	Strategy · capability model · concurrency
02	Elevator System	State machine · scheduling strategy
03	Movie Ticket Booking	Seat locking · optimistic concurrency · TTL holds
04	Splitwise	Split strategies · debt simplification graph
05	Vending Machine	State pattern – దీనికి ఇది canonical problem
06	ATM	State + Chain of Responsibility (cash dispenser)
07	LRU & LFU Cache	Data structure design · $O(1)$ · eviction as strategy
08	Rate Limiter	Algorithms as strategies · thread safety
09	Logging Framework	Chain of Responsibility · appenders · levels
10	Notification Service	Observer + Strategy + Template Method
11	Chess	Polymorphic move rules · validation pipeline
12	Text Editor Undo/Redo	Command + Memento
13	In-memory File System	Composite pattern
14	Cart & Discounts	Decorator / rule engine · pricing pipeline
15	Meeting Scheduler	Interval handling · conflict detection · recurrence
16	Library Management	Entity modelling · reservations · fines

ఈ 8 తప్పులు చేస్తే round అక్కడే అయిపోతుంది

LLD LO COMMON MISTAKES

1. **Anemic classes** – అన్నీ getters/setters, behaviour లేదు. అది struct, object కాదు.
2. **God class** – `ParkingLotService` lo 40 methods. బాధ్యతలు విడగొట్టలేదని అర్థం.
3. **Enum + if-else chain** – కొత్త type వచ్చినప్పుడు 10 చోట్ల edit. Polymorphism వాడాలి.
4. **అనవసర Singleton** – testability చచ్చిపోతుంది. Dependency injection మేలు.
5. **Pattern కోసం pattern** – ఏ మార్పు రావచ్చని ఆ pattern పెట్టారో చెప్పలేకపోతే అది over-engineering.
6. **Concurrency ని ప్రస్తావించకపోవడం** – multiple users ఉన్నారని మీరే చెప్పి, race conditions మాట్లాడకపోవడం.
7. **Validation లేకపోవడం** – invalid input వచ్చినప్పుడు ఏమవుతుందో ఆలోచించకపోవడం.
8. **మౌనంగా code రాయడం** – ఆలోచనని బయటికి చెప్పకపోతే interviewer కి మీ బుర్ర కనిపించదు.

Pattern ని ఎప్పుడు వాడాలి – ఒక పట్టిక

Pattern అనేది ఒక పదం: "ఇది భవిష్యత్తులో మారుతుంది" అని మీరు కట్టేది. మారదని తెలిస్తే pattern వద్దు.

సూచన (signal)	Pattern	ఉదాహరణ
"ఈ algorithm మారొచ్చు"	Strategy	Pricing, spot allocation, rate limiting
"Object ప్రవర్తన దాని state ని బట్టి మారుతుంది"	State	Vending machine, ATM, order lifecycle
"ఈ object మారితే ఇతరులకి తెలియాలి"	Observer	Notifications, event bus, UI updates
"Behaviour ని పొరలుగా పేర్చాలి"	Decorator	Discounts, surge pricing, middleware
"ఒక request ని పలువురు handle చేయొచ్చు"	Chain of Responsibility	Logging, validation, ATM cash dispensing
"Object creation సంక్లిష్టం / మారుతుంది"	Factory	Config నుంచి objects create చేయడం
"Action ని object గా చేసి undo చేయాలి"	Command	Undo/redo, transaction log, job queue
"Tree structure – leaf, container ఒకేలా"	Composite	File system, UI components, org chart
"Algorithm skeleton ఒకటే, steps మారతాయి"	Template Method	Report generation, <code>canFit()</code> rule

Design a Parking Lot

Difficulty **Medium** · Frequency **అత్యధికం** · నేర్పే concepts: abstraction, Strategy, Factory, OCP, concurrency

1. The Ask

"Design a parking lot system. It has multiple floors, different types of spots, and different types of vehicles. A car comes in, gets a ticket, parks, and pays on the way out."

LLD interviews lo ఇది **అత్యంత సాధారణ problem**. అందుకే interviewer దీన్ని 100 సార్లు విన్నాడు – మీరు textbook answer ఇస్తే గుర్తుండరు. గుర్తుండాలంటే **extensibility** మీద గెలవాలి.

ఈ PROBLEM LO నిజమైన TEST ఇది

Interviewer చివర్లో ఖచ్చితంగా అడుగుతాడు: "ఇప్పుడు **electric vehicle charging spots add చెయ్యి**" లేదా "weekend కి వేరే **rate పెట్టు**". మీ design lo ఆ మార్పు కోసం **ఒక్క existing class ని కూడా touch చెయ్యకుండా** కొత్త class add చేయగలిగితే – మీరు గెలిచారు. అదే Open/Closed Principle. అందుకే మొదటి నుంచీ Strategy pattern వైపు design చేయాలి.

2. Clarifying Questions

ప్రశ్న	ఇది design ని ఎలా మారుస్తుంది
ఎన్ని floors, ఎన్ని spots?	10 spots అయితే array చాలు. 10,000 అయితే per-floor availability index కావాలి
ఎవ vehicle types?	Vehicle → Spot mapping rules ఇక్కడి నుంచి వస్తాయి
Pricing ఎలా – గంటకా, slab లా?	Strategy pattern అవసరమా కాదా అన్నది ఇది decide చేస్తుంది
Entry/exit gates ఎన్ని?	ఒకటి కంటే ఎక్కువ అయితే concurrency ఒక real problem అవుతుంది
Spot ఎలా కేటాయించాలి – దగ్గరిదా, ఎదైనానా?	Allocation strategy ని pluggable చేయాలా అన్నది
Payment modes?	Cash / Card / UPI – ఇది మరో Strategy
Reservation / monthly pass ఉందా?	ఉంటే Spot కి RESERVED అనే మూడో state వస్తుంది

SCOPE ని మరే కుదించండి

"నేను ఇప్పుడు **single lot, multiple floors, 4 vehicle types, hourly pricing, multiple gates** – ఇంత scope తో వెళ్తున్నాను. Reservation, valet, number-plate recognition ని తర్వాత extension గా చూపిస్తాను." ఇలా చెప్పి మొదలుపెడితే 45 నిమిషాల్లో ముగించగలుగుతారు.

3. Requirements & Use Cases

Actors

- **Driver** – వచ్చి park చేసి, పోతూ pay చేస్తాడు
- **Entry gate** – ticket ఇస్తుంది
- **Exit gate** – fee లెక్కించి vacate చేస్తుంది
- **Admin** – floors/spots add చేస్తాడు

Use cases

- Vehicle కి సరిపోయే spot వెతకడం
- Ticket issue చేయడం (entry time తో)
- Fee లెక్కించడం
- Payment collect చేసి spot ని free చేయడం
- Display board lo ఖాళీ spots చూపడం

4. Nouns → Classes, Verbs → Methods

ఇది LLD lo మరు నేర్చుకోవాల్సిన **ఒకే ఒక్క trick**. Problem statement ని తీసుకుని nouns, verbs విడదీయండి.

PROBLEM STATEMENT → CLASS CANDIDATES

"A **vehicle** enters a **parking lot** , the **gate** issues a **ticket** , it **parks** in a **spot** on a **floor** , then **pays** a **fee** ."

NOUNS → CLASSES

Vehicle · ParkingLot · Gate · Ticket
ParkingSpot · ParkingFloor · Fee
Fee ఒక్కటే class కాదు – అది Ticket నుంచి లెక్కించే విలువ. అలాంటివి వదిలేయాలి.

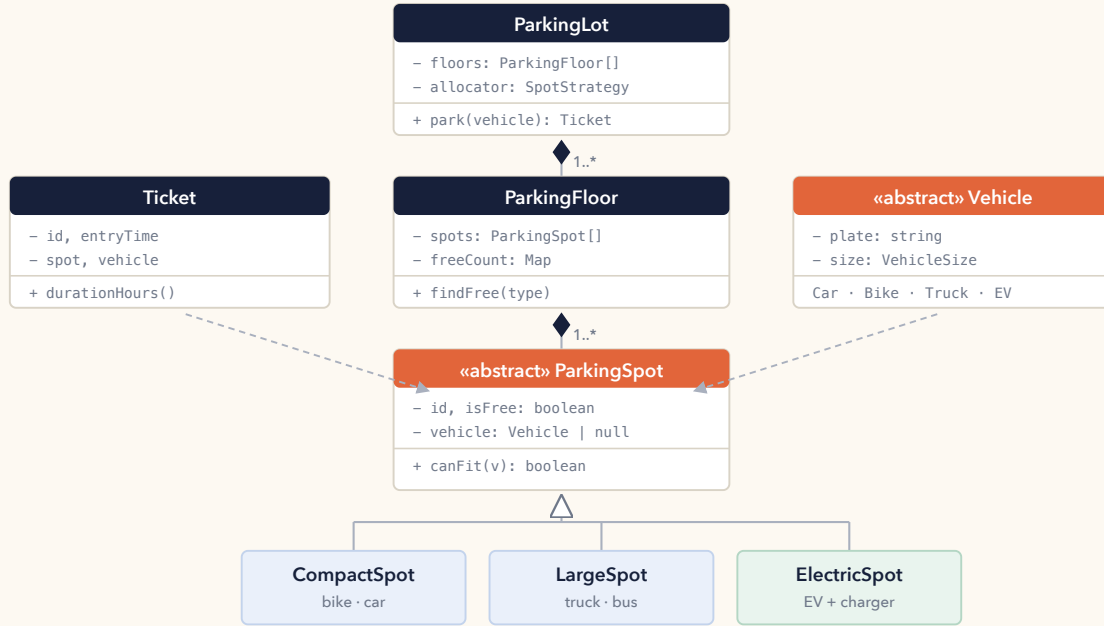
VERBS → METHODS

enter() · issueTicket() · park()
pay() · vacate() · findSpot()
ప్రతి method ఏ class కి చెందుతుందో నిర్ణయించడమే అసలు design work.

ప్రతి noun ఒక class కాదు. **Fee** ఒక computed value మాత్రమే – దానికి class వద్దు. ఇలా filter చేయడమే junior/senior తేడా.

5. Class Diagram

PARKING LOT · CORE CLASS STRUCTURE



◆ = composition (Lot చస్తే Floor కూడా చస్తుంది) · ▷ = inheritance · ↔ = association. **ElectricSpot అకుపచ్చగా ఉంది** – అది follow-up question lo add అయ్యే class. Design సరిగ్గా ఉంటే మిగతా ఏ class నీ ముట్టుకోకుండా అది వచ్చి చేరుతుంది.

ఇక్కడ చేసే అతి పెద్ద తప్పు

చాలా మంది **ParkingSpot** lo ఒక **type** field పెట్టి, తర్వాత ప్రతిచోటా `if (spot.type === "COMPACT") ... else if ...` రాస్తారు. అప్పుడు కొత్త spot type వచ్చినప్పుడు **10 చీట్లు if-else edit చేయాలి**. బదులుగా `canFit(vehicle)` అనే method ని subclass lo override చేస్తే – కొత్త type అంటే కొత్త class, existing code lo సున్నా మార్పు. **ఇదే polymorphism యొక్క అసలు వాడుక.**

6. Code – Vehicles & Spots

మొదట **capability model**. Vehicle తనకు ఏం కావాలో చెప్తుంది, Spot తను ఏం ఇవ్వగలదో చెప్తుంది. ఈ ఒక్క idea వల్ల భవిష్యత్తులో ఏ కొత్త spot type అయినా if-else లేకుండా చేరిపోతుంది.

```
// Size ordering ముఖ్యం – చిన్న vehicle పెద్ద spot lo పట్టేస్తుంది, తిరగబడి కాదు.
const VehicleSize = Object.freeze({ BIKE: 1, CAR: 2, TRUCK: 3 });

class Vehicle {
  constructor(plate, size) {
    if (new.target === Vehicle) throw new Error('Vehicle is abstract');
    this.plate = plate;
    this.size = size;
  }
  // ఈ vehicle కి spot నుంచి ఏం కావాలి. Default: ఏమీ వద్దు.
  get requirements() { return []; }
}
```

```

class Bike extends Vehicle { constructor(plate) { super(plate, VehicleSize.BIKE); }
}
class Car extends Vehicle { constructor(plate) { super(plate, VehicleSize.CAR); }
}
class Truck extends Vehicle { constructor(plate) { super(plate, VehicleSize.TRUCK); }
}

```

```

class ParkingSpot {
  constructor(id, distanceFromGate) {
    if (new.target === ParkingSpot) throw new Error('ParkingSpot is abstract');
    this.id = id;
    this.distance = distanceFromGate; // gate నుంచి దూరం (allocation కి)
    this.vehicle = null;
  }

  get isFree() { return this.vehicle === null; }
  get maxSize() { throw new Error('subclass must define maxSize'); }
  get features() { return new Set(); } // ఈ spot ఏం ఇవ్వగలదు

  // ఇక్కడ if-else లేదు. Rule ఒక్కటే, subclass లు data ఇస్తాయి.
  canFit(vehicle) {
    return this.isFree
      && vehicle.size <= this.maxSize
      && vehicle.requirements.every((r) => this.features.has(r));
  }

  assign(vehicle) {
    if (!this.isFree) throw new Error(`SPOT_TAKEN:${this.id}`);
    this.vehicle = vehicle;
  }
  release() { this.vehicle = null; }
}

class CompactSpot extends ParkingSpot { get maxSize() { return VehicleSize.CAR; } }
class LargeSpot extends ParkingSpot { get maxSize() { return VehicleSize.TRUCK; } }

```

ఇక్కడ ఏం జరిగిందో గమనించండి

`canFit` అనే rule **base class** లో ఒకే ఒక్కసారి రాశాం. Subclass లు logic రాయవు – కేవలం `maxSize`, `features` అనే data ఇస్తాయి. అంటే కొత్త spot type = 3 lines. ఇది interviewer కి బాగా కనిపించే structure.

7. Code – Floor, Strategies, Lot

```

class ParkingFloor {
  constructor(number, spots) { this.number = number; this.spots = spots; }
  availableFor(vehicle) { return this.spots.filter((s) => s.canFit(vehicle)); }
  get freeCount() { return this.spots.filter((s) => s.isFree).length; }
}

class Ticket {
  constructor(id, vehicle, spot, entryTime) {
    Object.assign(this, { id, vehicle, spot, entryTime });
  }
}

```

```

    }
}

```

Spot ఎంచుకునే విధానం **ఎప్పుడైనా మారొచ్చు** – అందుకే దాన్ని ParkingLot బయట, ఒక pluggable strategy గా ఉంచుతాం.

```

class SpotAllocationStrategy {
    pick(floors, vehicle) { throw new Error('abstract'); }
}

// "సరిపోయే వాటిలో అతి తక్కువ specialised spot, ఆ తర్వాత అతి దగ్గరిది."
// EV spot ని సాధారణ car ఆక్రమించకుండా ఈ ordering అవుతుంది.
class BestFitNearestStrategy extends SpotAllocationStrategy {
    pick(floors, vehicle) {
        const candidates = floors.flatMap((f) => f.availableFor(vehicle));
        if (candidates.length === 0) return null;
        return candidates.sort(
            (a, b) => (a.features.size - b.features.size) || (a.distance - b.distance)
        )[0];
    }
}

class HourlyPricing {
    constructor(ratePerHourBySize) { this.rates = ratePerHourBySize; }
    price(ticket, exitTime) {
        const ms = Math.max(exitTime - ticket.entryTime, 1);
        const hours = Math.ceil(ms / 3_600_000); // మొదలైన గంట పూర్తి గంట
        return hours * this.rates[ticket.vehicle.size];
    }
}

```

```

class ParkingLot {
    constructor({ floors, allocator, pricing }) {
        this.floors = floors;
        this.allocator = allocator; // ← injected, hardcoded కాదు
        this.pricing = pricing; // ← injected
        this.activeTickets = new Map();
        this.seq = 0;
    }

    park(vehicle, now = Date.now()) {
        const spot = this.allocator.pick(this.floors, vehicle);
        if (!spot) throw new Error('LOT_FULL');
        spot.assign(vehicle); // ← ఇక్కడే atomic check జరుగుతుంది
        const ticket = new Ticket(`T${++this.seq}`, vehicle, spot, now);
        this.activeTickets.set(ticket.id, ticket);
        return ticket;
    }

    unpark(ticketId, now = Date.now()) {
        const ticket = this.activeTickets.get(ticketId);
        if (!ticket) throw new Error('INVALID_TICKET');
        const amount = this.pricing.price(ticket, now);
        ticket.spot.release();
        this.activeTickets.delete(ticketId);
    }
}

```

```

    return { ticketId, spotId: ticket.spot.id, amount };
  }
}

```

```

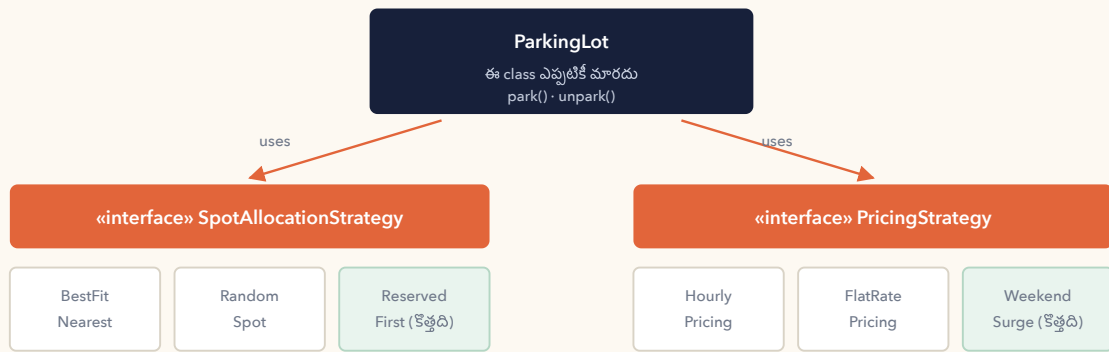
// ---- వాడుక ----
const floor1 = new ParkingFloor(1, [
  new CompactSpot('C-1', 5), new CompactSpot('C-2', 12), new LargeSpot('L-1', 30),
]);

const lot = new ParkingLot({
  floors: [floor1],
  allocator: new BestFitNearestStrategy(),
  pricing: new HourlyPricing({ [VehicleSize.BIKE]: 10, [VehicleSize.CAR]: 30,
    [VehicleSize.TRUCK]: 60 }),
});

const t0 = Date.now();
const ticket = lot.park(new Car('TS09AB1234'), t0);
console.log(ticket.spot.id); // "C-1" (దగ్గరిది)
console.log(lot.unpark(ticket.id, t0 + 2.5 * 3_600_000)); // 3 గంటలు × 30 = 90

```

STRATEGY INJECTION · ఏది మారుతుంది, ఏది మారదు



ఆకుపచ్చ boxes = interview lo follow-up గా అడిగే కొత్త requirements. అవి కొత్త files మాత్రమే – ParkingLot lo ఒక్క line కూడా మారదు. ఇదే Strategy pattern వాడటానికి కారణం.

8. Deep Dive – రెండు gates, ఒకే spot (Concurrency)

ఇదే ఈ problem lo **అసలు senior question**. మీ code single-threaded గా perfect గా ఉంది. కానీ నిజమైన lot lo 4 entry gates ఉంటాయి, ప్రతి ఒక్కటీ వేరే server process.

THE CHECK-THEN-ACT RACE • ఒకే SPOT రెండు CARS కి

GATE A



Gate A గెలిచింది.

Gate B retry
చేయాలి – crash
కాకూడదు.

GATE B



Gate B `findFree()` చేసినప్పుడు C-7 ఖాళీగానే ఉంది. కానీ `assign()` చేసేసరికి Gate A తీసుకుంది. దీన్ని **check-then-act race condition** అంటారు.

పరిష్కారం	ఎలా	ఎప్పుడు వాడాలి
Optimistic (సిఫార్సు)	<code>UPDATE spots SET vehicle_id=? WHERE id=? AND vehicle_id IS NULL</code> – ఈ update 0 rows ఇస్తే వేరే gate గెలిచింది, తర్వాతి spot try చేయండి	Contention తక్కువ ఉన్నచోట (parking lot సరిగ్గా అదే)
Pessimistic lock	<code>SELECT ... FOR UPDATE</code> తో row ని lock చేసి తర్వాత update	Contention చాలా ఎక్కువ ఉంటే. కానీ throughput తగ్గుతుంది
Redis atomic pop	ఒక్కో spot type కి free-spot list. <code>LPOP</code> atomic కాబట్టి race లేదు	చాలా ఎక్కువ gates, in-memory speed కావాలంటే
Single-threaded allocator	Allocation ని ఒకే service కి queue చేయడం	సులభం కానీ ఇదే bottleneck అవుతుంది

నా జవాబు

"నేను **optimistic approach** తీసుకుంటాను. Parking lot lo contention నిజానికి చాలా తక్కువ – 500 spots ఉంటే ఇద్దరు ఒకే spot కి డిక్లొస్టర్ అవకాశం అరుదు. కాబట్టి conditional update చాలు, దానికి lock overhead లేదు. Conflict వస్తే retry – 3 సార్లు retry చేసి కూడా fail అయితే అప్పుడు lot నిజంగా నిండిపోయినట్టు. నా `assign()` ఇప్పటికే `SPOT_TAKEN` throw చేస్తుంది కాబట్టి ఆ seam అప్పుడే ఉంది – దాన్ని DB level కి తీసుకెళ్ళడమే మిగిలింది."

9. Extensibility Test – follow-up ని ఎదుర్కోవడం

Interviewer: "Now add electric vehicles that need charging, and charge 1.5× on weekends."

// ↓ ఇవన్నీ కొత్త classes. పైన రాసిన ఒక్క class ని కూడా edit చేయలేదు.

```
class ElectricCar extends Car {
  get requirements() { return ['CHARGING']; }
}
```

```
class ElectricSpot extends ParkingSpot {
  get maxSize() { return VehicleSize.CAR; }
```

```

    get features() { return new Set(['CHARGING']); }
}

// Decorator – ఉన్న pricing ని చుట్టి weekend multiplier వేస్తుంది.
class WeekendSurgePricing {
    constructor(base, multiplier = 1.5) { this.base = base; this.multiplier = multiplier; }
    price(ticket, exitTime) {
        const day = new Date(ticket.entryTime).getDay(); // 0 = ఆదివారం, 6 = శనివారం
        const surge = (day === 0 || day === 6) ? this.multiplier : 1;
        return Math.round(this.base.price(ticket, exitTime) * surge);
    }
}

```

```

const floor2 = new ParkingFloor(2, [
    new CompactSpot('C-9', 8),
    new ElectricSpot('E-1', 40),
]);

const lot2 = new ParkingLot({
    floors: [floor2],
    allocator: new BestFitNearestStrategy(),
    pricing: new WeekendSurgePricing(
        new HourlyPricing({ [VehicleSize.BIKE]: 10, [VehicleSize.CAR]: 30,
        [VehicleSize.TRUCK]: 60 })
    ),
});

// సాధారణ car ఖరీదైన EV spot ని ఆక్రమించదు – features.size ordering దాన్ని అవుతుంది.
console.log(lot2.park(new Car('TS09AA0001')).spot.id); // "C-9"
console.log(lot2.park(new ElectricCar('TS09EV0002')).spot.id); // "E-1"

```

SCORE CARD

మార్చిన existing classes: **0**. కొత్తగా రాసినవి: **3**.

ఇదే మీరు బయటికి చెప్పాల్సిన వాక్యం – "notice that I didn't have to modify a single existing class".

Interviewer దీని కోసమే ఎదురు చూస్తున్నాడు.

10. వాడిన Patterns – ప్రతిదానికీ ఒక కారణం ఉండాలి

Pattern	ఎక్కడ	ఎందుకు (ఇదే అసలు ప్రశ్న)
Strategy	<code>SpotAllocationStrategy</code> , <code>PricingStrategy</code>	Algorithm భవిష్యత్తులో మారుతుందని తెలుసు. మారే భాగాన్ని బయట పెట్టాం
Decorator	<code>WeekendSurgePricing</code> ఇంకో pricing ని చుడుతుంది	Surge, discount, tax – వీటిని కలిపి పేర్చేచ్చు (stack)
Template Method	<code>ParkingSpot.canFit()</code>	Algorithm base lo, data subclass lo. Subclass logic duplicate చేయదు

Pattern	ఎక్కడ	ఎందుకు (ఇదే అసలు ప్రశ్న)
Factory	<code>SpotFactory.create(type, id)</code> (config నుంచి lot build చేసేటప్పుడు)	JSON config నుంచి spots create చేసేటప్పుడు <code>new</code> calls ఒకేచోట
Singleton	⚠ <code>ParkingLot</code> కి వాడొద్దు	చాలా మంది ఇక్కడ Singleton పెడతారు. కానీ అప్పుడు unit test lo రెండు lots create చేయలేరు. Dependency injection మేలు

PATTERN TRAP

"ఎవ patterns వాడారు?" అని అడిగినప్పుడు జాబితా చదవొద్దు. **ఒక్క pattern కి - ఏ మార్పు రావొచ్చని ఊహించి దాన్ని పెట్టానో** చెప్పండి. Pattern అనేది ఒక bet: "ఇది మారుతుంది" అని మీరు కట్టిన పందెం. Bet లేకుండా pattern వేస్తే అది over-engineering.

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Before I design anything, a few questions: how many floors and spots roughly, what vehicle types do we support, is pricing hourly or slab-based, how many entry and exit gates, and do we need reservations? I'll assume one lot with multiple floors, four vehicle types, hourly pricing, and multiple gates – and I'll treat reservations as an extension."

"Pulling the nouns out of the problem gives me **Vehicle, ParkingLot, ParkingFloor, ParkingSpot, Ticket and Gate**. Fee is a noun too, but it's a computed value rather than an entity, so I won't make a class for it."

"The central modelling decision is how a spot decides whether a vehicle fits. The naive version is a type enum on the spot and an if-else chain wherever we allocate – but then every new spot type means editing that chain in several places. Instead I'll have the vehicle declare **what it requires** and the spot declare **what it offers**. The fit rule lives once in the base class, and subclasses only supply data. A new spot type becomes three lines and touches nothing else."

"Allocation and pricing both get injected as strategies, because both are things I'm confident will change. Today it's nearest-spot-first and hourly rates; tomorrow it's reserved-spots-first and weekend surge. I want those to be new classes, not edits."

"One subtlety in allocation: I sort candidate spots by how **specialised** they are before sorting by distance. Otherwise a regular car parks in the EV bay just because it happens to be closer, and a real EV then has nowhere to charge."

"The part I'd want to dig into is concurrency. With four gates, two of them can read the same spot as free and both try to take it. That's a classic check-then-act race. I'd solve it optimistically – a conditional update that only succeeds if the spot is still free, and the losing gate retries with the next candidate. Parking-lot contention is genuinely low, so paying for pessimistic locking would be the wrong trade. My **assign** method already throws when a spot is taken, so the seam for that is in place."

"On patterns: Strategy for allocation and pricing, Decorator to stack surge or discount on top of a base price, and Template Method for the fit rule. I'd deliberately avoid making ParkingLot a Singleton – it's the reflex answer here, but it makes the whole thing untestable, since you can't stand up two independent lots in a unit test."

"If you want to throw a new requirement at me – EV charging, monthly passes, valet – I'd like to show that it lands as new classes rather than edits."

11. Follow-ups & Gotchas

Interviewer అడిగేది	మీ జవాబు
"Lot నిండిపోతే?"	<code>park()</code> <code>LOT_FULL</code> throw చేస్తుంది. Display board lo per-type free counts చూపిస్తాం. Floor కి <code>Map<spotType, freeCount></code> maintain చేస్తే $O(1)$ lo చెప్పొచ్చు

Interviewer అడిగేది	మీ జవాబు
"10,000 spots ఉంటే <code>filter()</code> నెమ్మది కదా?"	అవును, అది $O(n)$. Per floor, per type ఒక free-spot min-heap (distance key) పెడితే allocation $O(\log n)$. Interface మారదు – <code>availableFor()</code> implementation మాత్రమే మారుతుంది
"Ticket పోగొట్టుకుంటే?"	Number plate తో lookup – <code>Map<plate, ticket></code> అనే రెండో index. Lost-ticket penalty rate
"Payment fail అయితే?"	Spot ని release చెయ్యకూడదు. <code>Ticket</code> కి state machine: <code>ACTIVE → PAYMENT_PENDING → PAID → CLOSED</code> . <code>PAID</code> అయ్యాకే <code>release()</code>
"Multiple lots (city అంతా)?"	ఇది LLD నుంచి HLD కి jump. <code>ParkingLot</code> ఒక aggregate root అవుతుంది, geo-index (geohash) తో దగ్గరి lot వెతకడం
"Spot ఖాళీ అయినా vehicle అక్కడే ఉంటే?"	Sensor-based reconciliation job – physical state vs system state ని పోల్చి mismatch ని admin కి report చేయడం

ఈ 5 తప్పులు చేయకండి

1. అన్నీ getters/setters ఉన్న anemic classes – behaviour లేని class అంటే అది struct, object కాదు.
2. `ParkingLot` ని Singleton చేయడం – testability చచ్చిపోతుంది.
3. `type` enum + if-else chain – కొత్త type వచ్చినప్పుడు 10 చోట్ల edit.
4. Pricing ని `unpark()` లోపల hardcode చేయడం.
5. Concurrency ని అస్సలు ప్రస్తావించకపోవడం – multiple gates ఉన్నాయని మీరే చెప్పాక దాన్ని వదిలేయడం చాలా పెద్ద gap.

Design an Elevator System

Difficulty **Medium-Hard** · Frequency ఎక్కువ · నేర్పే concepts: state machine, scheduling strategy, two request types

1. The Ask

"Design the control system for a building with four elevators and fifteen floors."

ఈ **PROBLEM LO 90%** మంది మిస్ చేసే విషయం

ఇక్కడ రెండు పూర్తిగా వేర్వేరు రకాల requests ఉన్నాయి, మరియు అవి వేరే information ని మోసుకొస్తాయి:

Hall call (బయటి request) – "5వ అంతస్తులో ఎవరో ఖైదీ వెళ్ళాలనుకుంటున్నారు". ఇక్కడ floor + direction తెలుసు, కానీ ఎక్కడికి వెళ్ళాలో తెలియదు. ఇది ఏ elevator కి ఇవ్వాలో **system నిర్ణయిస్తుంది**.

Car call (లోపలి request) – "ఈ elevator lo ఎవరో 9 నోక్కారు". ఇక్కడ destination తెలుసు, మరియు ఏ elevator అనేది ఇప్పటికే నిర్ణయమైపోయింది.

ఈ తేడాని గుర్తించకుండా ఒక Request class రాస్తే, తర్వాత scheduling logic గందరగోళం అవుతుంది.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఎన్ని elevators, ఎన్ని floors?	Scheduling సంక్లిష్టత	4 elevators, 15 floors
Optimisation లక్ష్యం ఏమిటి?	Wait time నా, energy నా, throughput నా	సగటు wait time
Express elevators (కొన్ని floors మాత్రమే)?	ఉంటే elevator కి "servable floors" concept	ఇప్పుడు వద్దు, extension గా
Capacity limit ఉందా?	నిండిన elevator కొత్త hall calls తీసుకోకూడదు	అవును, 8 మంది
Emergency / maintenance mode?	State machine lo అదనపు states	అవును
Simulation నా real-time నా?	Tick-based design సులభం, testable	Tick-based

3. Nouns → Classes

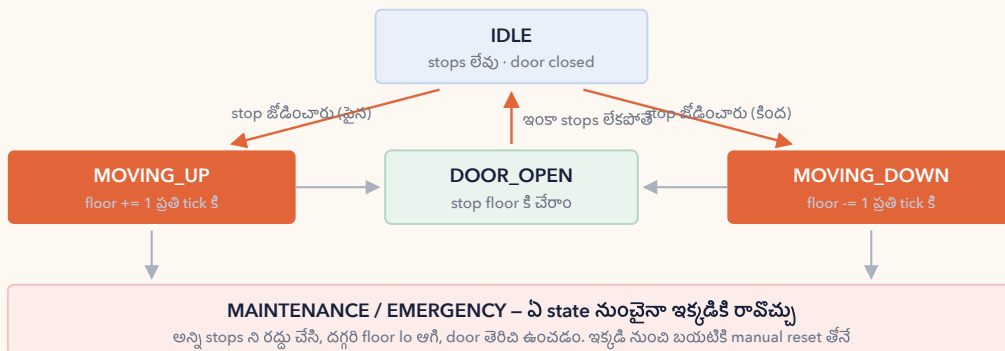
Class	బాధ్యత
Elevator	తన floor, direction, door, pending stops – తనని తాను నడుపుకోవడం
ElevatorSystem	Hall calls ని elevators కి కేటాయించడం, అందరినీ tick చేయడం
SchedulingStrategy	"ఈ hall call ఎవరికి ఇవ్వాలి" – pluggable
Direction, ElevatorState	Enums
Floor	⚠ ఒక number తప్ప floor కి behaviour లేదు → class వద్దు
Button	⚠ ఇది UI. Domain model lo అవసరం లేదు
Person	⚠ System వ్యక్తులని track చేయదు, capacity count మాత్రమే

"FLOOR ని CLASS చేయకూడదా?" – దీన్ని బయటికి చెప్పండి

"Floor కి state లేదు, behaviour లేదు – అది ఒక integer. దానికి class రాస్తే అది **అనవసరమైన abstraction**. కానీ ఒకవేళ ప్రతి floor కి తనదైన లక్షణాలు ఉంటే (access control, floor-specific display, restricted access) అప్పుడు నేను దాన్ని class చేస్తాను. **ఇప్పుడు లేని అవసరం కోసం abstraction సృష్టించను.**" – YAGNI ని ఇలా చూపించడం interviewer కి బాగా నచ్చుతుంది.

4. Deep Dive – State Machine

ELEVATOR STATE MACHINE • ప్రతి TRANSITION కి ఒక కారణం



Door అనేది **ఒక ప్రత్యేక state**, ఒక boolean flag కాదు. ఎందుకంటే door తెరిచి ఉన్నప్పుడు elevator కదలకూడదు – ఆ నియమాన్ని state machine సహజంగా అమలు చేస్తుంది. Boolean flag పెడితే "door open ఉందా?" అని ప్రతిచోటా check చేయాలి, ఒకచోట మర్చిపోతే bug.

5. Deep Dive – Scheduling (LOOK algorithm)

Strategy	ఎలా	సమస్య
FCFS	వచ్చిన క్రమంలో	✗ $1 \rightarrow 15 \rightarrow 2 \rightarrow 14$ – elevator పిచ్చిగా తిరుగుతుంది. దారుణమైన wait time
Nearest first (SSTF)	అతి దగ్గరి stop ముందు	⚠ Starvation – మధ్య floors lo traffic ఎక్కువ ఉంటే చివరి floors ఎప్పటికీ రావు
SCAN (lift algorithm)	ఒక దిక్కులో చివరిదాకా వెళ్ళి, తిరిగి	✅ Fair, కానీ stops లేకపోయినా చివరిదాకా వెళ్తుంది
LOOK ✅	ఒక దిక్కులో చివరి stop వరకే వెళ్ళి, తర్వాత తిరగడం	✅ SCAN యొక్క fairness + అనవసర ప్రయాణం లేదు. నిజమైన lifts ఇదే

STARVATION ని ప్రస్తావించండి

"Nearest-first అనేది సహజమైన ఎంపిక కానీ దానికి starvation సమస్య ఉంది – 15వ floor lo ఎవరో వేచి ఉండగా, elevator 3-4-5 floors మధ్య తిరుగుతూ ఉంటే వాళ్ళకి ఎప్పటికీ రాదు. LOOK ఈ సమస్యని పరిష్కరిస్తుంది ఎందుకంటే అది ఒక దిక్కులో ఉన్న అన్ని stops ని పూర్తి చేసిన తర్వాతే తిరుగుతుంది – ప్రతి request కి ఒక upper bound wait ఉంటుంది." – Fairness గురించి మాట్లాడటం seniority signal.

6. Code

```
const Direction = Object.freeze({ UP: 1, IDLE: 0, DOWN: -1 });

class Elevator {
  constructor(id, minFloor = 0, maxFloor = 14, capacity = 8) {
    Object.assign(this, { id, minFloor, maxFloor, capacity });
    this.currentFloor = minFloor;
    this.direction = Direction.IDLE;
    this.doorOpen = false;
    this.load = 0;
    this.stops = new Set(); // ఇంకా చేరాల్సిన floors
  }

  get isFull() { return this.load >= this.capacity; }

  addStop(floor) {
    if (floor < this.minFloor || floor > this.maxFloor) throw new Error('INVALID_FLOOR');
    if (floor === this.currentFloor && !this.doorOpen) { this.doorOpen = true; return; }
    this.stops.add(floor);
    if (this.direction === Direction.IDLE) this.#chooseDirection();
  }

  // LOOK: ప్రస్తుత దిక్కులో stops ఉన్నంతవరకు అటే వెళ్ళు, లేకపోతే తిరుగు.
  #hasStopAhead() {
    return [...this.stops].some((f) =>
      this.direction === Direction.UP ? f > this.currentFloor : f <
    );
  }
}
```

```

this.currentFloor);
}

#chooseDirection() {
  if (this.stops.size === 0) { this.direction = Direction.IDLE; return; }
  const nearest = [...this.stops].reduce((a, b) =>
    Math.abs(b - this.currentFloor) < Math.abs(a - this.currentFloor) ? b : a);
  this.direction = nearest > this.currentFloor ? Direction.UP : Direction.DOWN;
}

step() {
  if (this.doorOpen) { this.doorOpen = false; this.#chooseDirection(); return; }
  if (this.stops.size === 0) { this.direction = Direction.IDLE; return; }
  if (this.direction === Direction.IDLE || !this.#hasStopAhead())
  this.#chooseDirection();
  this.currentFloor += this.direction;
  if (this.stops.delete(this.currentFloor)) this.doorOpen = true; // చేరాం → తలుపు
}

toString() {
  const arrow = { 1: '↑', 0: '.', '-1': '↓' }[this.direction];
  return `${this.id}@${this.currentFloor}${arrow}${this.doorOpen ? '[open]' : ''}`;
}
}

```

```

// Scheduling ని బయట పెట్టడం - ఇదే ఈ design lo అతి ముఖ్యమైన నిర్ణయం.
class SchedulingStrategy {
  pick(elevators, floor, direction) { throw new Error('abstract'); }
}

class NearestCarStrategy extends SchedulingStrategy {
  #cost(e, floor, direction) {
    if (e.isFull) return Infinity; // నిండినది తీసుకోకూడదు
    const distance = Math.abs(e.currentFloor - floor);
    if (e.direction === Direction.IDLE) return distance;
    const movingToward = Math.sign(floor - e.currentFloor) === e.direction;
    if (movingToward && e.direction === direction) return distance; // దారిలోనే ఉంది -
    ఉత్తమం
    return distance + 100; // తిరిగి రావాలి - పెద్ద జరిమానా
  }
  pick(elevators, floor, direction) {
    const best = elevators.reduce((a, b) =>
      this.#cost(b, floor, direction) < this.#cost(a, floor, direction) ? b : a);
    if (this.#cost(best, floor, direction) === Infinity) return null; // అందరూ
    నిండారు
    return best;
  }
}

```

```

class ElevatorSystem {
  constructor(elevators, strategy) { this.elevators = elevators; this.strategy =
  strategy; }

  // బయటి request: floor + direction తెలుసు, elevator ని మనం ఎంచుకుంటాం
  hallCall(floor, direction) {
    const elevator = this.strategy.pick(this.elevators, floor, direction);

```

```

        if (!elevator) throw new Error('ALL_ELEVATORS_FULL');
        elevator.addStop(floor);
        return elevator.id;
    }

    // లోపలి request: elevator ఇప్పటికే తెలుసు, destination కొత్తది
    carCall(elevatorId, floor) {
        const e = this.elevators.find((x) => x.id === elevatorId);
        if (!e) throw new Error('NO_SUCH_ELEVATOR');
        e.addStop(floor);
    }

    step() { this.elevators.forEach((e) => e.step()); }
    snapshot() { return this.elevators.map((e) => e.toString()).join(' '); }
}

```

```

// ---- simulation ----
const sys = new ElevatorSystem(
    [new Elevator('E1'), new Elevator('E2')],
    new NearestCarStrategy()
);
sys.elevators[1].currentFloor = 8; // E2 పైన ఉంది

console.log(sys.hallCall(7, Direction.UP)); // "E2" - 1 floor దూరం, E1 కి 7
sys.carCall('E2', 11); // ఎక్కాక 11 నొక్కారు

for (let t = 1; t <= 6; t++) { sys.step(); console.log(t, sys.snapshot()); }
// 1 E1@0· E2@7↑[open] ← 7 కి చేరి తలుపు తెరిచింది
// 2 E1@0· E2@7↑
// 3 E1@0· E2@8↑
// 4 E1@0· E2@9↑
// 5 E1@0· E2@10↑
// 6 E1@0· E2@11↑[open] ← గమ్యానికి చేరింది

```

7. Extensibility Test

Interviewer: "Now add an express elevator that only serves floors 0, 10, 11, 12."

```

// కొత్త class. ఉన్న ఏ class నీ edit చేయలేదు.
class ExpressElevator extends Elevator {
    constructor(id, servableFloors) {
        super(id, Math.min(...servableFloors), Math.max(...servableFloors));
        this.servable = new Set(servableFloors);
    }

    canServe(floor) { return this.servable.has(floor); }
    addStop(floor) {
        if (!this.canServe(floor)) throw new Error('FLOOR_NOT_SERVED');
        super.addStop(floor);
    }
}

```

ఇక్కడ ఒక నిజాయితీ అవసరం

Express elevator ని strategy కూడా గౌరవించాలి – లేకపోతే system దానికి 5వ floor call ఇచ్చి exception వస్తుంది. అంటే base Elevator కి canServe(floor) అనే method ఉండాలి (default: true), మరియు strategy దాన్ని check చేయాలి.

ఇది ఒక చిన్న Liskov violation ప్రమాదం – subclass, parent కంటే తక్కువ అంగీకరిస్తోంది. దీన్ని interview lo మీరే గుర్తించి చెప్తే అది గొప్ప signal: "నేను దీన్ని canServe అనే explicit capability check తో పరిష్కరిస్తాను, exception మీద ఆధారపడను."

8. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
State	Elevator యొక్క IDLE/MOVING/DOOR_OPEN	Door open ఉన్నప్పుడు కదలకూడదు – ఆ నియమం structure lo ఉంది, if-else lo కాదు
Strategy	SchedulingStrategy	Algorithm మారుతుందని ఖచ్చితంగా తెలుసు (LOOK → ML-based → energy-saving)
Template Method	Elevator.step()	Skeleton base lo, canServe లాంటివి subclass lo
⚠ Singleton వద్దు	ElevatorSystem	ఒక building కి ఒకటే కదా అనిపిస్తుంది. కానీ test lo రెండు buildings కావాలి. DI మేలు

🔧 INTERVIEW SCRIPT · ఇలా చెప్పుండి (ENGLISH)

"Before modelling, let me confirm: how many elevators and floors, what are we optimising for – average wait time, throughput, or energy – and do we need express elevators or a maintenance mode? I'll assume four elevators, fifteen floors, optimising average wait time, with maintenance in scope and express as an extension."

"The first thing I want to call out is that there are **two different kinds of request**, and conflating them is the classic mistake here. A hall call comes from outside: it tells me a floor and a direction, but not the destination – and crucially, which elevator serves it is **my decision**. A car call comes from inside: it tells me a destination, and the elevator is already determined. Different data, different owner, so they get different entry points in my API."

"For the nouns, I'd model Elevator, ElevatorSystem and SchedulingStrategy. I would deliberately **not** create a Floor class – a floor has no state and no behaviour, it's an integer. If floors later gain properties like access control, I'd promote it then. I'm not going to build an abstraction for a requirement that doesn't exist."

"Each elevator is a state machine: idle, moving up, moving down, doors open, and maintenance. I'd make doors-open a genuine **state** rather than a boolean, because the rule 'the car must not move while doors are open' is then enforced by the structure instead of by remembering to check a flag everywhere."

"For scheduling I'd reject first-come-first-served immediately – it makes the car ping-pong across the building. Nearest-first is better but suffers **starvation**: while there's steady traffic on floors three to five, someone waiting on fifteen may never be served. So I'd use LOOK, the algorithm real lifts use – continue in the current direction until there are no more stops that way, then reverse. That bounds everyone's wait."

"Scheduling lives behind a strategy interface, because this is exactly the piece I expect to change. Today it's LOOK with a nearest-car assignment; tomorrow it's energy-optimised, or predictive based on time of day. I want that to be a new class, not an edit."

"My cost function for assigning a hall call prefers an elevator that's already moving toward that floor **in the same direction**, treats idle elevators as simple distance, and heavily penalises one that would have to reverse. Full elevators are excluded entirely."

"If you asked me to add an express elevator serving only some floors, that's a subclass with a servable-floor set. But I'd flag something: a subclass that **accepts less** than its parent is a Liskov substitution risk. Rather than have it throw when given an unserviceable floor, I'd add an explicit capability check on the base class that the scheduler consults, so the constraint is expressed in the interface rather than in exceptions."

9. Follow-ups

అడిగేది	జవాబు
"Capacity ని ఎలా track చేస్తారు?"	Weight sensor / door counter నుంచి load . Full అయితే strategy ఆ elevator ని skip చేస్తుంది, కానీ లోపలి car calls ఇంకా పని చేస్తాయి
"Peak hours (ఉదయం అందరూ పైకి)?"	Zoning – elevators ని floor groups కి కేటాయించడం. లేదా idle elevators ని ground floor కి తిరిగి పంపడం (parking policy). ఇది strategy మార్పు మాత్రమే
"Emergency / fire mode?"	ఒక ప్రత్యేక state. అన్ని stops రద్దు, ground floor కి, door తెరిచి ఉంచడం. Manual reset తప్ప బయటికి రాదు
"Thread safety?"	Real system lo hall calls concurrent గా వస్తాయి. Elevator యొక్క stops set ని lock చేయడం, లేదా ఒక్కో elevator కి ఒక single-threaded actor/event loop – అదే మంచిది , lock లేకుండా serialise అవుతుంది
"Display board?"	Observer pattern – elevator state మారితే subscribers కి notify. Elevator display గురించి తెలుసుకోవసరం లేదు
"Wait time ని ఎలా కొలుస్తారు?"	Hall call వచ్చిన time నుంచి door open అయ్యే time వరకు. దీన్ని metric గా emit చేసి strategies ని పోల్చడం – ఇది testable design యొక్క లాభం

Design Movie Ticket Booking (BookMyShow)

Difficulty **Medium-Hard** · Frequency **అత్యధికం (Indian companies)** · నేర్పే concepts: seat locking, TTL holds, entity modelling

1. The Ask

"Design the seat booking flow for a cinema. A user picks seats, pays, and gets a ticket. Two users must never get the same seat."

ఈ **PROBLEM LO** చాలా మంది చేసే మౌలిక తప్పు

Seat అనే ఒక్క class రాసి, దానిలో isBooked అనే flag పెట్టడం. ఇది తప్పు, ఎందుకంటే – ఒక భౌతిక seat (A5) ఒక screen కి చెందినది, అది శాశ్వతం. కానీ "ఆ seat బుక్ అయిందా?" అనేది ఒక్కో show కి వేరుగా ఉంటుంది. ఉదయం show lo A5 బుక్, సాయంత్రం show lo ఖాళీ.

సరైన నమూనా: Seat = భౌతిక seat (screen కి చెందినది) · Show = ఆ seat యొక్క show-specific status. ఈ ఒక్క విభజన చెప్తే మీరు domain ని సరిగ్గా చదివారని అర్థమవుతుంది.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Seat selection ఉందా, general admission నా?	Numbered seats అంటే ప్రతి seat ఒక contended resource	Numbered
Payment ఎంతసేపు? Seat ని ఎంతసేపు పట్టుకోవాలి?	Lock TTL	10 నిమిషాలు
Seat types వేరే rates?	Pricing strategy	అవును
Group booking (5 కలిసి)?	All-or-nothing lock	అవును
Distributed system నా single server నా?	Lock provider interface ని ఎలా design చేయాలి	ఇప్పుడు single, తర్వాత distributed
Cancellation?	Seat తిరిగి inventory కి	అవును

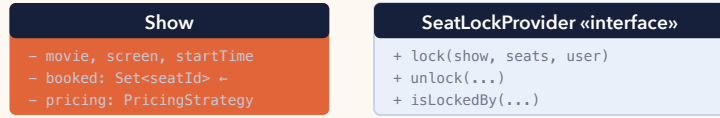
3. Nouns → Classes

ENTITY MODEL · భౌతికమైనవి VS SHOW-SPECIFIC

శాశ్వత / భౌతిక ENTITIES · ఒకసారి సృష్టించి పలు shows లో తిరిగి వాడేవి



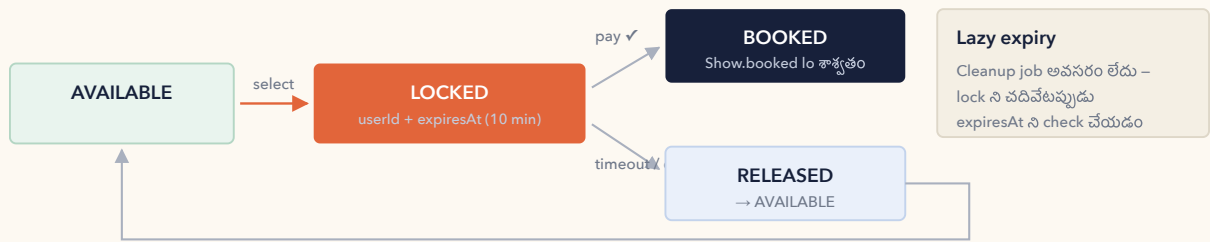
SHOW-SPECIFIC STATE · ఒక్క show కి వేరుగా



ఇక్కడ కీలకం: "seat బుక్ అయిందా" అనేది Seat లో కాదు, Show లో ఉంది
Seat ఒక భౌతిక వస్తువు · ఒకే Seat ని రోజుకి 5 shows వాడతాయి, ప్రతిదానికి వేరే availability

4. Deep Dive – Seat Locking

SEAT LIFECYCLE · LOCK అనేది TTL ఉన్న CLAIM



LOCK ని ఒక INTERFACE గా ఎందుకు చేయాలి

"ఇప్పుడు నేను InMemorySeatLockProvider రాస్తాను – interview లో అది చాలు. కానీ దాన్ని interface వెనక పెడతాను, ఎందుకంటే production లో 10 servers ఉంటే in-memory lock పనికిరాదు. అప్పుడు RedisSeatLockProvider (SET NX EX) లేదా DatabaseSeatLockProvider (conditional update) రాసి, ఒక్క line మార్పి swap చేస్తాను. ఈ ఒక్క ముందుచూపు – 'ఇది distributed అవుతుందని నాకు తెలుసు' – interviewer కి బలమైన signal."

5. Code

```
const SeatType = Object.freeze({ REGULAR: 'REGULAR', PREMIUM: 'PREMIUM', RECLINER: 'RECLINER' });
const SeatStatus = Object.freeze({ AVAILABLE: 'AVAILABLE', LOCKED: 'LOCKED', BOOKED: 'BOOKED' });

class Seat {
  // భౌతిక seat – screen కి చెందినది, show కి కాదు
  constructor(id, row, number, type = SeatType.REGULAR) {
    Object.assign(this, { id, row, number, type });
  }
}

class Screen { constructor(id, seats) { this.id = id; this.seats = seats; } }
```

```
class Movie { constructor(id, title, durationMin) { Object.assign(this, { id, title, durationMin }); } }
```

```
class SeatLockProvider { // ఇది ఒక interface - implementation
  swap అవుతుంది
  lock(showId, seatIds, userId) { throw new Error('abstract'); }
  unlock(showId, seatIds, userId) { throw new Error('abstract'); }
  isLocked(showId, seatId) { throw new Error('abstract'); }
  isLockedBy(showId, seatId, userId) { throw new Error('abstract'); }
}

class InMemorySeatLockProvider extends SeatLockProvider {
  constructor(ttlMs = 10 * 60 * 1000, now = () => Date.now()) {
    super();
    this.ttlMs = ttlMs; this.now = now;
    this.locks = new Map(); // "showId:seatId" → { userId,
    expiresAt }
  }
  #key(showId, seatId) { return `${showId}:${seatId}`; }
  #live(lock) { return Boolean(lock) && lock.expiresAt > this.now(); } // lazy
  expiry

  lock(showId, seatIds, userId) {
    // ALL-OR-NOTHING: ముందు అన్నిటిని check, తర్వాత ఏదైనా lock చేయడం.
    for (const s of seatIds) {
      const held = this.locks.get(this.#key(showId, s));
      if (this.#live(held) && held.userId !== userId) throw new
      Error(`SEAT_LOCKED:${s}`);
    }
    const expiresAt = this.now() + this.ttlMs;
    for (const s of seatIds) this.locks.set(this.#key(showId, s), { userId, expiresAt
  });
  }
  unlock(showId, seatIds, userId) {
    for (const s of seatIds) {
      const k = this.#key(showId, s);
      const held = this.locks.get(k);
      if (held && held.userId === userId) this.locks.delete(k); // ఇతరుల lock ని
      తీయకూడదు
    }
  }
  isLocked(showId, seatId) { return this.#live(this.locks.get(this.#key(showId,
  seatId))); }
  isLockedBy(showId, seatId, userId) {
    const held = this.locks.get(this.#key(showId, seatId));
    return this.#live(held) && held.userId === userId;
  }
}
```

```
class PricingStrategy { price(show, seatIds) { throw new Error('abstract'); } }
```

```
class TypeBasedPricing extends PricingStrategy {
  constructor(ratesByType) { super(); this.rates = ratesByType; }
  price(show, seatIds) {
    const byId = new Map(show.screen.seats.map((s) => [s.id, s]));
    return seatIds.reduce((sum, id) => sum + this.rates[byId.get(id).type], 0);
  }
}
```

```

    }
}

class Show {
  constructor(id, movie, screen, startTime, pricing) {
    Object.assign(this, { id, movie, screen, startTime, pricing });
    this.booked = new Set(); // ← show-specific state ఇక్కడ, Seat lo
    కాదు
  }
  statusOf(seatId, locks) {
    if (this.booked.has(seatId)) return SeatStatus.BOOKED;
    if (locks.isLocked(this.id, seatId)) return SeatStatus.LOCKED;
    return SeatStatus.AVAILABLE;
  }
  availableSeats(locks) {
    return this.screen.seats.filter((s) => this.statusOf(s.id, locks) ===
    SeatStatus.AVAILABLE);
  }
}

```

```

class BookingService {
  constructor(lockProvider) { this.locks = lockProvider; this.bookings = new Map();
  this.seq = 0; }

  // అడుగు 1 – seats ని పట్టుకోవడం (ఇంకా booking కాదు)
  selectSeats(show, seatIds, userId) {
    for (const s of seatIds) if (show.booked.has(s)) throw new
    Error(`SEAT_BOOKED:${s}`);
    this.locks.lock(show.id, seatIds, userId);
    return { showId: show.id, seatIds, userId, amount: show.pricing.price(show,
    seatIds) };
  }

  // అడుగు 2 – payment విజయవంతమయ్యేక
  confirm(show, seatIds, userId) {
    for (const s of seatIds) {
      if (!this.locks.isLockedBy(show.id, s, userId)) throw new
      Error('LOCK_EXPIRED');
    }
    for (const s of seatIds) show.booked.add(s);
    this.locks.unlock(show.id, seatIds, userId);
    const booking = { id: `B${++this.seq}`, showId: show.id, seatIds, userId };
    this.bookings.set(booking.id, booking);
    return booking;
  }

  release(show, seatIds, userId) { this.locks.unlock(show.id, seatIds, userId); }
}

```

```

// ---- వాడుక ----
const screen = new Screen('SCR-1', [
  new Seat('A1', 'A', 1),
  new Seat('A2', 'A', 2, SeatType.PREMIUM),
  new Seat('A3', 'A', 3),
]);
const show = new Show('SH-1', new Movie('M1', 'Baahubali', 170), screen, '2026-09-

```

```

10T18:00',
  new TypeBasedPricing({ REGULAR: 200, PREMIUM: 350, RECLINER: 500 }));

const locks = new InMemorySeatLockProvider();
const svc = new BookingService(locks);

console.log(svc.selectSeats(show, ['A1', 'A2'], 'u1').amount); // 550 (200 + 350)
try { svc.selectSeats(show, ['A2', 'A3'], 'u2'); }
catch (e) { console.log(e.message); } // SEAT_LOCKED:A2
console.log(show.statusOf('A3', locks)); // AVAILABLE ← A3
lock కాలేదు
console.log(svc.confirm(show, ['A1', 'A2'], 'u1').id); // B1
console.log(show.statusOf('A1', locks)); // BOOKED
console.log(svc.selectSeats(show, ['A3'], 'u2').amount); // 200

```

ఈ CODE LO మూడు సూక్ష్మతలు

- 1. All-or-nothing lock** – 5 seats అడిగితే, ఒక్కటి పోయినా ఏదీ lock చేయకూడదు. అందుకే ముందు అన్నిటిని check, తర్వాత lock. లేకపోతే user 3 seats పట్టుకుని ఇరుక్కుంటాడు.
- 2. unlock lo ownership check** – వేరేవాళ్ళ lock ని తీయకూడదు. ఇది లేకపోతే ఒక bug (లేదా దాడి) ద్వారా ఎవరైనా ఇతరుల seats ని విడిపించచ్చు.
- 3. now ని inject చేయడం** – `Date.now` ని hardcode చేయలేదు. దీంతో test lo సమయాన్ని ముందుకీ జరిపి **lock expiry ని నిజంగా test చేయొచ్చు**. Testability ని design lo చేర్చడం.

6. Extensibility Test

Interviewer: "Add weekend surge pricing, and a distributed lock for multiple servers."

```

// 1) Pricing – కొత్త strategy, ఉన్నదాన్ని చుడుతుంది (Decorator)
class SurgePricing extends PricingStrategy {
  constructor(base, multiplier = 1.5) { super(); this.base = base; this.multiplier = multiplier; }
  price(show, seatIds) {
    const day = new Date(show.startTime).getDay();
    const surge = (day === 0 || day === 6) ? this.multiplier : 1;
    return Math.round(this.base.price(show, seatIds) * surge);
  }
}

// 2) Distributed lock – అదే interface, వేరే implementation.
// BookingService lo ఒక్క line కూడా మారదు.
class RedisSeatLockProvider extends SeatLockProvider {
  constructor(redis, ttlSec = 600) { super(); this.redis = redis; this.ttl = ttlSec; }
  async lock(showId, seatIds, userId) {
    // నిజ implementation: అన్ని keys ని ఒకే Lua script lo SET NX EX –
    // ఏ ఒక్కటి fail అయినా అప్పటివరకు పెట్టినవి తీసేసి throw చేయడం (atomicity).
  }
}

```

7. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Strategy	PricingStrategy	Rates, surge, offers – ఇవి ఖచ్చితంగా మారతాయి
Decorator	SurgePricing(base)	Surge, discount, tax ని పొరలుగా పేర్చేచ్చు
Dependency Inversion	SeatLockProvider interface	In-memory → Redis మార్పు BookingService కి తెలియకుండా
State (సరళమైనది)	Seat status derived	Status ని store చేయలేదు – booked set + lock నుంచి derive చేస్తున్నాం. రెండు చోట్ల state ఉంటే అవి divergent అవుతాయి

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me clarify a few things: numbered seats or general admission, how long do we hold a seat during payment, are there different seat types and prices, and is this a single server or distributed? I'll assume numbered seats, a ten-minute hold, type-based pricing, and I'll design for distribution even if I implement it in memory."

"The modelling decision I want to get right first is where 'is this seat taken' lives. It's tempting to put a booked flag on Seat, but that's wrong: a seat is a **physical** object belonging to a screen, and the same seat is used by five different shows in a day with different availability in each. So Seat holds identity and type, and the **Show** holds which of its seats are booked."

"I'd also derive seat status rather than store it. A seat is booked if it's in the show's booked set, locked if the lock provider says so, otherwise available. If I stored a status field **and** had locks, those two would eventually disagree – and reconciling duplicated state is a permanent source of bugs."

"Booking is two phases. Selecting seats takes a **lock** with a TTL – not a permanent claim, because payment takes minutes and the user may abandon it. Confirming after payment moves the seats into booked and releases the locks."

"Three details in the locking that I'd write carefully. First, it's **all or nothing**: for a five-seat group I check every seat before locking any, otherwise a user ends up holding three seats they can't use. Second, unlocking verifies ownership – you can't release someone else's lock, which is both a correctness and a security concern. Third, expiry is **lazy**: I check the timestamp when reading the lock rather than relying on a cleanup job, so if that job is delayed, seats still free up correctly."

"I'd put locking behind an interface even though I'm implementing it in memory. With more than one server, in-memory locks are worthless – you need Redis with SET NX EX, or a conditional database update. Because it's an interface, swapping that is one line at construction and BookingService never knows."

"One thing I'd do for testability: inject the clock rather than calling Date.now directly. That's what makes lock expiry actually testable – I can advance time in a test instead of sleeping for ten minutes."

"Pricing goes behind a strategy too, and surge pricing then becomes a decorator wrapping the base strategy, so weekend pricing, discounts and taxes can stack without any of them knowing about each other."

8. Follow-ups

అడిగేది	జవాబు
"Payment fail అయితే?"	<code>release()</code> – lock ని వెంటనే వదిలేయడం. Payment status తెలియకపోతే (timeout) మాత్రం release చేయకూడదు – lock ని పొడిగించి PSP ని అడగాలి
"10 servers ఉంటే in-memory lock ఎందుకు పనికీరాదు?"	ప్రతి server కీ తన సొంత Map. Server 1 lo lock ఉంది, server 2 కీ తెలియదు → double booking. Shared store తప్పనిసరి

అడిగేది	జవాబు
"Seat map ని ఎలా వేగంగా చూపుతారు?"	<code>availableSeats()</code> $O(n)$ – 300 seats కి ఫర్వాలేదు. 50,000 seats stadium కి bitmap + cache
"Cancellation & refund?"	<code>show.booked.delete(seatId)</code> + refund transaction. కానీ show సమయానికి దగ్గరైతే cancellation policy – ఒక <code>CancellationPolicy</code> strategy
"ఒకే user 100 seats బుక్ చేస్తే?"	Per-user seat limit – booking service lo ఒక validation rule. Scalpers ని అపడానికి
"Show ముగిశాక data ఏం చేస్తారు?"	<code>Show</code> object ని archive. <code>booked</code> set చిన్నది కాబట్టి history కి store చేయడం చౌక

Design Splitwise

Difficulty **Medium** • Frequency ఎక్కువ • నేర్పే concepts: split strategies, balance netting, debt simplification

1. The Ask

"Design Splitwise. A group of friends adds expenses, splits them in different ways, and the app tells everyone who owes whom."

భాగం 1 • Splitting

Equal, exact amounts, percentages, shares – ఇవి వేర్వేరు **algorithms**. Strategy pattern కి పాఠ్యపుస్తక ఉదాహరణ.

భాగం 2 • Balance

"ఎవరు ఎవరికి ఎంత" – ఇది ఒక **directed graph**. దాన్ని ఎలా store చేయాలి అనేదే అసలు design ప్రశ్న.

భాగం 3 • Simplification

$A \rightarrow B, B \rightarrow C$ ని $A \rightarrow C$ గా కుదించడం. ఇది ఒక **algorithm question** – ఇక్కడే candidates వేరుపడతారు.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఎవి split types?	Strategy interface ని ఇది నిర్ణయిస్తుంది	Equal, exact, percentage
Groups ఉన్నాయా, లేక వ్యక్తుల మధ్యనా?	Group అంటే ఒక scope, కానీ balance వ్యక్తుల మధ్య	రెండూ
Simplify debts feature ఉందా?	ఇదే ఈ problem lo algorithmic భాగం	అవును
Multi-currency?	ఉంటే ప్రత్యేక conversion + rate lock	ఇప్పుడు ఒక్క currency
Settlement (డబ్బు తిరిగి ఇవ్వడం) ఎలా record?	ఒక reverse expense లాంటిది	Explicit settle operation
Expense edit / delete?	Balance ని తిరిగి లెక్కించాలి	అవును (follow-up)

MONEY ని ఎప్పుడూ FLOAT LO ఉంచొద్దు

$0.1 + 0.2 == 0.30000000000000004$. ₹1000 ని ముగ్గురికి పంచితే float lo ₹0.01 తప్పిపోతుంది, అది కాలక్రమేణా పేరుకుంటుంది. అందుకే **అంతా paisa lo integer గా** store చేస్తాను, display lo మాత్రం $\div 100$. పైగా **remainder ని explicit గా పంచుతాను** – ₹1000 $\div 3 = 33334 + 33333 + 33333$ paisa. ఈ ఒక్క వివరం చెప్తే మీరు financial code రాశారని అర్థమవుతుంది.

3. Nouns → Classes

Class	బాధ్యత
User	id, name
Expense	ఎవరు కట్టారు, ఎంత, ఎవరు పాల్గొన్నారు, ఏ strategy
SplitStrategy	మొత్తాన్ని participants మధ్య పంచడం – pluggable
BalanceSheet	"ఎవరు ఎవరికి ఎంత" – netting + simplification
ExpenseService	Expense ని తీసుకుని balance sheet ని update చేయడం
Split	⚠️ ఒక్కో participant కి ఒక object అవసరం లేదు – Map<userId, amount> చాలు

4. Deep Dive – Balance ని ఎలా store చేయాలి

మూడు మార్గాలు • ఏది ఎందుకు

A • ప్రతి expense ని store చేసి అడిగినప్పుడు లెక్కించడం

Balance = అన్ని expenses మీద ప్రతిసారీ loop
✓ చరిత్ర పూర్తిగా ఉంది
X 10,000 expenses తర్వాత నెమ్మది

B • ఒక్కో user కి net

A: +1250 • B: -250 • C: -1000
✓ చాలా చిన్నది, O(1) lookup
X "నేను ఎవరికి ఇవ్వాలి" అనేది తెలియదు – Splitwise UI సరిగ్గా అదే చూపిస్తుంది

C • జతల వారీ (pairwise) ✓

B → A : 250
C → A : 1000
✓ "B, A కి ₹250 ఇవ్వాలి" అని నేరుగా చెప్పగలం
✓ Net ని దీని నుంచి derive చేయొచ్చు

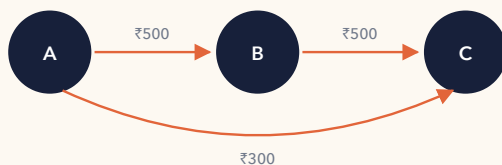
కీలకం: జతల మధ్య ఎప్పుడూ NET చేయాలి

B, A కి ₹1000 ఇవ్వాలి. తర్వాత A, B కి ₹750 ఇవ్వాలి. రెండు entries ఉంచకూడదు – వెంటనే net చేసి "B → A : 250" ఒక్కటే ఉంచాలి. లేకపోతే ledger ఉబ్బిపోతుంది, UI గందరగోళం.

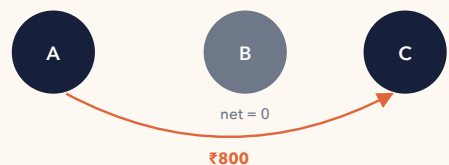
5. Deep Dive – Debt Simplification

3 TRANSACTIONS → 1 TRANSACTION

మొదట • 3 లావాదేవీలు



తర్వాత • 1 లావాదేవీ



Greedy algorithm: ప్రతి user యొక్క net ని లెక్కించి, అతి పెద్ద debtor ని అతి పెద్ద creditor తో జత చేయడం
ప్రతి అడుగులో కనీసం ఒక వ్యక్తి settle అవుతాడు → గరిష్టంగా (n-1) transactions

ఇది **optimal కాదు** – కనిష్ట transactions కనుక్కోవడం NP-hard (subset-sum కి తగ్గుతుంది). కానీ greedy ప్రతిసారీ కనీసం ఒకరిని settle చేస్తుంది కాబట్టి n-1 కి పరిమితం. Splitwise ఇదే వాడుతుంది. ఈ "optimal కాదు కానీ మంచిది, ఎందుకంటే optimal NP-hard" అనే వాక్యం interview lo చాలా బలమైనది.

6. Code

```
// అంతా PAISE లో - integers. Float money bug ని మూలంలోనే తుంచేయడం.
const rupees = (paise) => (paise / 100).toFixed(2);

class SplitStrategy {
  // Map<userId, paise> ని తిరిగి ఇవ్వాలి. మొత్తం ఎప్పుడూ total కి సమానం.
  compute(totalPaise, participants, meta) { throw new Error('abstract'); }
}

class EqualSplit extends SplitStrategy {
  compute(total, participants) {
    const base = Math.floor(total / participants.length);
    let remainder = total - base * participants.length; // మిగిలిన paise
    return new Map(participants.map((u) => {
      const extra = remainder > 0 ? 1 : 0; // మొదటివారికి 1 paisa
      remainder -= extra;
      return [u, base + extra];
    }));
  }
}

class ExactSplit extends SplitStrategy {
  compute(total, participants, amounts) { // amounts: { userId:
    paise }
    const sum = participants.reduce((s, u) => s + (amounts[u] ?? 0), 0);
    if (sum !== total) throw new Error(`SPLIT_MISMATCH: ${sum} != ${total}`);
    return new Map(participants.map((u) => [u, amounts[u]]));
  }
}

class PercentSplit extends SplitStrategy {
  compute(total, participants, percents) { // percents: { userId:
    number }
    const sum = participants.reduce((s, u) => s + (percents[u] ?? 0), 0);
    if (Math.abs(sum - 100) > 1e-9) throw new Error(`PERCENT_MISMATCH: ${sum} !=
    100`);
    const shares = new Map(participants.map((u) => [u, Math.floor(total * percents[u]
    / 100)]));
    // rounding వల్ల మిగిలినది మొదటివారికి - మొత్తం ఎప్పుడూ total కి సమానం కావాలి
    const assigned = [...shares.values()].reduce((a, b) => a + b, 0);
    shares.set(participants[0], shares.get(participants[0]) + (total - assigned));
    return shares;
  }
}
```

```
class BalanceSheet {
  constructor() { this.ledger = new Map(); } // debtor → Map(creditor → paise)

  #get(a, b) { return this.ledger.get(a)?.get(b) ?? 0; }
  #set(a, b, v) {
    if (!this.ledger.has(a)) this.ledger.set(a, new Map());
    if (v <= 0) this.ledger.get(a).delete(b); else this.ledger.get(a).set(b, v);
  }
}
```

```

// ఇక్కడ netting జరుగుతుంది - ఎప్పుడూ ఒక జతకి ఒకే దిక్కు మాత్రమే ఉంటుంది.
addDebt(debtor, creditor, amount) {
  if (debtor === creditor || amount <= 0) return;
  const reverse = this.#get(creditor, debtor);
  if (reverse >= amount) { this.#set(creditor, debtor, reverse - amount); return; }
  this.#set(creditor, debtor, 0);
  this.#set(debtor, creditor, this.#get(debtor, creditor) + (amount - reverse));
}

debts() {
  const out = [];
  for (const [debtor, m] of this.ledger)
    for (const [creditor, amount] of m) out.push({ from: debtor, to: creditor,
amount });
  return out;
}

netOf(userId) {
  let net = 0;
  for (const [debtor, m] of this.ledger)
    for (const [creditor, amount] of m) {
      if (debtor === userId) net -= amount;
      if (creditor === userId) net += amount;
    }
  return net;
}

// Greedy: అతి పెద్ద debtor ↔ అతి పెద్ద creditor. గరిష్ఠంగా n-1 transactions.
simplify() {
  const net = new Map();
  for (const { from, to, amount } of this.debts()) {
    net.set(from, (net.get(from) ?? 0) - amount);
    net.set(to, (net.get(to) ?? 0) + amount);
  }
  const debtors = [...net].filter(([, v]) => v < 0).map(([u, v]) => [u, -
v]).sort((a, b) => b[1] - a[1]);
  const creditors = [...net].filter(([, v]) => v > 0).sort((a, b) => b[1] - a[1]);

  const txns = [];
  let i = 0, j = 0;
  while (i < debtors.length && j < creditors.length) {
    const amount = Math.min(debtors[i][1], creditors[j][1]);
    txns.push({ from: debtors[i][0], to: creditors[j][0], amount });
    debtors[i][1] -= amount; creditors[j][1] -= amount;
    if (debtors[i][1] === 0) i++;
    if (creditors[j][1] === 0) j++;
  }
  return txns;
}
}

```

```

class ExpenseService {
  constructor(sheet) { this.sheet = sheet; this.expenses = []; }

  addExpense({ id, paidBy, amount, participants, strategy, meta }) {
    const shares = strategy.compute(amount, participants, meta);

```

```

    for (const [user, share] of shares) {
        if (user !== paidBy) this.sheet.addDebt(user, paidBy, share);    // paidBy కి బాకీ
    }
    this.expenses.push({ id, paidBy, amount, shares });
    return shares;
}

// డబ్బు తిరిగి ఇవ్వడం = వ్యతిరేక దిశలో debt → netting అదే చూసుకుంటుంది
settle(from, to, amount) { this.sheet.addDebt(to, from, amount); }
}

```

```

// ---- వాడుక ----
const sheet = new BalanceSheet();
const svc = new ExpenseService(sheet);

// A ₹3000 dinner కట్టాడు, ముగ్గురి మధ్య సమానంగా
svc.addExpense({ id: 'e1', paidBy: 'A', amount: 300000,
    participants: ['A', 'B', 'C'], strategy: new EqualSplit() });
console.log(sheet.debts());
// [ { from: 'B', to: 'A', amount: 100000 }, { from: 'C', to: 'A', amount: 100000 } ]

// B ₹1500 cab కట్టాడు, A మరియు B మధ్య
svc.addExpense({ id: 'e2', paidBy: 'B', amount: 150000,
    participants: ['A', 'B'], strategy: new EqualSplit() });
console.log(sheet.debts());
// [ { from: 'B', to: 'A', amount: 25000 }, { from: 'C', to: 'A', amount: 100000 } ]
// ↑ 1,00,000 - 75,000 = 25,000 గా NET అయింది. రెండు entries లేవు.

console.log(rupees(sheet.netOf('A')), rupees(sheet.netOf('C')));    // 1250.00
-1000.00

```

```

// ---- simplification demo ----
const s2 = new BalanceSheet();
s2.addDebt('A', 'B', 50000);    // A → B ₹500
s2.addDebt('B', 'C', 50000);    // B → C ₹500
s2.addDebt('A', 'C', 30000);    // A → C ₹300
console.log(s2.debts().length);    // 3
console.log(s2.simplify());
// [ { from: 'A', to: 'C', amount: 80000 } ]    ← 3 → 1. B పూర్తిగా బయటికి వెళ్ళాడు.

```

7. Extensibility Test

Interviewer: "Add split by shares – 'A gets 2 shares, B gets 1'."

```

// కొత్త class. ఉన్న ఏ class నీ ముట్టుకోలేదు.
class ShareSplit extends SplitStrategy {
    compute(total, participants, shares) {    // shares: { userId: number }
        const totalShares = participants.reduce((s, u) => s + shares[u], 0);
        const result = new Map(participants.map((u) => [u, Math.floor(total * shares[u] /
totalShares)]));
        const assigned = [...result.values()].reduce((a, b) => a + b, 0);
        result.set(participants[0], result.get(participants[0]) + (total - assigned));
    }
}

```

```

        return result;
    }
}

```

ఇక్కడ ఒక **PATTERN** గమనించండి

నాలుగు strategies lo మూడింటిలో **ఒకే rounding logic** repeat అయింది ("మిగిలిన paise ని మొదటివారికి ఇవ్వడం"). Interview lo దీన్ని మేరే గుర్తించి – "నేను దీన్ని base class lo ఒక protected helper `#distributeRemainder()` గా తీస్తాను" అని చెప్తే, అది **DRY ని గుర్తించే కన్ను** మీకు ఉందని చూపిస్తుంది. Duplication ని interviewer చూపించే ముందు మేరే చూపించడం విలువైనది.

8. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Strategy	<code>SplitStrategy</code>	Split రకాలు ఖచ్చితంగా పేరుగుతాయి (shares, adjustment, itemised)
Template Method (సూచించినది)	Rounding remainder logic	అన్ని strategies lo ఒకే skeleton
⚠️ Observer వద్దు	–	"Balance మారితే notify చేయాలి" అనిపిస్తుంది. కానీ ఇప్పుడు అవసరం లేదు – అది notification service పని

INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me confirm the split types we support, whether groups exist, and – the one that changes the difficulty – whether we need debt simplification. I'll assume equal, exact and percentage splits, groups as a scope, and simplification included."

"Before modelling I want to fix one thing: all money is stored as **integer paise**, never floats. Splitting a thousand rupees three ways in floating point loses a paisa, and those losses accumulate across thousands of expenses until the books don't balance. I'd also distribute the remainder explicitly – thirty-three thousand three hundred and thirty-four plus two lots of thirty-three thousand three hundred and thirty-three – so the shares always sum to exactly the total."

"Splitting is a textbook Strategy: equal, exact and percentage are genuinely different algorithms over the same inputs, and I'm confident more will be added. Each returns a map from user to amount, and each validates its own inputs – exact splits must sum to the total, percentages to a hundred."

"The more interesting decision is how to store balances. I could recompute from the expense log every time, which is accurate but gets slow. I could store one net number per person, which is compact but can't answer 'who do I pay?' – and that's the main thing the app shows. So I store it **pairwise**: a directed ledger of who owes whom."

"The key invariant is that I **net on write**. If B owes A a thousand and then A owes B seven-fifty, I don't keep two rows – I collapse them into B owes A two-fifty. Without that the ledger grows unbounded and the UI shows contradictory debts."

"For simplification, I compute each person's net position, then greedily match the largest debtor against the largest creditor. Each step fully settles at least one person, so it terminates in at most $n-1$ transactions. I'd be upfront that this is **not provably minimal** – finding the true minimum reduces to subset-sum and is NP-hard. Greedy is what Splitwise actually ships, and the bound is good enough."

"Settlement – actually paying someone back – I model as a debt in the opposite direction rather than a special case. The netting logic then handles it automatically, which means one code path instead of two."

"One thing I'd flag on my own code: three of my split strategies repeat the same remainder-distribution logic. I'd pull that into a shared helper on the base class. It's better to spot your own duplication than to have the interviewer point at it."

9. Follow-ups

అడిగేది	జవాబు
"Expense edit / delete ఎలా?"	Balance ని పాత expense ప్రకారం reverse చేసి, కొత్తది apply చేయడం. అందుకే expenses ని ఎప్పుడూ store చేయాలి – balance sheet ఒక derived cache మాత్రమే

అడిగేది	జవాబు
"Multi-currency?"	Expense lo currency + ఆ క్షణపు rate ని store చేయడం. Balance ని ఒక base currency lo. Rate ని expense సమయంలోనే lock చేయాలి – తర్వాత మారితే పాత లెక్క మారకూడదు
"Group balance vs overall balance?"	Ledger ని (groupId, debtor, creditor) గా key చేయడం. Overall = అన్ని groups మీద sum. Splitwise రెండూ చూపిస్తుంది
"Concurrency – ఇద్దరు ఒకేసారి expense add చేస్తే?"	addDebt read-modify-write. DB lo అయితే row lock లేదా optimistic version. Balance ని expenses నుంచి తిరిగి లెక్కించగలం కాబట్టి recovery సులభం
"Simplify ఎప్పుడు apply చేయాలి?"	ఎప్పుడూ automatic గా కాదు – user "who paid what" చూడాలనుకుంటాడు. Simplify ఒక explicit action, మరియు దాన్ని apply చేసినప్పుడు ledger ని replace చేయాలి
"1 million users ఉన్న groups?"	Splitwise design ఇందుకు కాదు. అప్పుడు pairwise ledger $O(n^2)$ – event-sourced net balance + on-demand pairwise computation కి మారాలి

Design a Vending Machine

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: State pattern – దీనికి ఇదే పాఠ్యపుస్తక problem

1. The Ask

"Design a vending machine. The user selects an item, inserts coins, and gets the item plus change."

ఈ PROBLEM ఎందుకు ఉంది

ఇది వేరే problem కాదు – **State pattern ని మకు తెలుసా** అని చూసే problem. కాబట్టి if-else తో పరిష్కరించి "పని చేస్తోంది కదా" అంటే – technically సరైనా, interview lo ఓడిపోతారు. Interviewer వెతుకుతున్నది ఒక్కటే: **object యొక్క ప్రవర్తన దాని state ని బట్టి మారుతున్నప్పుడు, ఆ state ని ఒక object గా చేయడం.**

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Change ఇవ్వాలా?	ఇస్తే coin inventory + change algorithm	అవును
Cancel / refund ఉందా?	ఒక అదనపు transition	అవును
Sold out అయితే?	ఒక state నా, ఒక check నా	Check (item-level)
Card / UPI payment?	Payment ని strategy గా వేరు చేయాలా	ఇప్పుడు coins, తర్వాత extension
Multiple items ఒకేసారి?	ఇది కొనుగోలు flow ని మారుస్తుంది	ఒకేసారి ఒక్కటి
Admin refill / collect?	అదనపు states	అవును

3. Deep Dive – ఎందుకు if-else విఫలమవుతుంది

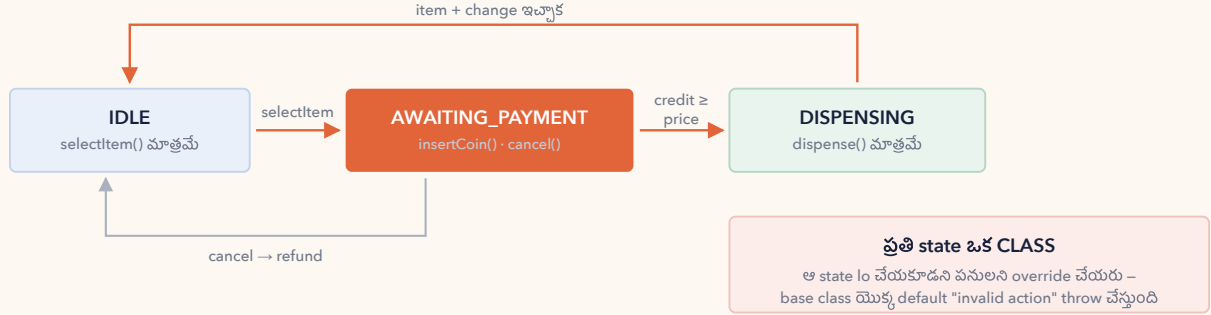
```
// ✗ ఇలా చేయకండి – ప్రతి method lo ఇదే గందరగోళం repeat అవుతుంది
insertCoin(coin) {
  if (this.state === 'IDLE')          throw new Error('ముందు item ఎంచుకోండి');
  else if (this.state === 'DISPENSING') throw new Error('ఇప్పుడు వద్దు');
  else if (this.state === 'MAINTENANCE') throw new Error('పని చేయడం లేదు');
  else if (this.state === 'AWAITING') { /* అసలు logic ఇక్కడ */ }
}
```

సమస్య ఏమిటంటే

1. కొత్త state (ఉదా. MAINTENANCE) జోడిస్తే – ప్రతి method ని edit చేయాలి. Method 6 ఉంటే 6 చోట్లు.
2. ఒక్క method lo మర్చిపోతే – silent bug. Compiler సహాయం చేయదు.
3. "ఏ state lo ఏం చేయొచ్చు" అనేది code అంతటా చెల్లాచెదురుగా ఉంటుంది. ఒకచోట చూసి అర్థం చేసుకోలేం.

4. State Pattern – పరిష్కారం

VENDING MACHINE • STATE TRANSITIONS



కీలకం: ప్రతి state class తనకీ చెల్లుబాటు అయ్యే actions ని మాత్రమే override చేస్తుంది. IdleState lo dispense() లేదు – కాబట్టి base class యొక్క "invalid action" version నడుస్తుంది. If-else లేదు, validation logic చెల్లాచెదురు కాదు.

5. Code

```
// Base state – డిఫాల్ట్ గా అన్ని actions చెల్లువు. చెల్లేవాటిని subclass override చేస్తుంది.
class VendingState {
    constructor(machine) { this.machine = machine; }
    get name() { return this.constructor.name; }
    #invalid(action) { throw new Error(`INVALID_ACTION: ${action} in ${this.name}`); }

    selectItem(code) { this.#invalid('selectItem'); }
    insertCoin(coin) { this.#invalid('insertCoin'); }
    dispense()      { this.#invalid('dispense'); }
    cancel()        { this.#invalid('cancel'); }
}

class IdleState extends VendingState {
    selectItem(code) {
        const slot = this.machine.inventory.get(code);
        if (!slot) throw new Error('NO_SUCH_ITEM');
        if (slot.qty === 0) throw new Error('OUT_OF_STOCK');
        this.machine.selected = code;
        this.machine.setState(new AwaitingPaymentState(this.machine));
    }
}

class AwaitingPaymentState extends VendingState {
    insertCoin(coin) {
        if (!this.machine.acceptedCoins.has(coin)) throw new
Error(`COIN_REJECTED:${coin}`);
```

```

        this.machine.credit += coin;
        if (this.machine.credit >= this.machine.priceOf(this.machine.selected)) {
            this.machine.setState(new DispensingState(this.machine));
        }
    }
    cancel() {
        const refund = this.machine.resetTransaction();
        this.machine.setState(new IdleState(this.machine));
        return refund;
    }
}

class DispensingState extends VendingState {
    dispense() {
        const code = this.machine.selected;
        const change = this.machine.credit - this.machine.priceOf(code);
        this.machine.inventory.get(code).qty -= 1;
        this.machine.resetTransaction();
        this.machine.setState(new IdleState(this.machine));
        return { item: code, change };
    }
}

```

```

class VendingMachine {
    constructor(inventory, acceptedCoins = [1, 2, 5, 10, 20]) {
        this.inventory = inventory; // Map: code → { name, price, qty }
        this.acceptedCoins = new Set(acceptedCoins);
        this.credit = 0;
        this.selected = null;
        this.state = new IdleState(this);
    }

    setState(state) { this.state = state; }
    get stateName() { return this.state.name; }
    priceOf(code) { return this.inventory.get(code).price; }
    resetTransaction() { const r = this.credit; this.credit = 0; this.selected = null;
return r; }

    // Public API – అంతా current state కి delegate అవుతుంది. ఇక్కడ ఒక్క if కూడా లేదు.
    selectItem(code) { return this.state.selectItem(code); }
    insertCoin(coin) { return this.state.insertCoin(coin); }
    dispense() { return this.state.dispense(); }
    cancel() { return this.state.cancel(); }
}

```

```

// ---- వాడుక ----
const vm = new VendingMachine(new Map([
    ['A1', { name: 'Chips', price: 25, qty: 2 }],
    ['B2', { name: 'Cola', price: 40, qty: 0 }],
]));

console.log(vm.stateName); // IdleState
try { vm.dispense(); } catch (e) { console.log(e.message); } // INVALID_ACTION:
dispense in IdleState
try { vm.selectItem('B2'); } catch (e) { console.log(e.message); } // OUT_OF_STOCK

```

```

vm.selectItem('A1');
console.log(vm.stateName); // AwaitingPaymentState
vm.insertCoin(10);
console.log(vm.stateName, vm.credit); // AwaitingPaymentState
10
vm.insertCoin(20);
console.log(vm.stateName); // DispensingState
console.log(vm.dispense()); // { item: 'A1', change:
5 }
console.log(vm.stateName, vm.inventory.get('A1').qty); // IdleState 1

```

ఈ CODE LO గమనించాల్సిన మూడు విషయాలు

1. **VendingMachine lo ఒక్క if (state === ...) కూడా లేదు.** అన్ని methods కీవలం delegate చేస్తాయి. ఇదే State pattern యొక్క ఫలితం.
2. **Invalid actions స్వయంచాలకంగా handle అవుతాయి** – base class default throw. IdleState lo dispense రాయనవసరం లేదు.
3. **State transition ని state క్లాస్ చేస్తుంది**, machine కాదు. అంటే "ఈ state నుంచి ఎక్కడికి వెళ్ళొచ్చు" అనేది ఆ state class lo, ఒకేచోట ఉంటుంది.

6. Extensibility Test

Interviewer: "Add a maintenance mode, and accept UPI payments."

```

// 1) కొత్త state – ఉన్న ఏ state class నీ ముట్టుకోలేదు.
class MaintenanceState extends VendingState {
  // ఏ user action నీ override చేయలేదు → అన్నీ ఆటోమేటిక్ గా INVALID_ACTION
  refill(code, qty) { this.machine.inventory.get(code).qty += qty; }
  exit() { this.machine.setState(new IdleState(this.machine)); }
}
// machine.setState(new MaintenanceState(machine)) – అంతే.

// 2) Payment ని strategy గా వేరు చేయడం
class PaymentMethod { collect(amount) { throw new Error('abstract'); } }
class CoinPayment extends PaymentMethod { /* credit పోగేయడం */ }
class UpiPayment extends PaymentMethod { /* QR చూపి callback కోసం వేచి ఉండటం */ }

```

MAINTENANCE STATE LO ఒక అందమైన విషయం

MaintenanceState lo నేను ఒక్క user action నీ override చేయలేదు. అంటే ఆ state lo selectItem , insertCoin , dispense – అన్నీ ఆటోమేటిక్ గా "INVALID_ACTION" throw చేస్తాయి. If-else approach lo ఇది చేయాలంటే 4 methods lo 4 కొత్త branches రాయాలి. ఇక్కడ సున్నా. ఇదే Open/Closed Principle నిజంగా పని చేస్తున్న రూపం.

7. Change Dispensing – ఒక దాచిన algorithm

Change ఇవ్వడం ఒక **coin change problem**. ₹5 change ఇవ్వాలి కానీ ₹5 coins అయిపోయాయి → ₹2 + ₹2 + ₹1.

Approach	సమస్య
Greedy (పెద్ద coin ముందు)	భారత/US currency కి పని చేస్తుంది. కానీ {1, 3, 4} లాంటి system lo 6 కి greedy 4+1+1 (3 coins) ఇస్తుంది, optimal 3+3 (2 coins)
DP	ఎప్పుడూ optimal, కానీ overkill
Greedy + availability check ✓	Coin inventory పరిమితం కాబట్టి greedy కూడా fail కావచ్చు → change ఇవ్వలేకపోతే transaction ని ముందే తిరస్కరించడం

ఇది ఒక PRODUCT DECISION

"Change ఇవ్వలేకపోతే ఏం చేయాలి? నేను **ముందే check చేసి** – 'exact change మాత్రమే' అని చూపిస్తాను, item ఇచ్చాక 'change లేదు' అనడం కంటే ఇది చాలా మేలు. అంటే **DispensingState** కి వెళ్ళే ముందు change సాధ్యమా అని చూడాలి." – ఇలా failure ని ముందుకి జరపడం మంచి design ఆలోచన.

8. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
State	ప్రతి state ఒక class	Behaviour state ని బట్టి మారుతుంది. కొత్త state = కొత్త class, సున్నా edits
Strategy	PaymentMethod	Coins, card, UPI – ఇవి మారతాయి
⚠ Singleton వద్దు	Machine	Test lo రెండు machines కావాలి
State objects ని share చేయొచ్చా?	Flyweight	State lo instance data లేకపోతే ఒకే object ని share చేయొచ్చు. కానీ ఇక్కడ machine reference ఉంది కాబట్టి కాదు – దీన్ని method parameter గా మార్చితే share చేయొచ్చు

INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Quick scoping: do we dispense change, is cancel supported, and do we need card or UPI alongside coins? I'll assume change and cancel are in scope, coins first with payment methods kept pluggable."

"The obvious implementation is a status field and a switch in every method. I want to explain why I'm not doing that. With five operations and four states, the validity rules get smeared across the whole class – and adding a fifth state means editing every single method. Miss one and you get a silent bug that no compiler catches."

"So I'd use the **State pattern**: each state is its own class. The base state class implements every operation as 'invalid action', and each concrete state overrides only the operations that are legal in it. Idle only implements select. Awaiting payment implements insert-coin and cancel. Dispensing only implements dispense."

"The consequence is that the machine class contains no conditionals at all – every public method just delegates to the current state object. And 'what can I do in this state' is answered by reading one small class rather than grepping five methods."

"State transitions live inside the states themselves, not in the machine. Inserting a coin that brings credit up to the price is what moves us to dispensing, and that logic belongs with awaiting-payment because that's where the condition is meaningful."

"To show why this pays off – if you ask me to add a maintenance mode, I write one new class that overrides **none** of the user operations. Every user action in maintenance automatically becomes an invalid-action error, because that's what the base class does. Zero edits to existing code. In the switch-based version that's four new branches in four methods."

"One detail people skip: dispensing change is a coin-change problem, and greedy is only optimal for well-behaved denominations – plus the machine can physically run out of the coins it needs. I'd check whether exact change is possible **before** entering the dispensing state and show an 'exact change only' message, rather than taking the money and then discovering we can't complete the transaction."

9. Follow-ups

అడిగేది	జవాబు
"State objects ని ప్రతిసారీ new చేయడం వ్యర్థం కాదా?"	Machine reference ని constructor lo కాకుండా method parameter గా పంపితే states stateless అవుతాయి → singleton instances గా share చేయొచ్చు (Flyweight). చిన్న optimisation, కానీ ఆలోచన సరైనది
"Concurrency – ఇద్దరు ఒకేసారి?"	భౌతిక machine కి ఒకే user. కానీ software lo state transition ని atomic గా ఉంచాలి – ఒక lock లేదా single-threaded event loop
"Transaction మధ్యలో power పోతే?"	Credit ని persist చేయాలి (NVRAM). తిరిగి వచ్చాక – credit ఉంటే refund చేసి Idle కి. డబ్బు తీసుకుని item ఇవ్వకపోవడం అన్నిటికంటే చెడ్డ failure

అడిగేది	జవాబు
"Item jam అయితే (motor fail)?"	<code>DispensingState</code> నుంచి <code>FailedState</code> కి – refund + admin alert. Hardware failure ని కూడా state machine lo చూపడం
"Inventory ని ఎలా track చేస్తారు?"	ఒక్క slot కి qty. Threshold కంటే తగ్గితే refill alert (Observer). ఇది vending machine యొక్క నిజమైన business value
"ఇదే pattern ఇంకెక్కడ వాడతారు?"	Order lifecycle, ATM, elevator (Problem 02), TCP connection, media player, game character states – ప్రవర్తన state ని బట్టి మారే ప్రతిచర్య

Design an ATM

Difficulty **Medium-Hard** · Frequency ఎక్కువ · నేర్పే concepts: State, Chain of Responsibility, plan-then-commit

1. The Ask

"Design an ATM. Insert card, enter PIN, withdraw cash – and the machine has to actually produce the right notes."

ఇది Vending Machine యొక్క పెద్దన్న. State pattern అదే, కానీ ఇక్కడ **రిండర్ ప్యాటర్న్** కూడా ఉంది – cash dispensing.

భాగం 1 · State

Idle → CardInserted →
Authenticated → Dispensing.
Problem 05 లో నేర్చుకున్నదే.

భాగం 2 · Cash dispensing

₹5,600 ఇవ్వాలంటే ఏ notes?
₹2000, ₹500, ₹200, ₹100
handlers ఒక **chain** గా.

భాగం 3 · Correctness

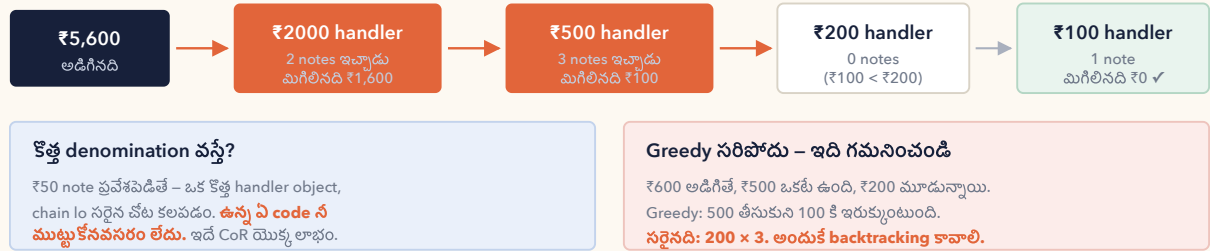
Account debit అయి notes
రాకపోతే? ఈ ఒక్క ప్రశ్నే ఈ problem
ని senior-level చేస్తుంది.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఏవి transactions?	Withdraw, balance, deposit, transfer	Withdraw + balance
Denominations ఏవి?	Dispensing chain	₹2000, ₹500, ₹200, ₹100
PIN attempts limit?	Card retention logic	3
Bank system ఎలా?	Interface గా mock చేస్తాను	Interface
Receipt?	ఒక చిన్న అదనపు step	Optional
Multi-currency / multi-bank?	ఇప్పుడు వద్దు	ఒకే bank

3. Deep Dive – Cash Dispensing (Chain of Responsibility)

₹5,600 ని NOTES గా విడగొట్టడం • ఒక్క HANDLER తనవంతు చేసి మిగిలినది తర్వాతివాడికి



4. Deep Dive – Plan, తర్వాత Commit

తప్పు క్రమం – డబ్బు మాయమవుతుంది

1. Account నుంచి ₹5,600 debit చెయ్యి.
2. Notes ఇవ్వు → **ATM lo సరైన notes లేవని ఇవ్వుడు తెలిసింది** → fail.
3. User కి డబ్బు రాలేదు, కానీ account నుంచి పోయింది. ఇది నిజ ప్రపంచంలో జరిగే bug.

సరైన క్రమం – **PLAN THEN COMMIT**

1. **Plan** – "ఈ మొత్తాన్ని ఇవ్వగలనా?" అని **ఏమీ మార్చకుండా** లెక్కించడం. సాధ్యం కాకపోతే ఇక్కడే ఆపడం, ఇంకా ఏమీ జరగలేదు.
2. **Debit** – account నుంచి తీయడం.
3. **Commit** – planned notes ని dispense చేయడం.

"**మార్చే ముందు ఆలోచించు**" – ఇది distributed transactions lo (two-phase commit), inventory systems lo, ఎక్కడైనా వర్తించే మౌలిక ఆలోచన. ఇక్కడ దీన్ని బయటికి చెప్పడం ముఖ్యం.

5. Code

```
// ---- Chain of Responsibility: cash dispenser ----
class CashDispenser {
  constructor(denomination, next = null) {
    this.denom = denomination; this.next = next; this.count = 0;
  }
  load(count) { this.count = count; return this; }

  // plan() ఏమీ మార్చదు. సాధ్యమైతే Map(denom → count), లేకపోతే null.
  // Greedy సరిపోదు కాబట్టి ఎక్కువ నుంచి తక్కువకి backtrack చేస్తున్నాం.
  plan(amount) {
    const max = Math.min(Math.floor(amount / this.denom), this.count);
    for (let use = max; use >= 0; use--) {
      const remaining = amount - use * this.denom;
      if (remaining === 0) return use > 0 ? new Map([[this.denom, use]]) : new Map();
    }
  }
}
```

```

        if (!this.next) continue;
        const rest = this.next.plan(remaining);
        if (rest) return use > 0 ? new Map([[this.denom, use], ...rest]) : rest;
    }
    return null; // ఈ chain తో ఈ మొత్తం సాధ్యం కాదు
}

commit(plan) {
    this.count -= plan.get(this.denom) ?? 0;
    if (this.next) this.next.commit(plan);
}
totalCash() { return this.denom * this.count + (this.next ? this.next.totalCash() : 0); }
}

```

```

// ---- Bank ఒక interface. ATM దాని అమలు గురించి తెలుసుకోవసరం లేదు. ----
class Bank {
    constructor() { this.accounts = new Map(); } // cardNo → { pin, balance }
    open(cardNo, pin, balance) { this.accounts.set(cardNo, { pin, balance }); return this; }
    verifyPin(cardNo, pin) { return this.accounts.get(cardNo)?.pin === pin; }
    balanceOf(cardNo) { return this.accounts.get(cardNo).balance; }
    debit(cardNo, amount) { this.accounts.get(cardNo).balance -= amount; }
    credit(cardNo, amount) { this.accounts.get(cardNo).balance += amount; }
}

```

```

// ---- State pattern (Problem 05 లాగే) ----
class AtmState {
    constructor(atm) { this.atm = atm; }
    get name() { return this.constructor.name; }
    #invalid(a) { throw new Error(`INVALID_ACTION: ${a} in ${this.name}`); }
    insertCard(cardNo) { this.#invalid('insertCard'); }
    enterPin(pin) { this.#invalid('enterPin'); }
    withdraw(amount) { this.#invalid('withdraw'); }
    checkBalance() { this.#invalid('checkBalance'); }
    ejectCard() { this.#invalid('ejectCard'); }
}

class IdleAtm extends AtmState {
    insertCard(cardNo) {
        this.atm.cardNo = cardNo; this.atm.pinAttempts = 0;
        this.atm.setState(new CardInserted(this.atm));
    }
}

class CardInserted extends AtmState {
    enterPin(pin) {
        if (this.atm.bank.verifyPin(this.atm.cardNo, pin)) {
            this.atm.setState(new Authenticated(this.atm));
            return;
        }
        this.atm.pinAttempts += 1;
        if (this.atm.pinAttempts >= 3) {
            this.atm.reset();
            this.atm.setState(new IdleAtm(this.atm));
            throw new Error('CARD_RETAINED'); // కార్డు మింగేసింది
        }
    }
}

```

```

    }
    throw new Error(`WRONG_PIN:${3 - this.atm.pinAttempts}_LEFT`);
  }
  ejectCard() { this.atm.reset(); this.atm.setState(new IdleAtm(this.atm)); }
}

class Authenticated extends AtmState {
  checkBalance() { return this.atm.bank.balanceOf(this.atm.cardNo); }

  withdraw(amount) {
    if (amount <= 0 || amount % 100 !== 0) throw new Error('INVALID_AMOUNT');
    if (amount > this.atm.bank.balanceOf(this.atm.cardNo)) throw new
Error('INSUFFICIENT_FUNDS');

    const plan = this.atm.dispenser.plan(amount);           // 1 · PLAN (ఏమీ మారలేదు)
    if (!plan) throw new Error('ATM_CANNOT_DISPENSE');

    this.atm.bank.debit(this.atm.cardNo, amount);           // 2 · DEBIT
    this.atm.dispenser.commit(plan);                         // 3 · COMMIT
    return Object.fromEntries(plan);
  }
  ejectCard() { this.atm.reset(); this.atm.setState(new IdleAtm(this.atm)); }
}

```

```

class Atm {
  constructor(bank, dispenser) {
    this.bank = bank; this.dispenser = dispenser;
    this.cardNo = null; this.pinAttempts = 0;
    this.state = new IdleAtm(this);
  }
  setState(s) { this.state = s; }
  get stateName() { return this.state.name; }
  reset() { this.cardNo = null; this.pinAttempts = 0; }

  insertCard(c)    { return this.state.insertCard(c); }
  enterPin(p)      { return this.state.enterPin(p); }
  withdraw(a)      { return this.state.withdraw(a); }
  checkBalance()   { return this.state.checkBalance(); }
  ejectCard()      { return this.state.ejectCard(); }
}

```

```

// ---- వాడుక ----
const dispenser = new CashDispenser(2000,
  new CashDispenser(500, new CashDispenser(200, new CashDispenser(100))));
dispenser.load(2);           // ₹2000 × 2
dispenser.next.load(3);      // ₹500 × 3
dispenser.next.next.load(5); // ₹200 × 5
dispenser.next.next.next.load(10); // ₹100 × 10

const bank = new Bank().open('4111', '1111', 50000);
const atm = new Atm(bank, dispenser);

console.log(atm.stateName); // IdleAtm
try { atm.withdraw(1000); } catch (e) { console.log(e.message); } // INVALID_ACTION:
withdraw in IdleAtm

```

```

atm.insertCard('4111');
try { atm.enterPin('9999'); } catch (e) { console.log(e.message); } //
WRONG_PIN:2_LEFT
atm.enterPin('1111');
console.log(atm.stateName); // Authenticated

console.log(atm.withdraw(5600)); // { '100': 1, '500': 3, '2000': 2 } (₹2000×2 +
₹500×3 + ₹100×1)
console.log(atm.checkBalance()); // 44400
atm.ejectCard();
console.log(atm.stateName); // IdleAtm

```

```

// ---- Backtracking నిజంగా అవసరమా? ఇదిగో నిరూపణ ----
// ₹600 కావాలి. ₹500 ఒక్కటే ఉంది, ₹200 మూడు ఉన్నాయి, ₹100 సున్నా.
const d2 = new CashDispenser(500, new CashDispenser(200, new CashDispenser(100)));
d2.load(1); d2.next.load(3); d2.next.next.load(0);

console.log(d2.plan(600));
// Greedy అయితే: ₹500 తీసుకుని ₹100 మిగులుతుంది → ₹100 లేదు → ఓటమి.
// మన plan() వెనక్కి తగ్గి ₹500 ని వదిలేసి ప్రయత్నిస్తుంది →
// Map(1) { 200 => 3 } ✓ సరైన జవాబు

```

6. Extensibility Test

Interviewer: "Add ₹50 notes, and add a deposit transaction."

```

// 1) కొత్త denomination – chain చివర ఒక handler. ఉన్న handlers ఏవో మారలేదు.
const withFifty = new CashDispenser(2000,
  new CashDispenser(500, new CashDispenser(200,
    new CashDispenser(100, new CashDispenser(50))));

// 2) కొత్త transaction – Authenticated state కి ఒక method, లేదా ఇంకా మంచిది:
// transaction ని ఒక Command object గా చేయడం (Problem 12 లో దీన్ని లోతుగా చూస్తాం).
class DepositTransaction {
  constructor(bank, cardNo, amount) { Object.assign(this, { bank, cardNo, amount }); }
}
execute() { this.bank.credit(this.cardNo, this.amount); }
undo() { this.bank.debit(this.cardNo, this.amount); }
}

```

7. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
State	ATM screens/flow	Card లేకుండా withdraw చేయకూడదు – ఆ నియమం structure లో
Chain of Responsibility	Cash dispenser	కొత్త denomination = కొత్త handler, సున్నా edits

Pattern	ఎక్కడ	ఎందుకు
Dependency Inversion	Bank interface	Real bank, mock bank – ATM కి తేడా తెలియదు. Test సౌధ్యం
Command (సూచించినది)	Transactions	Undo, audit log, transaction history కి

INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me scope: which transactions, what denominations, how many PIN attempts, and do we talk to a real banking system? I'll do withdrawal and balance enquiry, Indian denominations, three PIN attempts, and I'll model the bank as an interface so it's mockable."

"There are two distinct design problems here. The flow is a state machine – you can't withdraw before authenticating – and that's the same State pattern approach as a vending machine. The more interesting one is dispensing the actual notes."

"Dispensing is a Chain of Responsibility. Each denomination is a handler that takes as many notes as it usefully can and passes the remainder down the chain. The payoff is extensibility: when the RBI introduces a fifty-rupee note, I add one handler object and touch no existing code."

"But there's a trap. Pure greedy dispensing is **incorrect**. If someone asks for six hundred and I have one five-hundred note and three two-hundreds, greedy takes the five hundred and then can't make the last hundred – even though three two-hundreds would work. So each handler tries its maximum count and **backtracks downward** if the rest of the chain can't complete. With four denominations that search is trivially cheap, and it makes the result correct rather than usually-correct."

"The detail I care most about is ordering. The naive implementation debits the account and then dispenses – and if dispensing fails, the customer's money has vanished. So I split it into **plan, then commit**. Planning computes the note breakdown without mutating anything; if it returns nothing, we stop before touching the account. Only then do we debit, and only then do we physically dispense."

"That plan-then-commit shape is the same idea as two-phase commit, and it applies far beyond ATMs – any time you have a check and an irreversible action, you want to do all the checking while everything is still undoable."

"I'd model the bank as an interface for two reasons: the real one is a network call I can't run in a test, and it lets me demonstrate the ATM's logic in isolation. In production a network failure mid-withdrawal is the hardest case – I'd handle it with an idempotency key on the debit and a reconciliation job, which is exactly the payment-system reasoning."

8. Follow-ups

అడిగేది	జవాబు
"Dispense physical గా fail అయితే (note jam)?"	Reversal transaction – debit ని తిరిగి credit చేసి, incident ని log చేయడం. Real ATMs ఒక journal రాస్తాయి, రోజు చివర reconciliation lo సరిచేస్తారు
"Network fail అయితే (debit వెళ్ళిందే లేదో తెలియదు)?"	Payment system logic – idempotency key తో transaction, తర్వాత status query. Timeout ≠ failure
"ATM lo cash తక్కువైతే?"	<code>totalCash()</code> threshold కంటే తగ్గితే alert (Observer). Denomination-wise కూడా – ₹100 అయిపోతే చిన్న మొత్తాలు ఇవ్వలేం
"Card retention తర్వాత ఏమవుతుంది?"	Card ని physically retain, bank కి event, user కి branch visit. State machine lo ఇది Idle కి తిరిగి వెళ్తుంది కానీ card లేకుండా
"ఒకేసారి రెండు transactions?"	ATM physically single-user. కానీ account మీద online banking కూడా నడవొచ్చు – అందుకే balance check మరియు debit ని bank side lo atomic గా చేయాలి, ATM lo కాదు
"Receipt, mini statement?"	Bank interface కి కొత్త methods. ATM state machine మారదు – ఇది interface segregation యొక్క ప్రయోజనం

Design an LRU & LFU Cache

Difficulty **Medium** · Frequency **అత్యధికం** · నేర్పే concepts: $O(1)$ design, HashMap + DLL, eviction as strategy

1. The Ask

"Design a cache with a fixed capacity. `get` and `put` must both be $O(1)$. When it's full, evict the least recently used item."

ఇది LLD PROBLEM నా, DSA PROBLEM నా?

రెండూ. Core structure ఒక DSA question (HashMap + doubly linked list). కానీ LLD interview lo అడిగినప్పుడు వాళ్ళు ఇంకా ఏదో చూస్తున్నారు – **eviction policy ని ఎలా pluggable చేస్తారు, TTL ఎలా కలుపుతారు, thread safety ఎలా.** అంటే algorithm తో ఆపేయకుండా, దాన్ని ఒక reusable component గా design చేయాలి.

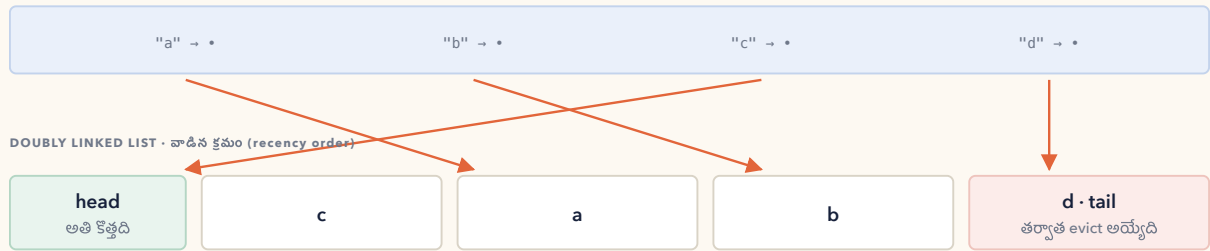
2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Eviction policy fixed నా pluggable నా?	Pluggable అంటే strategy interface	Pluggable (LRU, LFU)
TTL (expiry) కావాలా?	ఉంటే lazy expiry + cleanup	అవును
Thread-safe కావాలా?	Locking మ్యాహం	ప్రస్తావిస్తాం
Capacity items సంఖ్యనా, memory నా?	Memory-based అంటే size estimation	Item count
Miss అయితే ఎవరు load చేస్తారు?	Read-through cache నా, plain నా	Plain (loader ని follow-up lo)
Statistics (hit rate)?	Observability	అవును

3. Deep Dive – O(1) ఎలా సాధ్యం

HASHMAP + DOUBLY LINKED LIST · ఒక్కొక్కటి ఒక్క పని

HASHMAP · key → node pointer (వేగవంతమైన lookup)



ఎందుకు DOUBLY, singly కాదు?

Node నీ O(1) lo తీసివేయాలంటే దాని prev కావాలి.
Singly list lo prev కనుక్కోవడం O(n) – అదే మొత్తం ఆలోచనని చంపుతుంది.

ఎందుకు రెండూ కావాలి

HashMap: "ఈ key ఎక్కడ ఉంది" – O(1), కానీ క్రమం లేదు.
DLL: క్రమం ఉంది, కానీ వెతకడం O(n). కలిపితే రెండూ O(1).

4. Code – LRU

```
class Node {
    constructor(key, value) { this.key = key; this.value = value; this.prev = null;
    this.next = null; }
}

// Sentinel head/tail – దీంతో null checks అన్నీ పోతాయి, edge cases తగ్గుతాయి.
class DoublyLinkedList {
    constructor() {
        this.head = new Node(null, null);
        this.tail = new Node(null, null);
        this.head.next = this.tail;
        this.tail.prev = this.head;
    }
    addFront(node) {
        node.next = this.head.next; node.prev = this.head;
        this.head.next.prev = node; this.head.next = node;
        return node;
    }
    remove(node) {
        node.prev.next = node.next; node.next.prev = node.prev;
        node.prev = node.next = null;
        return node;
    }
    removeLast() {
        return this.tail.prev === this.head ? null : this.remove(this.tail.prev);
    }
    keysFrontToBack() {
        const out = [];
        for (let n = this.head.next; n !== this.tail; n = n.next) out.push(n.key);
        return out;
    }
}
```

```
class LRUCache {
    constructor(capacity) {
```

```

    if (capacity <= 0) throw new Error('CAPACITY_MUST_BE_POSITIVE');
    this.capacity = capacity;
    this.map = new Map(); // key → Node
    this.list = new DoublyLinkedList(); // recency క్రమం
    this.stats = { hits: 0, misses: 0, evictions: 0 };
  }

  get(key) {
    const node = this.map.get(key);
    if (!node) { this.stats.misses += 1; return undefined; }
    this.list.remove(node);
    this.list.addFront(node); // వాడం కాబట్టి ముందుకి
    this.stats.hits += 1;
    return node.value;
  }

  put(key, value) {
    const existing = this.map.get(key);
    if (existing) {
      existing.value = value;
      this.list.remove(existing);
      this.list.addFront(existing);
      return;
    }
    if (this.map.size >= this.capacity) {
      const lru = this.list.removeLast(); // అతి పాతది
      this.map.delete(lru.key);
      this.stats.evictions += 1;
    }
    this.map.set(key, this.list.addFront(new Node(key, value)));
  }

  get size() { return this.map.size; }
  keys() { return this.list.keysFrontToBack(); } // కొత్తది → పాతది
}

```

```

// ---- వాడుక ----
const c = new LRUCache(3);
c.put('a', 1); c.put('b', 2); c.put('c', 3);
console.log(c.keys()); // [ 'c', 'b', 'a' ]

c.get('a'); // 'a' ని ముందుకి తెచ్చింది
console.log(c.keys()); // [ 'a', 'c', 'b' ]

c.put('d', 4); // నిండింది → అతి పాతది 'b' evict
console.log(c.keys()); // [ 'd', 'a', 'c' ]
console.log(c.get('b')); // undefined
console.log(c.stats); // { hits: 1, misses: 1, evictions: 1 }

```

JAVASCRIPT LO ఒక పార్ట్ కట్ - తెలుసుకోండి, కానీ జాగ్రత్త

JS Map insertion order ని కాపాడుతుంది. అంటే LRU ని 10 lines lo రాయొచ్చు: get lo delete చేసి మళ్ళీ set చేయడం, evict కి map.keys().next().value .

Interview lo ఏం చేయాలి: "JavaScript lo Map ordered కాబట్టి దీన్ని 10 lines lo రాయొచ్చు - కానీ మీకు కావలసినది ఆ trick కాదు, అంతర్గత structure అని అనుకుంటున్నాను, కాబట్టి HashMap + DLL ని explicit గా రాస్తాను." ఇలా చెప్పడం - మీకు రెండూ తెలుసని, మరియు interview యొక్క ఉద్దేశం అర్థమైందని చూపిస్తుంది.

5. Code - LFU (ఇది చాలా కష్టం)

LFU సమస్య: "అతి తక్కువ సార్లు వాడినది" ని $O(1)$ lo కనుక్కోవాలి. Sorting $O(n \log n)$, scanning $O(n)$ - రెండూ పనికిరావు.

పరిష్కారం: ఒక్కో frequency కి ఒక bucket + minFreq ని track చేయడం.

```
class LFUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.values = new Map(); // key → value
    this.freqOf = new Map(); // key → frequency
    this.buckets = new Map(); // frequency → Set(keys) ← Set క్రమం కాపాడుతుంది
    this.minFreq = 0;
  }

  #bump(key) {
    const f = this.freqOf.get(key);
    const bucket = this.buckets.get(f);
    bucket.delete(key);
    if (bucket.size === 0) {
      this.buckets.delete(f);
      if (this.minFreq === f) this.minFreq += 1; // ఇదే  $O(1)$  ని కాపాడే ఉపాయం
    }
    const nf = f + 1;
    this.freqOf.set(key, nf);
    if (!this.buckets.has(nf)) this.buckets.set(nf, new Set());
    this.buckets.get(nf).add(key);
  }

  get(key) {
    if (!this.values.has(key)) return undefined;
    this.#bump(key);
    return this.values.get(key);
  }

  put(key, value) {
    if (this.capacity <= 0) return;
    if (this.values.has(key)) { this.values.set(key, value); this.#bump(key); return; }

    if (this.values.size >= this.capacity) {
      const bucket = this.buckets.get(this.minFreq);
      const victim = bucket.values().next().value; // అదే freq lo అత్యంత పాతది (LRU
    }
  }
}
```

```

    tie-break)
        bucket.delete(victim);
        if (bucket.size === 0) this.buckets.delete(this.minFreq);
        this.values.delete(victim);
        this.freqOf.delete(victim);
    }
    this.values.set(key, value);
    this.freqOf.set(key, 1);
    if (!this.buckets.has(1)) this.buckets.set(1, new Set());
    this.buckets.get(1).add(key);
    this.minFreq = 1;
}
}

```

```

// ---- వాడుక ----
const f = new LFUCache(2);
f.put('a', 1); f.put('b', 2);
f.get('a'); // 'a' freq = 2, 'b' freq = 1
f.put('c', 3); // సిండిది → అతి తక్కువ freq ఉన్న 'b' evict
console.log(f.get('b')); // undefined
console.log(f.get('a'), f.get('c')); // 1 3

```

‘MINFREQ’ ఎందుకు ఎప్పుడూ సరిగ్గా ఉంటుంది – ఇది చెప్తే మీరు గెలిచారు

`minFreq` రెండే సందర్భాల్లో మారుతుంది: (1) కొత్త key చేరినప్పుడు అది 1 కి reset అవుతుంది – ఎందుకంటే కొత్తదాని frequency 1, అంతకంటే తక్కువ ఉండదు. (2) `minFreq` bucket ఖాళీ అయినప్పుడు అది **సరిగ్గా 1 మాత్రమే** పెరుగుతుంది – ఎందుకంటే bucket ఖాళీ అవ్వాలంటే ఆ key `f` నుంచి `f+1` కి వెళ్ళి ఉండాలి. అందుకే scan అవసరం లేదు, $O(1)$. **ఈ invariant ని మాటల్లో నిరూపించడం** interviewer కి చాలా బలమైన signal.

6. Eviction ని Strategy గా మార్చడం

```

class EvictionPolicy {
    onAccess(key) {} // get/put జరిగినప్పుడు
    onInsert(key) {} // కొత్త key చేరినప్పుడు
    onRemove(key) {} // key తీసేసినప్పుడు
    evictCandidate() { throw new Error('abstract'); } // ఎవరిని బయటికి పంపాలి
}

class Cache { // ఇప్పుడు Cache కి LRU/LFU గురించి తెలియదు
    constructor(capacity, policy) { this.capacity = capacity; this.policy = policy;
    this.store = new Map(); }
    get(key) { if (this.store.has(key)) this.policy.onAccess(key); return
    this.store.get(key); }
    put(key, value) {
        if (!this.store.has(key) && this.store.size >= this.capacity) {
            const victim = this.policy.evictCandidate();
            this.store.delete(victim); this.policy.onRemove(victim);
        }
        const isNew = !this.store.has(key);
        this.store.set(key, value);
        isNew ? this.policy.onInsert(key) : this.policy.onAccess(key);
    }
}

```

```
}  
}
```

ఇక్కడ ఒక నిజాయితీ

ఈ abstraction అందంగా ఉంది, కానీ **ఖరీదు ఉంది** – policy కి తన సొంత data structures కావాలి, cache కి కూడా ఒకటి ఉంది, రెండూ sync lo ఉండాలి. LRU-only cache lo node pointer ని నేరుగా వాడితే అది వేగం.

"**Policies నిజంగా మారతాయా** అన్నదే ప్రశ్న. Redis లాంటి general-purpose cache రాస్తుంటే అవును. ఒక service lo ఒక్క LRU cache కావాలంటే – ఈ abstraction over-engineering. నేను interview lo రెండూ చూపించి, ఏ సందర్భంలో ఏది అని చెప్తాను." – ఈ nuance చూపడం maturity.

7. TTL, Thread Safety, Statistics

అవసరం	ఎలా
TTL	Node lo <code>expiresAt</code> . <code>get</code> lo expired అయితే – తీసేసి <code>undefined</code> ఇవ్వడం (lazy expiry). ప్లస్ ఒక background sweeper (లేకపోతే ఎప్పటికీ చదవని expired items memory తింటాయి)
Thread safety	Java lo <code>ReentrantLock</code> లేదా <code>ConcurrentHashMap</code> + segment locks. JS single-threaded కాబట్టి అవసరం లేదు – కానీ దీన్ని బయటికి చెప్పాలి , లేకపోతే "concurrency ఆలోచించలేదు" అనుకుంటారు
Lock contention	ఒకే lock అంటే bottleneck. Striped locks – key hash ప్రకారం N segments, ఒక్కో దానికి ఒక lock (ConcurrentHashMap ఇదే చేస్తుంది)
Statistics	hits, misses, evictions – hit rate తెలియకపోతే cache size సరైనదా అని తెలియదు
Read-through	<code>get(key, loader)</code> – miss అయితే loader ని పిలిచి, ఫలితాన్ని cache చేయడం. ప్లస్ request coalescing (ఒకే key కి 100 concurrent misses → ఒకే loader call)

INTERVIEW SCRIPT · ఇలా చెప్పుండి (ENGLISH)

"A few questions: is the eviction policy fixed or should it be pluggable, do we need TTL, and does this need to be thread-safe? I'll build LRU first, make the policy swappable, and cover TTL and locking as I go."

"The requirement that drives everything is $O(1)$ for both operations. A hash map gives me $O(1)$ lookup but has no notion of order. A list gives me order but $O(n)$ search. So I use both: a hash map from key to **node**, and a doubly linked list holding recency order."

"It has to be **doubly** linked. To evict or promote a node in constant time I need to unlink it, and unlinking requires knowing its predecessor. In a singly linked list finding the predecessor is $O(n)$, which destroys the whole point."

"I'd use sentinel head and tail nodes. They cost two objects and remove every null check from insert and remove – the edge cases where the list is empty or has one element simply stop existing."

"I should mention: JavaScript's Map preserves insertion order, so I could write LRU in about ten lines using delete-and-reinsert. But I assume you want to see the underlying structure rather than a language trick, so I'll build it explicitly."

"LFU is meaningfully harder, because finding the least **frequently** used item in constant time isn't obvious. The trick is a bucket per frequency plus a tracked minimum frequency. The reason that minimum stays correct without scanning is worth stating: it resets to one whenever a new key arrives, since nothing can have a lower count than a brand new entry – and when the minimum's bucket empties, it can only ever increase by exactly one, because a bucket empties precisely when its last key was promoted to the next frequency."

"I'd also use insertion-ordered sets inside each bucket, which gives LRU tie-breaking within the same frequency for free – which is what you want, since 'both used twice, evict the older one' is the sensible behaviour."

"On making the policy pluggable: I can define an eviction policy interface with access, insert, remove and evict-candidate hooks. But I'd be honest about the trade-off – the policy then needs its own data structures alongside the cache's, and keeping them synchronised costs both performance and a class of bugs. If I'm building a general cache library, that abstraction earns its keep. If a service needs one LRU cache, it's over-engineering."

"For TTL I'd expire **lazily** on read, plus a background sweeper – without the sweeper, entries that are never read again occupy memory forever. And in a threaded language I'd use striped locks rather than one global lock, since a single lock on a hot cache becomes the bottleneck it was meant to relieve."

8. Follow-ups

అడిగేది	జవాబు
"LRU vs LFU ఎప్పుడు?"	LRU : temporal locality ఉన్నచోట (ఇప్పుడే చూసినది మళ్ళీ చూస్తారు) – web sessions, file access. LFU : కొన్ని items శాశ్వతంగా popular – CDN, product catalogue. LFU యొక్క సమస్య: ఒకప్పుడు popular అయిన item శాశ్వతంగా cache lo ఇరుక్కుంటుంది
"ఆ LFU సమస్యని ఎలా పరిష్కరిస్తారు?"	Aging – కాలానుగుణంగా అన్ని frequencies ని సగం చేయడం, లేదా window-based LFU (గత గంటలో frequency). Redis LFU logarithmic counter + decay వాడుతుంది
"Distributed cache కి ఇదే design?"	కాదు – అప్పుడు consistent hashing తో shards, ఒక్కో node lo ఇలాంటి local cache. Eviction node స్థాయిలో, global కాదు
"Memory-based capacity?"	ఒక్కో entry size ని అంచనా వేసి (serialized bytes) track చేయడం. Evict చేసేటప్పుడు target bytes కి చేరేదాకా పలు items తీయడం
"Cache stampede (ఒకే key కి 1000 concurrent misses)?"	Request coalescing – ఆ key కోసం ఒక in-flight promise ని store చేసి, మిగతా callers అదే promise ని await చేయడం. Loader ఒక్కసారే నడుస్తుంది
"Write policy – write-through నా write-back నా?"	Write-through : cache + DB రెండూ వెంటనే – సురక్షితం, నెమ్మది. Write-back : cache lo రాసి తర్వాత flush – వేగం, కానీ crash అయితే data loss. Cache కి durability లేదని గుర్తుంచుకోవాలి

Design a Rate Limiter (class design)

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: algorithms as strategies, clock injection, rule engine

1. The Ask

"Design a rate limiter library. Different endpoints need different limits and different algorithms."

HLD PROBLEM 02 తే తేడా

HLD book lo మనం అడిగింది – "50 servers మీద counter ఎక్కడ ఉంచాలి, Redis race ని ఎలా ఆపాలి". అది **distributed systems** ప్రశ్న.

ఇక్కడ ప్రశ్న వేరు – "ఈ 5 algorithms ని ఒకే interface వెనక ఎలా పెట్టాలి, rules ని ఎలా configure చేయాలి, సమయాన్ని ఎలా test చేయాలి". ఇది **object design** ప్రశ్న. ఒకే domain, రెండు వేర్వేరు interviews.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఏవి algorithms support చేయాలి?	Strategy interface ని ఇది నిర్ణయిస్తుంది	Fixed window, sliding window, token bucket
Key ఏమిటి – user, IP, endpoint?	Composite key design	apiKey + endpoint
ఒక్కే endpoint కి వేరే limit?	Rule engine కావాలి	అవును
In-memory నా distributed నా?	Storage ని abstract చేయాలా	ఇప్పుడు memory, తర్వాత Redis
Response ఏమిటి – boolean నా, ఎంత మిగిలిందో నా?	Retry-After ఇవ్వాలంటే మరింత సమాచారం కావాలి	Rich result
Thread-safe?	Locking	ప్రస్తావిస్తాం

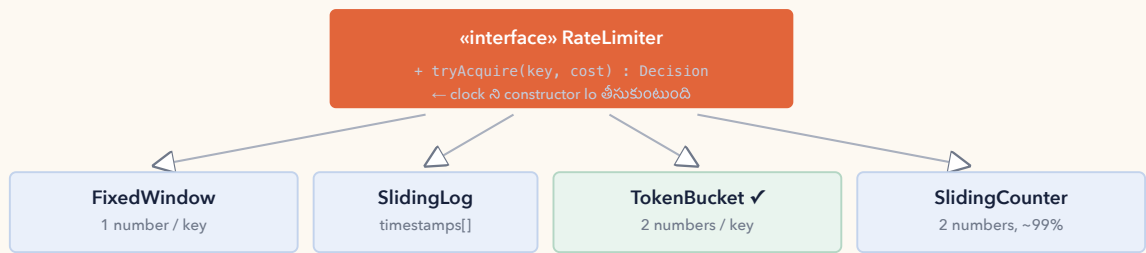
3. Deep Dive – Clock ని Inject చేయడం

ఇదే ఈ PROBLEM LO అతి ముఖ్యమైన DESIGN నిర్ణయం

Rate limiter అంతా సమయం మీద ఆధారపడి ఉంటుంది. `Date.now()` ని నేరుగా పిలిస్తే – "1 నిమిషం తర్వాత limit reset అవుతుందా?" అని test చేయాలంటే నిజంగా ఒక నిమిషం sleep చేయాలి. అది test suite ని పనికిరానిదిగా చేస్తుంది.

అందుకే Clock ని constructor lo inject చేస్తాను. Test lo FakeClock పెట్టి సమయాన్ని కావలసినంత ముందుకి జరపొచ్చు. "సమయాన్ని ఒక dependency గా చూడటం" – ఇది testable design యొక్క గుర్తు, మరియు చాలా మంది candidates దీన్ని ఆలోచించరు.

CLASS STRUCTURE • ఒక INTERFACE, పలు ALGORITHMS



అన్నీ ఒకే interface – కాబట్టి config lo ఒక string మార్చి algorithm ని swap చేయొచ్చు
Production lo A/B కూడా చేయొచ్చు: 10% traffic కి token bucket, 90% కి sliding counter – ఏది మెరుగో చూడటానికి

4. Code

```
class Clock { now() { return Date.now(); } }
class FakeClock extends Clock {
    constructor(t = 0) { super(); this.t = t; }
    now() { return this.t; }
    advance(ms) { this.t += ms; return this; }
}
```

```
// అన్ని limiters ఇచ్చే జవాబు – కేవలం boolean కాదు.
// Retry-After header ఇవ్వాలంటే retryAfterMs కావాలి.
class Decision {
    constructor(allowed, remaining, retryAfterMs = 0) {
        Object.assign(this, { allowed, remaining, retryAfterMs });
    }
}
```

```
class RateLimiter {
    constructor(clock = new Clock()) { this.clock = clock; }
    tryAcquire(key, cost = 1) { throw new Error('abstract'); }
}
```

```
class FixedWindowLimiter extends RateLimiter {
    constructor(limit, windowMs, clock) { super(clock); this.limit = limit;
    this.windowMs = windowMs; this.state = new Map(); }
```

```

tryAcquire(key, cost = 1) {
    const now = this.clock.now();
    const windowStart = Math.floor(now / this.windowMs) * this.windowMs;
    let entry = this.state.get(key);
    if (!entry || entry.windowStart !== windowStart) {
        entry = { windowStart, count: 0 };
        this.state.set(key, entry);
    }
    if (entry.count + cost > this.limit) {
        return new Decision(false, this.limit - entry.count, windowStart +
this.windowMs - now);
    }
    entry.count += cost;
    return new Decision(true, this.limit - entry.count);
}
}

class TokenBucketLimiter extends RateLimiter {
    constructor(capacity, refillPerSec, clock) {
        super(clock); this.capacity = capacity; this.refillPerSec = refillPerSec;
        this.state = new Map();
    }
    tryAcquire(key, cost = 1) {
        const now = this.clock.now();
        let b = this.state.get(key);
        if (!b) { b = { tokens: this.capacity, ts: now }; this.state.set(key, b); }

        // Lazy refill - background timer లేదు. గడిచిన కాలానికి సరిపడా tokens కలపడం.
        const elapsedSec = (now - b.ts) / 1000;
        b.tokens = Math.min(this.capacity, b.tokens + elapsedSec * this.refillPerSec);
        b.ts = now;

        if (b.tokens < cost) {
            const waitMs = Math.ceil(((cost - b.tokens) / this.refillPerSec) * 1000);
            return new Decision(false, Math.floor(b.tokens), waitMs);
        }
        b.tokens -= cost;
        return new Decision(true, Math.floor(b.tokens));
    }
}

class SlidingLogLimiter extends RateLimiter {
    constructor(limit, windowMs, clock) { super(clock); this.limit = limit;
        this.windowMs = windowMs; this.state = new Map(); }
    tryAcquire(key, cost = 1) {
        const now = this.clock.now();
        const cutoff = now - this.windowMs;
        const log = (this.state.get(key) ?? []).filter((t) => t > cutoff);    // పాతవి తీసేయడం
        if (log.length + cost > this.limit) {
            this.state.set(key, log);
            return new Decision(false, this.limit - log.length, log[0] + this.windowMs -
now);
        }
        for (let i = 0; i < cost; i++) log.push(now);
        this.state.set(key, log);
        return new Decision(true, this.limit - log.length);
    }
}

```

```
}
}
```

```
// Rule engine - ఏ request కి ఏ limiter అన్నది ఇది నిర్ణయిస్తుంది.
class RateLimitRule {
  constructor(name, matcher, limiter) { Object.assign(this, { name, matcher, limiter }); }
}

class RuleBasedRateLimiter {
  constructor(rules, fallback) { this.rules = rules; this.fallback = fallback; }
  tryAcquire(request) {
    const rule = this.rules.find((r) => r.matcher(request));
    const limiter = rule ? rule.limiter : this.fallback;
    const key = `${request.apiKey}:${rule ? rule.name : 'default'}`;
    return { rule: rule?.name ?? 'default', ...limiter.tryAcquire(key) };
  }
}
```

```
// ---- Token bucket: burst allow చేస్తుంది, తర్వాత refill ----
const clock = new FakeClock(0);
const tb = new TokenBucketLimiter(5, 1, clock); // capacity 5, సెకనుకి 1 token

console.log([1,2,3,4,5,6].map(() => tb.tryAcquire('u1').allowed));
// [ true, true, true, true, true, false ] ← 5 burst గా వెళ్ళాయి, 6వది ఆగింది

console.log(tb.tryAcquire('u1').retryAfterMs); // 1000 (1 token కోసం 1 సెకను)
clock.advance(2000); // 2 సెకన్లు గడిచాయి → 2 tokens
console.log([1,2,3].map(() => tb.tryAcquire('u1').allowed));
// [ true, true, false ]
```

```
// ---- Fixed window యొక్క boundary bug - నిజంగా చూపిద్దాం ----
const c2 = new FakeClock(0);
const fw = new FixedWindowLimiter(3, 1000, c2); // సెకనుకి 3

c2.t = 999; // window [0, 1000) చివరన
console.log([1,2,3].map(() => fw.tryAcquire('u').allowed)); // [ true, true, true ]
c2.t = 1001; // కొత్త window మొదలు
console.log([1,2,3].map(() => fw.tryAcquire('u').allowed)); // [ true, true, true ]
// → 2 milliseconds lo 6 requests, limit "సెకనుకి 3" అయినా. ఇదే boundary problem.
```

ఈ చివరి TEST ఎందుకు విలువైనది

Fixed window యొక్క boundary bug ని మాటల్లో వివరించడం ఒక level. దాన్ని ఒక test గా నిరూపించడం ఇంకో level. Interview lo ఇలా చెప్పండి: "నేను fixed window ని implement చేస్తాను, కానీ దాని లోపాన్ని ఒక test తో document చేస్తాను - తర్వాత ఎవరైనా దాన్ని ఎంచుకునేటప్పుడు ఏం కొంటున్నారో తెలిసి కొంటారు." లోపాలని దాచకుండా document చేయడం - ఇది library రాసేవాళ్ళ ఆలోచన.

5. Extensibility & Distributed

```
// Storage ని వేరు చేస్తే అదే limiter distributed అవుతుంది.
class LimiterStore {
  get(key) {}
  set(key, value) {}
  // నిజమైన Redis store lo ఈ మొత్తం read-modify-write ఒక Lua script అవుతుంది.
}

// Composite - పలు limiters ని కలిపి "అన్నీ pass అయితేనే allow"
class CompositeLimiter extends RateLimiter {
  constructor(limiters) { super(); this.limiters = limiters; }
  tryAcquire(key, cost = 1) {
    const results = this.limiters.map((l) => l.tryAcquire(key, cost));
    const denied = results.find((r) => !r.allowed);
    return denied ?? results[0];
  }
}
// వాడుక: "నిమిషానికి 100 AND గంటకి 1000" - రెండు limiters, ఒక composite.
```

COMPOSITE LO ఒక సూక్ష్మమైన BUG - దీన్ని బయటికి చెప్పండి

పై code lo, మొదటి limiter allow చేసి **token తీసుకుంది**, కానీ రెండోది deny చేసింది. అంటే మొదటి limiter నుంచి ఒక token **వృథాగా పోయింది**. సరైన పరిష్కారం: ముందు అన్నిటిని *check* చేసి (peek), అన్నీ ok అయితేనే *commit* చేయడం - ATM lo plan-then-commit లాంటిదే. ఈ bug ని **మీరే గుర్తించి చెప్పడం** interviewer కి బలమైన signal.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Strategy	RateLimiter implementations	Algorithm ని config తో మార్చాలి, A/B test చేయాలి
Composite	CompositeLimiter	పలు limits ని కలపడం, ఒకే interface
Dependency Injection	Clock, LimiterStore	Testability + memory→Redis మార్పు
Chain of Responsibility (ప్రత్యామ్నాయం)	Rule matching	Rules ని ఒక chain గా కూడా చేయొచ్చు - మొదటి match గెలుస్తుంది

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me clarify what varies: which algorithms do we need, is the key per user or per user-and-endpoint, are limits configurable per endpoint, and is this in-process or distributed? I'll build it in memory with a storage seam so it can become distributed later."

"The core abstraction is a single interface – try-acquire, taking a key and a cost. I'd return a rich result rather than a boolean, because the caller needs to emit a Retry-After header, and only the limiter knows when capacity next becomes available."

"Behind that interface I'd implement fixed window, sliding log and token bucket. They differ dramatically in memory: fixed window is one integer per key, token bucket is two numbers, and sliding log stores every request timestamp – which is exact but doesn't scale."

"The design decision I care most about is **injecting the clock**. Everything here is time-dependent, and if I call Date.now directly, testing 'does the limit reset after a minute' means actually sleeping for a minute. With a clock dependency I substitute a fake and advance time instantly. Treating time as a dependency rather than an ambient global is what makes time-based logic testable at all."

"That pays off immediately. I can write a test that **demonstrates** fixed window's boundary flaw – three requests at 999 milliseconds and three more at 1001, six requests in two milliseconds against a limit of three per second. I'd ship fixed window with that test attached, so anyone choosing it knows exactly what they're buying. Documenting a known weakness is better than hiding it."

"On top of the algorithms I'd put a small rule engine that matches a request to a rule and derives the key, so limits are configuration rather than code. And a composite limiter lets me express 'a hundred per minute and a thousand per hour' as two limiters combined."

"There's a subtle bug in the naive composite that I'd fix explicitly: if the first limiter allows and consumes a token but the second denies, that first token is lost. So the composite has to **peek** at all limiters first and only commit if all of them allow – the same plan-then-commit shape as dispensing cash from an ATM."

"To make this distributed, the state map becomes a storage interface. The important detail is that read-modify-write must be atomic – in Redis that means the whole thing runs as a Lua script, not as separate GET and SET calls."

7. Follow-ups

అడిగేది	జవాబు
"Thread safety ఎలా?"	ఒక్కే key కి lock (striped locks) – global lock వద్దు, అది bottleneck. లేదా atomic operations. JS single-threaded కాబట్టి ఇక్కడ సమస్య లేదని చెప్పాలి
"Memory leak – inactive keys?"	TTL / LRU eviction. state Map ని Problem 07 lo రాసిన LRU cache గా చేయడం – సరిగ్గా ఇక్కడ అది పనికొస్తుంది

అడిగేది	జవాబు
"Rules ని runtime lo మార్చాలంటే?"	Config store (etcd/S3) నుంచి poll చేసి <code>rules</code> array ని atomic గా replace. Deploy అవసరం లేదు
"Different cost per request?"	<code>tryAcquire(key, cost)</code> – ఇప్పటికే ఉంది. భారీ API call కి 10 tokens, తేలికైనదానికి 1
"Limiter చస్తే?"	Fail-open – తప్పు జరిగితే allow. కానీ ఇది per-rule configurable ఉండాలి (login endpoint కి fail-closed)
"Sliding window counter ని ఎలా రాస్తారు?"	$count = current \times (elapsed/window) + previous \times (1 - elapsed/window)$. రెండే numbers, boundary bug లేదు, ~99% ఖచ్చితం – general purpose కి ఉత్తమ రాజీ

Design a Logging Framework

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: appenders, formatters, level filtering, CoR యొక్క సరైన వాడుక

1. The Ask

"Design a logging library like log4j. Different log levels, multiple destinations, configurable formats."

ఇది ఒక ఉచ్చు ఉన్న PROBLEM

చాలా blogs ఈ problem కి **Chain of Responsibility** చెప్తాయి – DebugHandler → InfoHandler → ErrorHandler. కానీ నిజమైన frameworks (log4j, winston, zap) CoR వాడవు. అవి appenders యొక్క జాబితా వాడతాయి.

ఎందుకు అనేది interview lo చెప్పగలిగితే – మీరు pattern ని కేవలం గుర్తుపెట్టుకోలేదని, ఎప్పుడు అది సరైనదో అర్థం చేసుకున్నారని అర్థమవుతుంది. ఇదే ఈ problem యొక్క నిజమైన విలువ.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఎవ levels?	TRACE...FATAL – సంఖ్యలుగా compare చేయాలి	6 levels
ఎవ destinations?	Console, file, network, DB	Pluggable
Format configurable నా?	Plain vs JSON (structured logging)	అవును
Async కావాలా?	File/network write ని hot path నుంచి తీయాలి	అవును
Hierarchical loggers (<code>app.db</code>)?	Config inheritance	అవును
Thread-safe?	పలు threads ఒకే appender కి రాయడం	ప్రస్తావిస్తాం

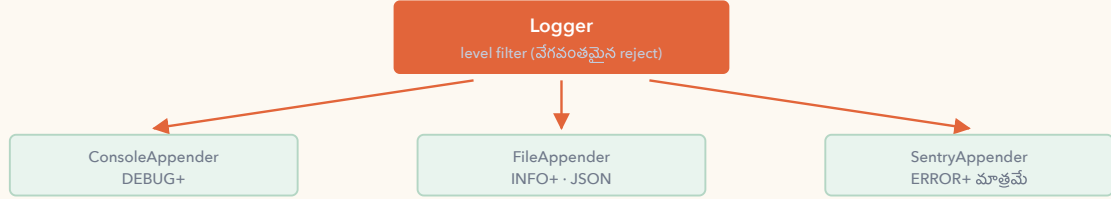
3. Deep Dive – CoR సరైనదేనా?

CHAIN VS LIST · తేడా ఏమిటి

A · CHAIN OF RESPONSIBILITY – ముస్తాఖాంజ్ చెప్పింది



B · APPENDER LIST – నిజమైన frameworks చేసింది



తేడా ఇదే: CoR lo ఒక handler "ఇది నాది" అని తీసుకుని chain ని ఆపేయొచ్చు. కానీ logging lo ఒక ERROR message **console, file, Sentry** – మూడు చోట్లకీ వెళ్ళాలి. ఎవరూ దాన్ని "వినియోగించుకోరు". అంటే ఇది fan-out, chain కాదు.

COR ఎప్పుడు సరైనది – ఈ తేడాని చెప్పండి

CoR సరైనది ఎప్పుడంటే: ఒక request ని ఒక్కడే handle చేయాలి, మరియు ఎవరు handle చేస్తారన్నది క్రమం మీద ఆధారపడి ఉంటుంది. ఉదాహరణలు – ATM cash dispenser (Problem 06): ₹2000 handler ముందు రావాలి, తర్వాత ₹500. Validation pipeline: మొదటి failure అక్కడే ఆపుతుంది. HTTP middleware: ఒక middleware request ని short-circuit చేయొచ్చు.

Logging అలా కాదు – ప్రతి appender ప్రతి (సరిపోయే) message ని చూడాలి, ఎవరూ ఎవరినీ ఆపరు. అందుకే **list**, chain కాదు.

4. Code

```
const Level = Object.freeze({ TRACE: 10, DEBUG: 20, INFO: 30, WARN: 40, ERROR: 50, FATAL: 60 });
const levelName = (v) => Object.keys(Level).find((k) => Level[k] === v);

class LogEvent {
  constructor(level, message, context, timestamp, loggerName) {
    Object.assign(this, { level, message, context, timestamp, loggerName });
  }
}

// ---- Formatters: event → string ----
class Formatter { format(event) { throw new Error('abstract'); } }

class PlainFormatter extends Formatter {
  format(e) {
    const ctx = e.context ? ' ' + JSON.stringify(e.context) : '';
    return `${new Date(e.timestamp).toISOString()} [${levelName(e.level)}] ${e.loggerName} - ${e.message}${ctx}`;
  }
}

class JsonFormatter extends Formatter { // structured logging – machines
```

```

చదవడానికి
format(e) {
    return JSON.stringify({
        ts: e.timestamp, level: levelName(e.level), logger: e.loggerName, msg:
e.message, ...e.context,
    });
}
}

```

```

// ---- Appenders: string → ఎక్కడికీ ----
class Appender {
    constructor({ minLevel = Level.TRACE, formatter = new PlainFormatter() } = {}) {
        this.minLevel = minLevel; this.formatter = formatter;
    }
    handle(event) {
        if (event.level < this.minLevel) return; // ఒక్కో appender కి తన సొంత threshold
        this.write(this.formatter.format(event), event);
    }
    write(line, event) { throw new Error('abstract'); }
}

class ConsoleAppender extends Appender { write(line) { console.log(line); } }

class MemoryAppender extends Appender { // test కోసం – sink ని inject చేయడం
    అవసరం లేదు
    constructor(opts) { super(opts); this.lines = []; }
    write(line) { this.lines.push(line); }
}

// Decorator – ఏ appender నైనా async చేయడం. Appender కి ఇది తెలియదు.
class AsyncAppender extends Appender {
    constructor(inner, flushEvery = 100) { super({ minLevel: inner.minLevel });
    this.inner = inner; this.buffer = []; this.flushEvery = flushEvery; }
    handle(event) {
        this.buffer.push(event);
        if (this.buffer.length >= this.flushEvery) this.flush();
    }
    flush() { for (const e of this.buffer) this.inner.handle(e); this.buffer = []; }
}

```

```

class Logger {
    constructor(name, level = Level.INFO, appenders = [], clock = { now: () =>
Date.now() }) {
        Object.assign(this, { name, level, appenders, clock });
    }

    #log(level, message, context) {
        if (level < this.level) return; // ← వేగవంతమైన reject. String format
        కూడా చేయము.
        const event = new LogEvent(level, message, context, this.clock.now(), this.name);
        for (const a of this.appenders) a.handle(event); // fan-out – chain కాదు
    }

    trace(m, c) { this.#log(Level.TRACE, m, c); }
    debug(m, c) { this.#log(Level.DEBUG, m, c); }
    info(m, c) { this.#log(Level.INFO, m, c); }
}

```

```
warn(m, c) { this.#log(Level.WARN, m, c); }
error(m, c) { this.#log(Level.ERROR, m, c); }
fatal(m, c) { this.#log(Level.FATAL, m, c); }
}
```

```
// ---- వాడుక ----
const all = new MemoryAppender({ minLevel: Level.DEBUG, formatter: new
PlainFormatter() });
const alerts = new MemoryAppender({ minLevel: Level.ERROR, formatter: new
JsonFormatter() });

const log = new Logger('app.db', Level.DEBUG, [all, alerts], { now: () => 0 });

log.trace('too noisy'); // logger level DEBUG →
వదిలేయబడింది
log.debug('connecting', { host: 'db-1' });
log.error('query failed', { code: 'ETIMEDOUT' });

console.log(all.lines);
// [ '1970-01-01T00:00:00.000Z [DEBUG] app.db - connecting {"host":"db-1"}',
//   '1970-01-01T00:00:00.000Z [ERROR] app.db - query failed {"code":"ETIMEDOUT"}' ]

console.log(alerts.lines);
// [ '{"ts":0,"level":"ERROR","logger":"app.db","msg":"query
failed","code":"ETIMEDOUT"}' ]
//   ↑ ERROR మాత్రమే వచ్చింది, మరియు JSON format లో - ఒకే event, రెండు రూపాలు
```

రెండు స్థాయిల LEVEL CHECK ఎందుకు

Logger level – "ఈ message ని అసలు తయారు చేయాలా?" ఇది hot path optimisation. `log.debug()` ని production లో కోట్ల సార్లు పిలుస్తారు; level check ముందే జరిగితే **event object సృష్టించము, string format చేయము**. ఇది నిజమైన performance తేడా.

Appender level – "ఈ message నా destination కి వెళ్ళాలా?" Console కి అంతా, Sentry కి errors మాత్రమే. ఒకే logger, వేర్వేరు thresholds.

5. Hierarchical Loggers

```
class LoggerFactory {
  constructor(rootLevel = Level.INFO, appenders = []) {
    this.config = new Map(['', { level: rootLevel, appenders }]);
    this.cache = new Map();
  }
  configure(name, { level, appenders }) { this.config.set(name, { level, appenders }); this.cache.clear(); }

  // "app.db.pool" కి config లేకపోతే → "app.db" → "app" → root. అత్యంత నిర్దిష్టమైనది గెలుస్తుంది.
  get(name) {
    if (this.cache.has(name)) return this.cache.get(name);
    let parts = name.split('.');
    let cfg = null;
```

```

while (parts.length >= 0) {
    const key = parts.join('.');
    if (this.config.has(key)) { cfg = { ...this.config.get(''),
...this.config.get(key) }; break; }
    if (parts.length === 0) break;
    parts.pop();
}
const logger = new Logger(name, cfg.level, cfg.appenders);
this.cache.set(name, logger);
return logger;
}
}

```

HIERARCHY ఎందుకు విలువైనది

Production lo ఒక bug debug చేయాలి. మొత్తం app ని DEBUG కి మార్చితే logs lo మునిగిపోతారు. Hierarchy తో – `app.payment` ని మాత్రం DEBUG కి, మిగతా అంతా INFO. **ఒక్క config line, deploy లేకుండా.** ఇది logging framework యొక్క అసలు value.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Strategy	<code>Formatter</code>	Plain, JSON, XML – format మారుతుంది
Decorator	<code>AsyncAppender(inner)</code>	ఏ appender నైనా async చేయొచ్చు, దానికి తెలియకుండా
Observer (వదులుగా)	Appender list	Logger appenders గురించి కేవలం interface ద్వారా తెలుసు
❌ Chain of Responsibility	–	వాడలేదు, ఎందుకంటే ఇది fan-out. ఇలా చెప్పడమే ఈ problem lo మీ బలం

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me confirm scope: which levels, which destinations, is the format configurable, do we need async writes, and do we want hierarchical logger names? I'll build all of those, with async as a decorator."

"I want to address something up front, because this problem is usually answered with Chain of Responsibility – a debug handler passing to an info handler passing to an error handler. I don't think that's the right pattern here, and I'd like to explain why."

"Chain of Responsibility is for when **one** handler should take a request and the rest shouldn't see it, and where order determines who takes it. The ATM cash dispenser is a genuine chain: the two-thousand handler must run before the five-hundred one. But in logging, an error message should reach the console **and** the file **and** Sentry. Nobody consumes it. That's a fan-out, not a chain – so the right structure is a **list of appenders**, which is exactly what log4j and winston do."

"The pieces are: a Logger that owns a level and a list of appenders; Appenders that own a destination, their own minimum level, and a Formatter; and Formatters that turn an event into a string."

"Level checking happens at two places, deliberately. The **logger** level is a hot-path optimisation – debug is called millions of times in production, and checking before we build the event object means we never allocate or format a string we're going to throw away. The **appender** level is routing – everything to console, only errors to Sentry."

"Formatting is a strategy because structured JSON logging and human-readable console logging are both legitimate and depend on where the logs are going. Async is a **decorator** that wraps any appender and buffers – the underlying appender doesn't know or care that it's now asynchronous."

"The feature that makes a logging framework actually useful in production is hierarchical logger names. When I'm debugging a payment issue, I don't want to turn the whole application to debug and drown. I want to set app.payment to debug and leave everything else at info – one config change, no deploy. Names resolve from most specific to least, falling back to the root."

"On failure: a logger must never take down the application. If a file appender's disk is full, it should drop messages and report a metric, not throw into the caller's code path. Logging is observability, not business logic."

7. Follow-ups

అడిగేది	జవాబు
"Thread safety?"	Appender write ని synchronize చేయాలి, లేకపోతే lines కలిసిపోతాయి. మెరుగైనది – async appender తో ఒక queue + ఒక writer thread (lock contention పోతుంది)
"Async lo crash అయితే buffer lo ఉన్నది?"	పోతాయి. అందుకే FATAL/ERROR ని ఎప్పుడూ sync గా రాయడం. Trade-off: వేగం vs "చివరి log line" – crash debug కి అదే ముఖ్యం

అడిగేది	జవాబు
"Log rotation?"	<code>FileAppender</code> లోపల – size లేదా time ఆధారంగా. <code>app.log</code> → <code>app.log.1</code> → gzip. Appender బాధ్యత, logger కి తెలియదు
"Structured logging ఎందుకు?"	<code>"user 42 failed"</code> ని grep చేయలేం. <code>{"user":42,"event":"login_failed"}</code> ని query చేయొచ్చు. Log aggregation (ELK, Datadog) కి ఇది తప్పనిసరి
"Context propagation (request id)?"	<code>AsyncLocalStorage</code> (Node) / <code>MDC</code> (Java) – ప్రతి log call lo request id ని పంపనవసరం లేకుండా అది ఆటోమేటిక్ గా చేరుతుంది
"Sampling – చాలా ఎక్కువ logs ఉంటే?"	Rate limiter ని appender lo కలపడం (Problem 08!) – "ఒకే message ని నిమిషానికి 10 సార్లు మాత్రమే". Log storms ని ఆపడానికి ఇది తప్పనిసరి

Design a Notification Service

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: event bus, channels, templates, preferences, retry decorator

1. The Ask

"Design a notification service. When an order ships, the customer should get an email, an SMS and a push – based on what they've opted into."

HLD PROBLEM 05 తే తడా

HLD lo మనం అడిగింది – "రోజుకి బిలియన్ notifications, Kafka partitions, DLQ, circuit breaker". అది infrastructure ప్రశ్న.

ఇక్కడ ప్రశ్న – "Order service కి email గురించి తెలియకుండా ఎలా ఉంచాలి? కొత్త channel (WhatsApp) వస్తే ఎన్ని files మారతాయి? User preferences ని ఎక్కడ చూడాలి?" ఇది coupling ప్రశ్న. ఈ problem యొక్క గుండె – ఎవరికి ఎవరి గురించి తెలియాలి అనేదే.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఎవ channels?	Strategy interface	Email, SMS, Push
User preferences ఉన్నాయా?	ప్రతి send ముందు check	అవును, channel + event type ప్రకారం
Templates ఎలా?	Content ని code నుంచి వేరు చేయాలి	Per event type
Retry కావాలా?	Decorator vs channel లోపల	అవును, decorator గా
Sync నా async నా?	ఇప్పుడు sync, తర్వాత queue	Interface ఒకటే
Dedupe?	ఒకే event రెండుసార్లు వస్తే	అవును, idempotency key తో

3. Deep Dive – Coupling ని ఎలా విడగొట్టాలి

తప్పు మార్గం VS OBSERVER

✗ నేరుగా పిలవడం - ORDER SERVICE కి అన్నిటి గురించే తెలుసు



కొత్త channel (WhatsApp) వస్తే –

OrderService ని edit చేయాలి, PaymentService ని కూడా.
Event పంపే ప్రతి service నీ. అంటే 20 files.

✓ EVENT BUS - ఎవరికీ ఎవరి గురించే తెలియదు



OrderService కి notifications అనే మాట తెలియదు – అది కేవలం "order shipped" అని ప్రకటిస్తుంది

కొత్త channel అంటే ఒక కొత్త Channel class + config lo ఒక entry. Business services ఏమే మారవు.

4. Code

```
// ---- Observer: event bus ----
class EventBus {
  constructor() { this.handlers = new Map(); }

  subscribe(type, handler) {
    if (!this.handlers.has(type)) this.handlers.set(type, new Set());
    this.handlers.get(type).add(handler);
    return () => this.handlers.get(type).delete(handler); // unsubscribe function
  }

  publish(event) {
    const errors = [];
    for (const handler of this.handlers.get(event.type) ?? []) {
      // ఒక subscriber పడిపోతే మిగతావాళ్ళు అగకూడదు – ఇది Observer lo చాలా ముఖ్యమైన నియమం
      try { handler(event); } catch (e) { errors.push({ handler: handler.name, error: e.message }); }
    }
    return errors;
  }
}
```

```
// ---- Strategy: channels ----
class Channel {
  get name() { throw new Error('abstract'); }
  send(user, message) { throw new Error('abstract'); }
}

class MemoryChannel extends Channel { // test / demo కోసం
  constructor(name) { super(); this._name = name; this.sent = []; }
  get name() { return this._name; }
  send(user, message) { this.sent.push({ to: user.id, subject: message.subject }); }
}

class FlakyChannel extends Channel { // మొదటి N calls fail అవుతాయి
  constructor(inner, failTimes) { super(); this.inner = inner; this.left = failTimes;
}
```

```

    }
    get name() { return this.inner.name; }
    send(user, message) {
        if (this.left-- > 0) throw new Error('PROVIDER_TIMEOUT');
        return this.inner.send(user, message);
    }
}

// Decorator - ఏ channel నైనా retry చేయగలిగేలా చేయడం. Channel కి ఇది తెలియదు.
class RetryingChannel extends Channel {
    constructor(inner, attempts = 3) { super(); this.inner = inner; this.attempts = attempts; }
    get name() { return this.inner.name; }
    send(user, message) {
        let lastError;
        for (let i = 0; i < this.attempts; i++) {
            try { return this.inner.send(user, message); } catch (e) { lastError = e; }
        }
        throw lastError; // అన్ని ప్రయత్నాలు విఫలం
    }
}

```

```

// ---- Template Method: content ----
class Template {
    render(event, user) { return { subject: this.subject(event, user), body: this.body(event, user) }; }
    subject(event, user) { throw new Error('abstract'); }
    body(event, user) { throw new Error('abstract'); }
}

class OrderShippedTemplate extends Template {
    subject(e) { return `Order ${e.payload.orderId} has shipped`; }
    body(e, user) { return `Hi ${user.name}, order ${e.payload.orderId} is on its way.`; }
}

// ---- Preferences ----
class PreferenceService {
    constructor(prefs) { this.prefs = prefs; } // userId → { channels: [], muted: [] }
    channelsFor(user, eventType) {
        const p = this.prefs.get(user.id);
        if (!p || p.muted.includes(eventType)) return []; // opt-out ని ఇక్కడే గౌరవించడం
        return p.channels;
    }
}

```

```

class NotificationService {
    constructor({ channels, templates, prefs }) {
        Object.assign(this, { channels, templates, prefs });
        this.seen = new Set(); // idempotency
        this.failures = [];
    }

    handle(event) {
        const template = this.templates.get(event.type);
    }
}

```

```

        if (!template) return; // మనకి సంబంధం లేని event

        for (const user of event.payload.recipients) {
            const key = `${event.id}:${user.id}`;
            if (this.seen.has(key)) continue; // duplicate event → ఏమీ చేయము
            this.seen.add(key);

            const message = template.render(event, user);
            for (const channelName of this.prefs.channelsFor(user, event.type)) {
                const channel = this.channels.get(channelName);
                if (!channel) continue;
                try { channel.send(user, message); }
                catch (e) { this.failures.push({ user: user.id, channel: channelName, error: e.message }); }
            }
        }
    }
}

```

```

// ---- వాడుక ----
const emailInner = new MemoryChannel('email');
const email = new RetryingChannel(new FlakyChannel(emailInner, 2), 3); // 2 సార్లు fail, 3వసారి pass
const sms = new MemoryChannel('sms');

const prefs = new PreferenceService(new Map([
    ['u1', { channels: ['email', 'sms'], muted: [] }],
    ['u2', { channels: ['email'], muted: ['ORDER_SHIPPED'] }], // u2 ఈ event ని mute చేశాడు
]));

const notifier = new NotificationService({
    channels: new Map([['email', email], ['sms', sms]]),
    templates: new Map([['ORDER_SHIPPED', new OrderShippedTemplate()]]),
    prefs,
});

const bus = new EventBus();
bus.subscribe('ORDER_SHIPPED', (e) => notifier.handle(e));

const event = { id: 'evt-1', type: 'ORDER_SHIPPED', payload: {
    orderId: 'A-99',
    recipients: [{ id: 'u1', name: 'Yaswanth' }, { id: 'u2', name: 'Ravi' } ]},
};

bus.publish(event);
console.log(emailInner.sent); // [ { to: 'u1', subject: 'Order A-99 has shipped' } ] ← u2 mute
console.log(sms.sent); // [ { to: 'u1', subject: 'Order A-99 has shipped' } ]

bus.publish(event); // అదే event మళ్ళీ (Kafka retry లాంటిది)
console.log(emailInner.sent.length, notifier.failures.length); // 1 0
// ↑ dedupe పని చేసింది ↑ retry వల్ల ఏ failure నమోదు కాలేదు

```

ఈ DEMO LO నాలుగు విషయాలు ఏకకాలంలో నిరూపితమయ్యాయి

1. **Observer** – OrderService (publisher) కి notifications గురించి తెలియదు.
2. **Preferences** – u2 mute చేశాడు కాబట్టి అతనికి ఏమీ వెళ్ళలేదు, code lo ఒక్క if కూడా లేకుండా.
3. **Decorator** – email channel 2 సార్లు fail అయినా, retry wrapper వల్ల విజయవంతమైంది.
MemoryChannel కి retry గురించి తెలియదు.
4. **Idempotency** – అదే event మళ్ళీ వచ్చినా రెండో notification వెళ్ళలేదు.

5. Extensibility Test

Interviewer: "Add WhatsApp, and add quiet hours – no notifications between 10pm and 8am except OTP."

```
// 1) కొత్త channel – ఒక class + config lo ఒక entry. OrderService కదలదు.
class WhatsAppChannel extends Channel {
  get name() { return 'whatsapp'; }
  send(user, message) { /* WhatsApp Business API call */ }
}

// 2) Quiet hours – preferences lo ఒక decorator
class QuietHoursPreferences extends PreferenceService {
  constructor(inner, clock, urgentTypes = ['OTP', 'SECURITY_ALERT']) {
    super(inner.prefs); this.inner = inner; this.clock = clock; this.urgent =
    urgentTypes;
  }
  channelsFor(user, eventType) {
    const hour = new Date(this.clock.now()).getHours();
    const quiet = hour >= 22 || hour < 8;
    if (quiet && !this.urgent.includes(eventType)) return []; // వాయిదా / రద్దు
    return this.inner.channelsFor(user, eventType);
  }
}
```

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Observer	EventBus	Publisher కి subscribers గురించి తెలియకూడదు. కొత్త subscriber = సున్నా edits
Strategy	Channel	Email, SMS, push, WhatsApp – ఇవి ఖచ్చితంగా పెరుగుతాయి
Decorator	RetryingChannel , QuietHoursPreferences	Retry, rate limit, logging – వీటిని పేర్చేస్తున్నా
Template Method	Template.render()	Skeleton ఒకటే (subject + body), content subclass lo

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me scope: which channels, do we honour per-user preferences, are templates per event type, and do we need retries and deduplication? I'll build all of those synchronously, with the interface shaped so it can move behind a queue later."

"The question this design really answers is **who has to know about whom**. The naive version has the order service call an email service, an SMS service and a push service directly. The problem shows up the day you add WhatsApp: now you edit the order service, the payment service, and every other service that emits an event."

"So the order service publishes an event to a bus and knows nothing else. The notification service subscribes. Adding a channel touches one new class and one config entry, and no business code at all."

"One rule I'd enforce in the bus: if one subscriber throws, the others must still run. A publisher shouldn't be able to break unrelated listeners, so I catch per handler and collect errors rather than letting the first failure abort the fan-out."

"Channels are a strategy behind a common interface, and retry is a **decorator** rather than something inside each channel. That way retry logic exists once instead of three times, and each channel stays focused on talking to its provider. The same slot takes rate limiting or logging wrappers."

"Preferences get checked **before** any rendering or sending, because doing work for someone who opted out is pure waste – and in some jurisdictions sending anyway is a legal problem, not just a rude one."

"Deduplication uses an idempotency key of event ID plus user ID. This matters because in production the bus is at-least-once, so the same event will be redelivered. Without that key, a retry means the customer gets told twice that their order shipped."

"If you asked me for quiet hours, I'd add it as a preferences decorator with a carve-out for urgent types like OTP – with the clock injected, so I can test the ten-p.m. boundary without waiting for ten p.m."

"To scale this out, the notification service moves behind a queue and the same channel classes are used by workers. The class design doesn't change – which is the point of keeping the transport out of these objects."

7. Follow-ups

అడిగేది	జవాబు
"Async ఎలా మారుస్తారు?"	<code>handle()</code> event ని queue lo పెడుతుంది, workers అదే <code>NotificationService</code> ని పిలుస్తారు. ఒక్క class కూడా మారదు – ఇదే transport ని domain నుంచి వేరుగా ఉంచడం యొక్క లాభం

అడిగేది	జవాబు
"Dedupe set ఎప్పటికీ పెరుగుతుంది కదా?"	అవును – TTL ఉన్న store కావాలి (Redis <code>SET NX EX 86400</code>) లేదా LRU (Problem 07). In-memory <code>Set</code> interview demo కి మాత్రమే
"Templates ని ఎక్కడ ఉంచాలి?"	Code lo కాదు – DB/config lo, versioned. అప్పుడు marketing team deploy లేకుండా content మార్చొచ్చు. Rendering కి Handlebars లాంటిది
"Digest (10 likes = 1 notification)?"	Aggregation window – events ని buffer చేసి, window ముగిశాక ఒక summary notification. ఇది ఒక కొత్త subscriber, ఉన్నదాన్ని మార్చనవసరం లేదు
"Localization?"	Template ని <code>(eventType, locale)</code> తో lookup చేయడం. User preference lo locale
"Channel fallback (push fail → SMS)?"	ఒక <code>FallbackChannel(primary, secondary)</code> decorator. కానీ జాగ్రత్త – SMS ఖరీదు, కాబట్టి ఇది business నిర్ణయం, technical కాదు

Design Chess

Difficulty **Hard** · Frequency **మధ్యస్థం** · నేర్పే concepts: polymorphic rules, simulate-then-check, Command undo

1. The Ask

"Design a chess game. Model the board and pieces, and validate whether a move is legal."

ఈ **PROBLEM 10** రెండు వేర్వేరు రకాల నియమాలు ఉన్నాయి

Piece-specific నియమాలు – "గుర్రం L ఆకారంలో కదులుతుంది". ఇవి ఒక్కో piece కి ప్రత్యేకం → **polymorphism**.

Global నియమాలు – "ఈ కదలిక తర్వాత నా రాజు check lo ఉండకూడదు". ఇది **ఏ piece కీ చెందదు** – ఇది board యొక్క మొత్తం స్థితి గురించి. దీన్ని piece లోపల పెట్టడానికి ప్రయత్నిస్తే design గందరగోళం అవుతుంది.

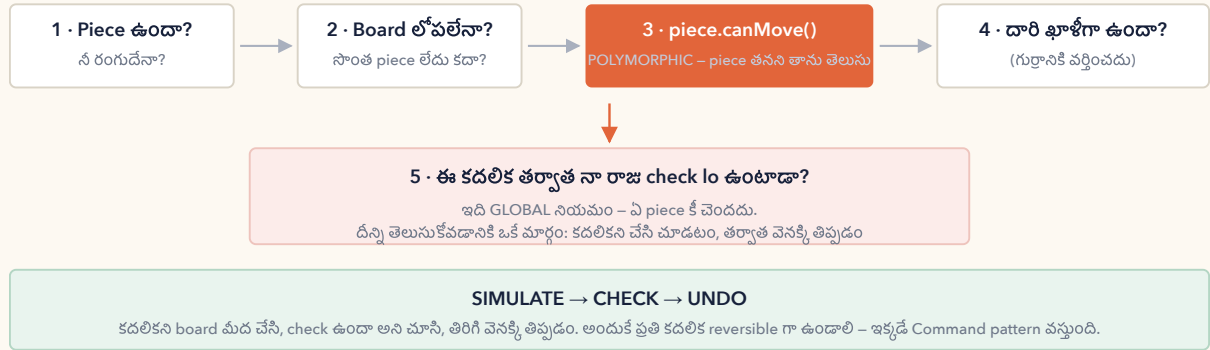
ఈ రెండింటినీ విడదీయడమే – **piece rules polymorphic గా, global rules ఒక pipeline గా** – ఈ problem యొక్క సరైన జవాబు.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
పూర్తి నియమాలా (castling, en passant, promotion)?	Scope భారీగా పెరుగుతుంది	Core moves + check. మిగతావి extensions
Checkmate/stalemate detection?	ఒక అదనపు algorithm	అవును (సూత్రప్రాయంగా)
AI opponent?	పూర్తిగా వేరే problem (minimax)	లేదు
Undo?	Command pattern	అవును
Timer / clock?	ఒక అదనపు component	లేదు
Multiplayer / networking?	ఇది LLD కాదు, HLD	లేదు

3. Deep Dive – Move Validation Pipeline

ఒక కదలిక చెల్లుబాటు అవ్వాలంటే 5 పరీక్షలు • క్రమం ముఖ్యం



4. Code

```
const Color = Object.freeze({ WHITE: 'W', BLACK: 'B' });
// Row 0 = rank 8 (నలుపు వైపు), row 7 = rank 1 (తెలుపు వైపు).

class Board {
  constructor() { this.grid = Array.from({ length: 8 }, () => Array(8).fill(null)); }
  at([r, c]) { return this.inside([r, c]) ? this.grid[r][c] : null; }
  place([r, c], piece) { this.grid[r][c] = piece; }
  inside([r, c]) { return r >= 0 && r < 8 && c >= 0 && c < 8; }

  // Sliding pieces (rook/bishop/queen) కోసం - మధ్యలో ఏమీ ఉండకూడదు
  pathClear([fr, fc], [tr, tc]) {
    const dr = Math.sign(tr - fr), dc = Math.sign(tc - fc);
    let r = fr + dr, c = fc + dc;
    while (r !== tr || c !== tc) { if (this.grid[r][c]) return false; r += dr; c += dc; }
    return true;
  }

  *pieces() {
    for (let r = 0; r < 8; r++) for (let c = 0; c < 8; c++) if (this.grid[r][c])
      yield [[r, c], this.grid[r][c]];
  }

  findKing(color) {
    for (const [pos, p] of this.pieces()) if (p.symbol === 'K' && p.color === color)
      return pos;
    return null;
  }
}
```

```
// ప్రతి piece తన నియమాన్ని తానే తెలుసుకుంటుంది. ఎక్కడా switch లేదు.
class Piece {
  constructor(color) { this.color = color; this.hasMoved = false; }
  get symbol() { throw new Error('abstract'); }
  canMove(board, from, to) { throw new Error('abstract'); }
}
```

```

class Rook extends Piece {
    get symbol() { return 'R'; }
    canMove(b, f, t) { return (f[0] === t[0] || f[1] === t[1]) && b.pathClear(f, t); }
}
class Bishop extends Piece {
    get symbol() { return 'B'; }
    canMove(b, f, t) { return Math.abs(t[0] - f[0]) === Math.abs(t[1] - f[1]) &&
b.pathClear(f, t); }
}
class Queen extends Piece {                                     // rook + bishop - కూర్పు ద్వారా
    get symbol() { return 'Q'; }
    canMove(b, f, t) {
        const straight = f[0] === t[0] || f[1] === t[1];
        const diagonal = Math.abs(t[0] - f[0]) === Math.abs(t[1] - f[1]);
        return (straight || diagonal) && b.pathClear(f, t);
    }
}
class Knight extends Piece {
    get symbol() { return 'N'; }
    canMove(b, f, t) {                                         // దూకుతుంది - path check అవసరం
        const dr = Math.abs(t[0] - f[0]), dc = Math.abs(t[1] - f[1]);
        return (dr === 1 && dc === 2) || (dr === 2 && dc === 1);
    }
}
class King extends Piece {
    get symbol() { return 'K'; }
    canMove(b, f, t) { return Math.abs(t[0] - f[0]) <= 1 && Math.abs(t[1] - f[1]) <= 1
&& (f[0] !== t[0] || f[1] !== t[1]); }
}
class Pawn extends Piece {
    get symbol() { return 'P'; }
    canMove(b, [fr, fc], [tr, tc]) {
        const dir = this.color === Color.WHITE ? -1 : 1;
        const startRow = this.color === Color.WHITE ? 6 : 1;
        const target = b.at([tr, tc]);
        if (fc === tc && !target) {                             // ముందుకి
            if (tr === fr + dir) return true;
            return fr === startRow && tr === fr + 2 * dir && !b.at([fr + dir, fc]);
        }
        return Math.abs(tc - fc) === 1 && tr === fr + dir && Boolean(target);    // వికర్ణ
capture
    }
}

```

```

// Command pattern - ప్రతి కదలిక reversible. Undo కీ, simulation కీ ఇదే వాడతాం.
class MoveCommand {
    constructor(board, from, to) { Object.assign(this, { board, from, to }); }
    execute() {
        this.piece = this.board.at(this.from);
        this.captured = this.board.at(this.to);
        this.hadMoved = this.piece.hasMoved;
        this.board.place(this.to, this.piece);
        this.board.place(this.from, null);
        this.piece.hasMoved = true;
        return this;
    }
}

```

```

undo() {
  this.board.place(this.from, this.piece);
  this.board.place(this.to, this.captured);
  this.piece.hasMoved = this.hadMoved;
}
}

```

```

class Game {
  constructor(board, turn = Color.WHITE) { this.board = board; this.turn = turn;
  this.history = []; }

  isInCheck(color) {
    const king = this.board.findKing(color);
    if (!king) return false;
    for (const [pos, piece] of this.board.pieces()) {
      if (piece.color !== color && piece.canMove(this.board, pos, king)) return true;
    }
    return false;
  }

  // చెల్లుబాటు అయితే null, లేకపోతే కారణం.
  validate(from, to) {
    const piece = this.board.at(from);
    if (!piece) return 'NO_PIECE';
    if (piece.color !== this.turn) return 'NOT_YOUR_TURN';
    if (!this.board.inside(to)) return 'OFF_BOARD';

    const target = this.board.at(to);
    if (target && target.color === piece.color) return 'OWN_PIECE';
    if (!piece.canMove(this.board, from, to)) return 'ILLEGAL_FOR_PIECE';

    // SIMULATE → CHECK → UNDO. ఇదే global నియమాన్ని అమలు చేసే ఏకైక మార్గం.
    const move = new MoveCommand(this.board, from, to).execute();
    const exposed = this.isInCheck(piece.color);
    move.undo();
    return exposed ? 'KING_IN_CHECK' : null;
  }

  move(from, to) {
    const error = this.validate(from, to);
    if (error) throw new Error(error);
    this.history.push(new MoveCommand(this.board, from, to).execute());
    this.turn = this.turn === Color.WHITE ? Color.BLACK : Color.WHITE;
  }

  undo() {
    const last = this.history.pop();
    if (!last) return;
    last.undo();
    this.turn = this.turn === Color.WHITE ? Color.BLACK : Color.WHITE;
  }
}

```

```

// ---- వాడుక: pinned piece ----
// తెల్ల రాజు e1, తెల్ల ఏనుగు e2, నల్ల ఏనుగు e8. e-file మీద తెల్ల ఏనుగు "pinned".
const b = new Board();

```

```

b.place([7, 4], new King(Color.WHITE)); // e1
b.place([6, 4], new Rook(Color.WHITE)); // e2
b.place([0, 4], new Rook(Color.BLACK)); // e8
b.place([0, 0], new King(Color.BLACK)); // a8

const game = new Game(b);

console.log(game.validate([6, 4], [6, 0])); // 'KING_IN_CHECK' ← e-file వదిలితే రాజు
బయటపడతాడు
console.log(game.validate([6, 4], [5, 4])); // null ← e-file మీదే ఉంటే
చెల్లుతుంది
console.log(game.validate([6, 4], [5, 3])); // 'ILLEGAL_FOR_PIECE' ← ఏనుగు వికర్ణంగా
వెళ్ళదు
console.log(game.validate([7, 4], [6, 4])); // 'OWN_PIECE' ← సొంత piece మీదికి

game.move([6, 4], [5, 4]);
console.log(game.turn, b.at([5, 4]).symbol); // B R
game.undo();
console.log(game.turn, b.at([6, 4]).symbol); // W R

```

ఈ DESIGN LO మూడు అందమైన విషయాలు

1. **Game lo ఒక్క if (piece instanceof Rook) కూడా లేదు.** ప్రతి piece తన నియమాన్ని తానే చెప్తుంది. కొత్త piece (fairy chess) = ఒక కొత్త class.
2. **Command pattern ని రెండు పనులకి వాడాలి** – undo కి, మరియు **validation** కి. "Simulate then undo" చేయాలంటే కదలిక reversible కావాలి; అది ఇప్పటికే ఉంది కాబట్టి అదనపు code అవసరం లేదు. ఒక abstraction, రెండు ప్రయోజనాలు.
3. **isInCheck కూడా canMove నే వాడుతుంది** – "నా రాజుని ఎవరైనా తినగలరా?" అనేది "వాళ్ళు ఆ గడికి కదలగలరా?" అనే ప్రశ్న. Logic duplicate కాలేదు.

5. Extensibility Test

Interviewer: "Add castling and en passant."

ఇక్కడ నిజాయితీగా ఉండండి

Castling మరియు en passant ఈ design lo సరిగ్గా ఇమడవు, మరియు అది చెప్పడమే సరైన జవాబు – ఎందుకంటే ఇవి ఒక్క piece కి సంబంధించినవి కావు:

- **Castling** = రాజు + ఏనుగు రెండూ కదులుతాయి, రెండూ ఇంతకుముందు కదలకుండా ఉండాలి, మధ్య గడులు ఖాళీగా ఉండాలి, రాజు check గుండా వెళ్ళకూడదు. ఇది ఒక ప్రత్యేక move type.
- **En passant** = గత కదలిక ఏమిటో తెలియాలి. అంటే ఇది **state-dependent**, board మాత్రమే చాలదు.

పరిష్కారం: MoveCommand ని ఒక hierarchy చేయడం – NormalMove, CastlingMove, EnPassantMove, PromotionMove. ఒక MoveGenerator ఏ రకమో నిర్ణయిస్తుంది. hasMoved flag ని నేను ఇప్పటికే పెట్టాను – అది castling కోసమే.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Polymorphism / Strategy	<code>Piece.canMove()</code>	ఒక్క piece కి ఒక్క నియమం. Switch statement ని పూర్తిగా తొలగించింది
Command	<code>MoveCommand</code>	Undo, replay, మరియు validation simulation – మూడూ ఒకే abstraction తో
Template Method (సూచించినది)	Validation pipeline	అడుగుల క్రమం స్థిరం, ఒక్క అడుగు మార్చు
Factory (extension)	<code>MoveGenerator</code>	Normal / castling / en passant – ఏ command object సృష్టించాలో నిర్ణయించడం

🔧 INTERVIEW SCRIPT · ఇలా చెప్పుండి (ENGLISH)

"First, scope. Full rules including castling, en passant and promotion, or core movement plus check detection? And do we need an engine or just legality? I'll build the board, pieces, check detection and undo, and then talk about how the special moves fit – because they're where the design gets interesting."

"The key modelling insight is that there are **two different kinds of rule**. Piece rules – a knight moves in an L – belong to the piece, and I express them polymorphically with a canMove method per class. That eliminates the giant switch statement that this problem invites."

"But 'you may not leave your own king in check' belongs to **no piece at all**. It's a property of the whole board after the move. Trying to push that into the piece classes is how this design usually goes wrong."

"So validation is a pipeline: is there a piece and is it yours, is the destination on the board and not your own piece, does the piece's own rule allow it, is the path clear – and finally the global check rule."

"That last one can only be answered by **simulating**: make the move, ask whether your king is attacked, then undo. Which means every move has to be reversible – and that's exactly what the Command pattern gives me. I wanted commands anyway for undo, so validation gets the capability for free. One abstraction, two uses."

"Check detection reuses canMove as well: 'is my king attacked' is just 'can any enemy piece move to my king's square'. So the movement rules are written once and serve both move validation and check detection."

"If you ask me to add castling and en passant, I'd say honestly that they don't fit the current model – and that's the interesting part. Castling moves **two** pieces and depends on move history. En passant depends on what the **previous** move was, so the board alone isn't sufficient state. The right answer is to make MoveCommand a hierarchy – normal, castling, en-passant, promotion – with a generator deciding which to construct. You'll notice I already track hasMoved on each piece; that's there for castling."

"Checkmate then falls out of what I have: you're in check, and every legal move you could make still leaves you in check. Since I can already enumerate and validate moves, checkmate is a loop rather than new machinery."

7. Follow-ups

అడిగేది	జవాబు
"Checkmate ఎలా detect చేస్తారు?"	<code>isInCheck(color)</code> నిజం, మరియు ఆ రంగు యొక్క ప్రతి piece యొక్క ప్రతి కదలిక validate చేసినా అన్నీ fail. Stalemate = check లేదు కానీ ఏ చెల్లుబాటు కదలికా లేదు
"అది నెమ్మది కాదా?"	16 pieces × ~28 గడులు = ~450 simulations. ప్రతిదానికీ <code>isInCheck</code> = 32 piece checks. మొత్తం ~15,000 operations – ఒక కదలికకి ఇది క్షణంలో అవుతుంది. Engine రాస్తూంటే bitboards కావాలి, కానీ game UI కి ఇది సరిపోతుంది

అడిగేది	జవాబు
"Board ని ఎలా represent చేయాలి?"	2D array చదవడానికి సులభం. Engine కి bitboards (64-bit integer per piece type) – bitwise operations తో అపారమైన వేగం. Interview lo 2D array చాలు, bitboards ని ప్రస్తావిస్తే బోనస్
"Draw conditions?"	50-move rule (counter), threefold repetition (position hashes – Zobrist hashing), insufficient material. వీటికి game-level state కావాలి, board కాదు
"Networked multiplayer?"	ఇది LLD కాదు. కానీ Command objects సహజంగా serializable – move ని network మీద పంపి రెండు వైపులా apply చేయొచ్చు. అదే lock-step multiplayer
"Move history / PGN export?"	history ఇప్పటికే commands జాబితా. ప్రతి command కి toNotation() జోడిస్తే PGN వస్తుంది

Design Undo / Redo (Text Editor)

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: Command vs Memento, command merging, stack invariants

1. The Ask

"Design undo and redo for a text editor. Typing, deleting, pasting – all undoable."

ఈ **PROBLEM 10** రెండు సరైన జవాబులు ఉన్నాయి

Command – "ఏం జరిగిందో" store చేసి, దాన్ని తిప్పికొట్టడం. ("position 5 lo 'abc' చేర్చాను → undo అంటే position 5 నుంచి 3 అక్షరాలు తీసేయడం")

Memento – "ఏం ఉండేదో" store చేసి, దాన్ని తిరిగి పెట్టడం. ("undo ముందు document ఇలా ఉండేది → దాన్ని తిరిగి పెట్టు")

రెండూ సరైనవే. Interviewer చూసేది – మీరు రెండూ తెలిసి, ఎందుకు ఒకటి ఎంచుకున్నారో చెప్పగలరా అన్నది.

2. Command vs Memento

ఒకే **PROBLEM**, రెండు దృక్పథాలు

COMMAND · "ఏం జరిగింది"

```
InsertCommand(pos: 5, text: "abc")
undo() → delete(5, 3)
```

- ✓ Memory చాలా తక్కువ – ఒక్క operation మాత్రమే
- ✓ 1 GB document కి కూడా ఇదే
- ✗ ప్రతి operation కి inverse రాయాలి

MEMENTO · "ఏం ఉండేది"

```
snapshot = { text: "Hello world" }
undo() → restore(snapshot)
```

- ✓ చాలా సులభం – inverse అల్చించనవసరం లేదు
- ✓ ఏ operation కైనా పని చేస్తుంది
- ✗ ప్రతి అక్షరానికి పూర్తి copy – పెద్ద docs కి అసాధ్యం

నిజమైన editors రెండూ కలిపి వాడతాయి

Commands ని stack lo, కానీ ప్రతి 100 commands కి ఒక snapshot. Undo చాలా వెనక్కి వెళ్ళాలంటే – దగ్గరి snapshot నుంచి మొదలుపెట్టి ముందుకి replay. Git కూడా ఇదే చేస్తుంది (packfiles lo full objects + deltas).

నా CHOICE

"నేను **Command** ఎంచుకుంటాను. కారణం memory – 50 MB document lo ఒక అక్షరం type చేసినప్పుడు 50 MB snapshot తీయలేను. Command lo అది 20 bytes. Inverse రాయడం అదనపు పని కానీ operations పరిమితం (insert, delete, replace, format) కాబట్టి అది ఒకసారి చేసే పని.

కానీ ఒక సూక్ష్మత – **కొన్ని operations కి inverse ని లెక్కించలేం**. ఉదాహరణకి 'delete' – తీసేసిన text ఏమిటో తెలియకుండా undo చేయలేం. అందుకే DeleteCommand execute చేసేటప్పుడు తీసేసిన text ని **తనలో దాచుకుంటుంది**. అంటే అది ఒక చిన్న memento – **ఆచరణలో రెండు patterns కలిసిపోతాయి**."

3. Code

```
class TextDocument {
  constructor(text = '') { this.text = text; }
  insert(pos, str) { this.text = this.text.slice(0, pos) + str +
this.text.slice(pos); }
  delete(pos, len) {
    const removed = this.text.slice(pos, pos + len);
    this.text = this.text.slice(0, pos) + this.text.slice(pos + len);
    return removed;
  }
}

class Command {
  execute() { throw new Error('abstract'); }
  undo() { throw new Error('abstract'); }
  canMergeWith(next) { return false; } // typing ని ఒకే undo step గా చేయడానికి
  mergeWith(next) { throw new Error('not mergeable'); }
}

class InsertCommand extends Command {
  constructor(doc, pos, str) { super(); Object.assign(this, { doc, pos, str }); }
  execute() { this.doc.insert(this.pos, this.str); }
  undo() { this.doc.delete(this.pos, this.str.length); }

  // వరుసగా type చేస్తున్నప్పుడు మాత్రమే merge. Space/newline దగ్గర ఆపడం –
  // అది సహజమైన "word boundary", users అలానే ఆలోచిస్తారు.
  canMergeWith(next) {
    return next instanceof InsertCommand
      && next.doc === this.doc
      && next.pos === this.pos + this.str.length
      && !/\s/.test(this.str.at(-1));
  }

  mergeWith(next) { return new InsertCommand(this.doc, this.pos, this.str +
next.str); }
}

class DeleteCommand extends Command {
  constructor(doc, pos, len) { super(); Object.assign(this, { doc, pos, len });
  this.removed = null; }
  execute() { this.removed = this.doc.delete(this.pos, this.len); } // ← ఇక్కడ
  memento దాగి ఉంది
}
```

```

    undo()    { this.doc.insert(this.pos, this.removed); }
}

```

```

class History {
  constructor(limit = 100) { this.undoStack = []; this.redoStack = []; this.limit =
limit; }

  execute(command) {
    command.execute();
    const last = this.undoStack.at(-1);
    if (last && last.canMergeWith(command)) {
      this.undoStack[this.undoStack.length - 1] = last.mergeWith(command);
    } else {
      this.undoStack.push(command);
      if (this.undoStack.length > this.limit) this.undoStack.shift(); // జ్ఞాపకశక్తికి ఒక
హద్దు
    }
    this.redoStack.length = 0; // ← కీలకం: కొత్త action వస్తే redo చరిత్ర చెల్లదు
  }

  undo() {
    const command = this.undoStack.pop();
    if (!command) return false;
    command.undo();
    this.redoStack.push(command);
    return true;
  }

  redo() {
    const command = this.redoStack.pop();
    if (!command) return false;
    command.execute();
    this.undoStack.push(command);
    return true;
  }
}

```

```

// ---- వాడుక ----
const doc = new TextDocument('');
const history = new History();

history.execute(new InsertCommand(doc, 0, 'H'));
history.execute(new InsertCommand(doc, 1, 'e'));
history.execute(new InsertCommand(doc, 2, 'llo'));
console.log(doc.text, history.undoStack.length);
// Hello 1 ← మూడు keystrokes ఒకే undo step గా merge అయ్యాయి

history.execute(new InsertCommand(doc, 5, ' ')); // space ఇంకా "Hello" తోనే
కలుస్తుంది
history.execute(new InsertCommand(doc, 6, 'world')); // కానీ దాని తర్వాతిది కొత్త step
console.log(doc.text, history.undoStack.length);
// Hello world 2 ← ["Hello ", "world"] – సరిగ్గా రెండు పదాలు, రెండు undo steps

history.execute(new DeleteCommand(doc, 0, 6));
console.log(doc.text); // world

```

```

history.undo(); console.log(doc.text); // Hello world ← delete
తిరిగిచ్చింది
history.undo(); console.log(doc.text); // "Hello " ← 'world' మొత్తం
ఒకసారి పోయింది
history.redo(); console.log(doc.text); // Hello world
history.execute(new InsertCommand(doc, 11, '!')); // కొత్త action
console.log(doc.text, history.redoStack.length); // Hello world! 0 ← redo చరిత్ర
తుడిచిపెట్టుకుపోయింది

```

REDOSTACK.LENGTH = 0` - ఈ ఒక్క LINE ఎందుకు ముఖ్యం

User "Hello world" రాశాడు, undo చేసి "Hello" కి వచ్చాడు, ఇప్పుడు "Hi" అని type చేశాడు. ఇప్పుడు redo నొక్కితే ఏం జరగాలి? **ఏమీ కాకూడదు** - ఎందుకంటే "world" అనేది ఇప్పుడు ఉన్న చరిత్రలో భాగం కాదు. ఆ శాఖ (branch) రద్దయింది.

ఇది మర్చిపోతే - redo చేసినప్పుడు document విచిత్రమైన, అసాధ్యమైన స్థితికి వెళ్తుంది. **ఇది undo/redo lo అత్యంత సాధారణమైన bug**, మరియు interview lo దీన్ని ప్రస్తావిస్తే మీరు దీన్ని నిజంగా రాశారని అర్థమవుతుంది.

4. Command Merging – ఇది ఎందుకు అవసరం

MERGING తక్కువ

"Hello world" అని type చేస్తే అది **11 commands**. User Ctrl+Z నొక్కితే ఒక్క అక్షరం మాత్రమే పోతుంది - 11 సార్లు నొక్కాలి. ఏ editor కూడా అలా ప్రవర్తించదు.

Merge నియమాలు (నిజమైన editors వాడేవి): (1) వరుస positions అయితేనే, (2) ఒకే రకమైన operation అయితేనే, (3) space/newline దగ్గర ఆపడం, (4) cursor వేరే చోటికి వెళ్తే ఆపడం, (5) కొంత సమయం (~1 సెకను) విరామం వస్తే ఆపడం. ఈ నియమాలు **user ఆలోచించే "ఒక పని" ని** అనుకరిస్తాయి.

5. Extensibility Test

Interviewer: "Now the editor supports formatting (bold, colour). Does your design hold?"

```

// ✓ కొత్త command classes. History, Document ఏవీ మారలేదు.
class FormatCommand extends Command {
  constructor(doc, from, to, attribute) { super(); Object.assign(this, { doc, from, to, attribute }); }
  execute() { this.previous = this.doc.getAttributes(this.from, this.to);
  this.doc.applyAttribute(this.from, this.to, this.attribute); }
  undo() { this.doc.setAttributes(this.from, this.to, this.previous); } // ←
  memento మళ్ళీ
}

// బహుళ commands ని ఒకే undo step గా (Composite)
class MacroCommand extends Command {
  constructor(commands) { super(); this.commands = commands; }
  execute() { for (const c of this.commands) c.execute(); }
  undo() { for (const c of [...this.commands].reverse()) c.undo(); } // ← వ్యతిరేక క్రమం!
}

```

```
}  
// వాడుక: "Find and replace all" = 50 replacements, కానీ ఒకే Ctrl+Z
```

`MACROCOMMAND.UNDO()` LO `REVERSE()` – ఎందుకు

Commands వరుసగా అమలయ్యాయి, ప్రతిదీ తర్వాతిదాని కోసం స్థితిని మార్చింది. Undo చేసేటప్పుడు అదే క్రమంలో వెళ్తే positions తప్పుతాయి. **ఎప్పుడూ వ్యతిరేక క్రమంలో undo చేయాలి** – ఇది database transaction rollback, saga compensation – అన్నిచోట్లా ఒకే నియమం.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Command	ప్రతి operation ఒక object	Undo, redo, macro, logging, replay – అన్నీ ఒకే abstraction తో
Memento (పాక్షికంగా)	<code>DeleteCommand.removed</code> , <code>FormatCommand.previous</code>	Inverse ని లెక్కించలేని చోట పాత state ని దాచుకోవడం
Composite	<code>MacroCommand</code>	పలు commands ఒక command లా ప్రవర్తిస్తాయి
Stack invariant	<code>redoStack.length = 0</code>	Pattern కాదు, కానీ correctness కి ఇదే కీలకం

🔧 INTERVIEW SCRIPT · ఇలా చెప్పుండి (ENGLISH)

"There are two legitimate approaches here and I'd like to compare them, because the choice depends entirely on document size."

"**Memento** stores what the document looked like before – undo just restores a snapshot. It's beautifully simple and works for any operation without thinking about inverses. But it copies the whole document per edit, so on a fifty-megabyte file, typing one character costs fifty megabytes. That's not viable."

"**Command** stores what happened – insert three characters at position five – and undo applies the inverse. Twenty bytes instead of fifty megabytes. The cost is that I have to define an inverse for every operation, but the operation set is small and fixed."

"I'd choose Command. But I want to point out where the two blend: some operations have no computable inverse. Undoing a delete requires knowing **what** was deleted, so the delete command captures the removed text when it executes. That captured text is a small memento living inside a command. In practice the patterns aren't alternatives so much as different points on a spectrum."

"The structure is two stacks. Executing pushes onto undo and – crucially – **clears redo**. That single line is the most common bug in undo implementations. If you type, undo, then type something different, redo must do nothing, because that future no longer exists. Without clearing it, redo drags the document into a state that was never reachable."

"Command **merging** is what makes this usable rather than technically correct. Typing 'Hello world' is eleven insert commands, and no user expects eleven presses of Ctrl-Z. So consecutive inserts at adjacent positions merge into one undo step, and I break the merge at whitespace – because that matches how people think about a unit of work. Real editors also break on cursor movement and on a pause of about a second."

"For extensibility: formatting becomes another command class, and 'replace all' becomes a **macro** command holding fifty sub-commands that undo as one. One detail there – the macro must undo its children in **reverse** order, because each one changed the state the next depended on. That's the same rule as rolling back a transaction or compensating a saga."

"And I'd bound the undo stack, because unbounded history is a memory leak in an application people leave open for weeks."

7. Follow-ups

అడిగేది	జవాబు
"Undo చరిత్రని persist చేయాలంటే?"	Commands ని serializable గా చేయడం (type + params). File మూసి తెరిచినా undo చరిత్ర ఉంటుంది. Memento అయితే ఇది భారీ
"Collaborative editing lo undo?"	చాలా కష్టం – "నా చివరి కదలిక" ఇప్పుడు వేరేవాళ్ళ edits కింద ఉంది. దీన్ని selective undo అంటారు, position ని transform చేయాలి (Problem 12 in HLD book – OT/CRDT)

అడిగేది	జవాబు
"Memory ఎంత?"	Command లో operation + parameters మాత్రమే. 100 commands × ~50 B = 5 KB. Snapshot approach lo అదే 100 × document size
"Snapshots ని ఎప్పుడు కలపాలి?"	Undo చాలా వెనక్కి వెళ్ళే feature ఉంటే (version history). ప్రతి 100 commands కి snapshot → replay దూరాన్ని పరిమితం చేస్తుంది
"Command fail అయితే?"	<code>execute()</code> throw చేస్తే stack lo push చేయకూడదు. లేకపోతే undo చేయనిదాన్ని undo చేస్తాం
"ఇదే pattern ఇంకెక్కడ?"	Photoshop history, IDE refactoring undo, database transaction log, event sourcing, game replay, chess (Problem 11) – అన్నీ ఒకే ఆలోచన

Design an In-Memory File System

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: Composite, recursion, Visitor, path resolution

1. The Ask

"Design an in-memory file system. Create directories and files, navigate paths, compute sizes, search."

ఈ **PROBLEM** యొక్క ఒకే ఒక్క ఆలోచన

"ఒక directory యొక్క size ఎంత?" అని అడిగితే – దాని లోపల files ఉన్నాయి, ఇంకా directories ఉన్నాయి, వాటిలో మళ్ళీ... ఈ recursion ని `if (node is File) ... else if (node is Directory) ...` తో రాస్తే code ప్రతిచోటా ఆ if-else ని repeat చేస్తుంది.

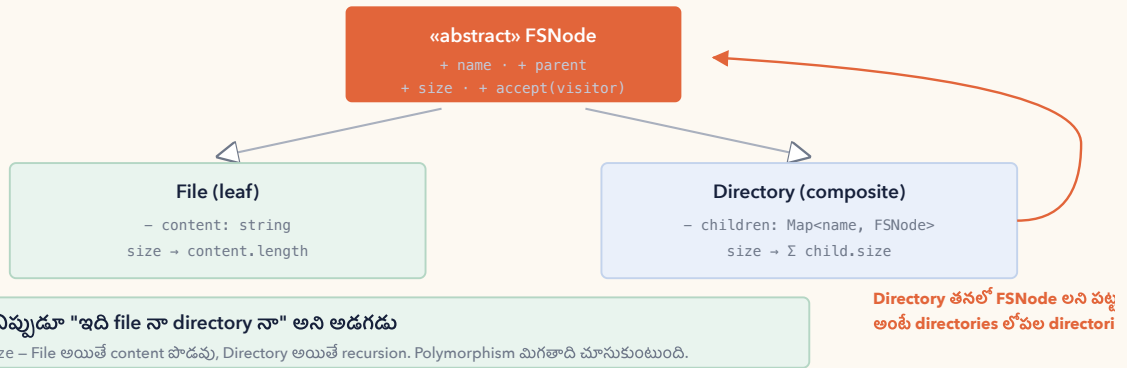
Composite pattern చెప్పేది ఒక్కటే: **leaf మరియు container ని ఒకే interface కింద తీసుకురా**. అప్పుడు caller "ఇది file నా directory నా" అని అడగనవసరం లేదు – `node.size` అంటే చాలు.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఏవి operations?	API surface	mkdir, write, read, ls, rm, find
Absolute paths మాత్రమేనా, relative కూడానా?	<code>.</code> <code>..</code> handling	రెండూ
Symlinks?	Cycle detection కావాలి	ఇప్పుడు వద్దు
Permissions?	ప్రతి node కి owner + mode	Extension గా
File size ఎంత?	In-memory కాబట్టి string చాలు	చిన్నవి
Thread safety?	Tree mutations concurrent	ప్రస్తావిస్తాం

3. Deep Dive – Composite

ఒకే INTERFACE, రెండు రకాల NODES



4. Code

```
class FSNode {
    constructor(name) { this.name = name; this.parent = null; }
    get size() { throw new Error('abstract'); }
    accept(visitor) { throw new Error('abstract'); }
    get path() {
        if (!this.parent) return '/';
        const parentPath = this.parent.path;
        return parentPath === '/' ? `/${this.name}` : `${parentPath}/${this.name}`;
    }
}

class FileNode extends FSNode { // LEAF
    constructor(name, content = '') { super(name); this.content = content; }
    get size() { return this.content.length; }
    accept(visitor) { return visitor.visitFile(this); }
}

class DirectoryNode extends FSNode { // COMPOSITE
    constructor(name) { super(name); this.children = new Map(); }

    add(node) {
        if (this.children.has(node.name)) throw new Error(`EXISTS:${node.name}`);
        node.parent = this;
        this.children.set(node.name, node);
        return node;
    }

    remove(name) {
        const node = this.children.get(name);
        if (!node) throw new Error(`NOT_FOUND:${name}`);
        node.parent = null;
        this.children.delete(name);
        return node;
    }

    get(name) { return this.children.get(name) ?? null; }

    // ఇదే composite యొక్క గుండె – leaf నా composite నా అని అడగకుండా recursion
    get size() { return [...this.children.values()].reduce((sum, c) => sum + c.size,
```

```

    0); }
    accept(visitor) { return visitor.visitDirectory(this); }
}

```

```

class FileSystem {
    constructor() { this.root = new DirectoryNode(''); }

    #segments(path) { return path.split('/').filter((s) => s.length > 0 && s !== '.'); }

    // Path ని resolve చేయడం. create=true అయితే లేని directories ని సృష్టిస్తుంది (mkdir -p).
    #walk(path, { create = false } = {}) {
        let node = this.root;
        for (const segment of this.#segments(path)) {
            if (segment === '..') { node = node.parent ?? this.root; continue; }
            let next = node.get(segment);
            if (!next) {
                if (!create) return null;
                next = node.add(new DirectoryNode(segment));
            }
            if (next instanceof FileNode) return null; // file గుండా వెళ్ళలేం
            node = next;
        }
        return node;
    }

    mkdirp(path) { return this.#walk(path, { create: true }); }

    writeFile(path, content) {
        const segments = this.#segments(path);
        const fileName = segments.pop();
        const dir = this.#walk(segments.join('/'), { create: true });
        const existing = dir.get(fileName);
        if (existing instanceof FileNode) { existing.content = content; return existing; }
        if (existing) throw new Error(`IS_DIRECTORY:${fileName}`);
        return dir.add(new FileNode(fileName, content));
    }

    resolve(path) {
        const segments = this.#segments(path);
        const last = segments.pop();
        if (last === undefined) return this.root;
        const dir = this.#walk(segments.join('/'));
        return dir ? dir.get(last) : null;
    }

    ls(path) {
        const node = this.resolve(path);
        if (!(node instanceof DirectoryNode)) throw new Error('NOT_A_DIRECTORY');
        return [...node.children.keys()].sort();
    }

    rm(path) {
        const node = this.resolve(path);
        if (!node || !node.parent) throw new Error('NOT_FOUND');
        return node.parent.remove(node.name); // subtree మొత్తం ఒక్కసారి పోతుంది
    }
}

```

```

    }
}

```

```

// ---- Visitor: tree మీద కొత్త operations, nodes ని మార్చకుండా ----
class FSVisitor {
    visitFile(file) {}
    visitDirectory(dir) {}
}

class TreePrinter extends FSVisitor {
    constructor() { super(); this.lines = []; this.depth = 0; }
    visitFile(f) { this.lines.push(`${' '.repeat(this.depth)}${f.name} (${f.size})`); }
    visitDirectory(d) {
        this.lines.push(`${' '.repeat(this.depth)}${d.parent ? `${d.name}/` : '/'}`);
        this.depth += 1;
        for (const child of d.children.values()) child.accept(this);
        this.depth -= 1;
    }
}

class SearchVisitor extends FSVisitor {
    constructor(predicate) { super(); this.predicate = predicate; this.found = []; }
    visitFile(f) { if (this.predicate(f)) this.found.push(f.path); }
    visitDirectory(d) { for (const child of d.children.values()) child.accept(this); }
}

```

```

// ---- వాడుక ----
const fs = new FileSystem();
fs.mkdirp('/home/yaswanth/docs');
fs.writeFile('/home/yaswanth/docs/notes.txt', 'hello world'); // 11 bytes
fs.writeFile('/home/yaswanth/todo.md', '# todo'); // 6 bytes
fs.writeFile('/tmp/cache.bin', 'xxx'); // 4 bytes

console.log(fs.ls('/home/yaswanth')); // [ 'docs', 'todo.md' ]
console.log(fs.resolve('/home').size); // 17 ← recursion: 11 + 6
console.log(fs.root.size); // 21 ← + /tmp lo 4
console.log(fs.resolve('/home/yaswanth/docs/notes.txt').path); //
/home/yaswanth/docs/notes.txt

const printer = new TreePrinter();
fs.root.accept(printer);
console.log(printer.lines.join('\n'));
// /
//   home/
//     yaswanth/
//       docs/
//         notes.txt (11)
//         todo.md (6)
//   tmp/
//     cache.bin (4)

const search = new SearchVisitor((f) => f.name.endsWith('.md'));
fs.root.accept(search);
console.log(search.found); // [ '/home/yaswanth/todo.md' ]

```

```
fs.rm('/home/yaswanth/docs');
console.log(fs.resolve('/home').size); // 6 ← subtree మొత్తం పోయింది
```

ఈ CODE LO COMPOSITE యొక్క రెండు బహుమతులు

1. **size ఎక్కడా if-else లేకుండా recursion చేసింది.** `DirectoryNode.size` తన children యొక్క `size` ని అడుగుతుంది – అవి files కావచ్చు, directories కావచ్చు, దానికి తేడా తెలియదు.
2. **rm ఒక్క line.** Directory తీసేస్తే దాని subtree మొత్తం ఒక్కసారే పోతుంది – recursive delete రాయనవసరం లేదు, ఎందుకంటే tree ownership ద్వారానే అది జరుగుతుంది.

5. Visitor ఎందుకు – ఇది ఒక ముఖ్యమైన ప్రశ్న

"PRINT, SEARCH, DU, CHMOD – వటన్నిటినీ FSNode LO METHODS గా పెట్టొచ్చు కదా?"

పెట్టొచ్చు – కానీ అప్పుడు ప్రతి కొత్త operation కీ `FSNode`, `FileNode`, `DirectoryNode` మూడింటినీ edit చేయాలి. Node classes ఉబ్బిపోతాయి, మరియు అవి "నేను ఏమిటి" (data) కి బదులు "నా మీద ఏం చేయొచ్చు" (behaviour) తో నిండిపోతాయి.

Visitor ఆ దిశని తిప్పేస్తుంది: కొత్త operation = ఒక కొత్త visitor class, node classes lo సున్నా మార్పు.

కానీ ధర ఉంది: కొత్త node type (ఉదా. `SymLinkNode`) జోడిస్తే – ప్రతి visitor ని edit చేయాలి. అంటే: operations తరచుగా మారితే **Visitor**; node types తరచుగా మారితే **methods**. File system lo node types దాదాపు ఎప్పటికీ మారవు, operations పెరుగుతూనే ఉంటాయి – అందుకే ఇక్కడ Visitor సరైనది.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Composite	<code>FSNode</code> → File / Directory	Leaf మరియు container ని ఒకేలా చూడటం. Recursion ఉచితం
Visitor	<code>TreePrinter</code> , <code>SearchVisitor</code>	కొత్త operations node classes ని ముట్టుకోకుండా
Iterator (సూచించినది)	<code>*pieces()</code> లాంటి generator	Tree traversal ని ఒకసారి రాసి అందరూ వాడటం

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me confirm the operations – mkdir, write, read, ls, rm, find – and whether we need symlinks or permissions. I'll build the core tree and treat symlinks as an extension, because they introduce cycles."

"The structural insight is that a directory contains both files and other directories, and almost every operation is recursive. If I model files and directories as unrelated classes, every caller has to branch on which one it's holding, and that branch appears everywhere."

"So I'd use **Composite**: an abstract node type with two implementations. A file is a leaf whose size is its content length. A directory is a composite whose size is the sum of its children's sizes – and it doesn't need to know whether those children are files or directories. That one line gives me recursion for free with no type checks anywhere."

"The same property makes delete trivial. Removing a directory removes its entire subtree, because ownership is structural – I never write a recursive delete."

"Path resolution I'd keep in one place: split on slashes, ignore empty segments and dots, walk up on double-dot, and fail if a path segment turns out to be a file. Every public method – mkdir, write, resolve – goes through that single walk function, so path semantics are defined once rather than reimplemented per operation."

"For operations over the tree – printing, searching, disk usage – I'd use **Visitor** rather than adding methods to the node classes. And I want to be explicit about the trade-off, because Visitor is often applied where it doesn't belong. Visitor makes it cheap to add new **operations** and expensive to add new **node types**: a new node type forces edits to every visitor. Here that's the right bet, because file systems have exactly two node kinds and they don't change, while the list of things you want to do to a tree grows forever."

"If symlinks came into scope, that would be a third node type and I'd need cycle detection during traversal – a visited set of nodes. That's also where Visitor's cost would show up, since every existing visitor would need a visitSymlink method."

7. Follow-ups

అడిగేది	జవాబు
"Directory size ని cache చేయొచ్చా?"	అవును, కానీ invalidation సమస్య – ఒక file మారితే దాని పైన ఉన్న అన్ని directories ని invalidate చేయాలి. parent pointer ఉంది కాబట్టి పైకి నడవొచ్చు. చిన్న trees కి recursion చౌక, పెద్ద వాటికి cache విలువ చేస్తుంది
"Symlinks ఎలా?"	ఒక SymlinkNode – target path ని పట్టుకుంటుంది. Resolve చేసేటప్పుడు follow చేయాలి, cycle detection తప్పనిసరి (visited set + hop limit)
"Permissions?"	ప్రతి node కి owner , mode . Resolve path lo ప్రతి అడుగులో execute permission check – Unix కూడా అలానే చేస్తుంది

అడిగేది	జవాబు
"Thread safety?"	Tree mutation కి lock. Fine-grained కావాలంటే per-directory lock , కానీ deadlock ప్రమాదం (ఎప్పుడూ ఒకే క్రమంలో – root నుంచి కిందికి – lock తీసుకోవాలి)
"చాలా పెద్ద directory (1M files)?"	Map lookup $O(1)$ కాబట్టి get ఫర్వాలేదు. కానీ ls sorted చేయడం $O(n \log n)$ – sorted structure (B-tree) పెట్టడం లేదా pagination
"Real file system తో తేడా?"	Real ones లో inodes (metadata) మరియు directory entries (పేర్లు) వేరు – అందుకే hard links సాధ్యం. మన model లో name మరియు node కలిసి ఉన్నాయి, కాబట్టి hard links రావు

Design a Shopping Cart with Discounts

Difficulty **Medium** · Frequency ఎక్కువ (e-commerce companies) · నేర్పే concepts: pricing pipeline, rule ordering, explainability

1. The Ask

"Design the pricing engine for a shopping cart. Items, quantity discounts, category offers, coupons, shipping, tax."

ఈ **PROBLEM LO** అసలు కష్టం ఏమిటంటే

Pricing ఒక **సూత్రం (formula)** కాదు – అది ఒక **క్రమం (pipeline)**. క్రమం మారితే జవాబు మారుతుంది:

- Discount ముందు, tax తర్వాత → customer తక్కువ కడతాడు
- Tax ముందు, discount తర్వాత → customer ఎక్కువ కడతాడు

ఇది కేవలం technical నిర్ణయం కాదు – ఇది చట్టపరమైనది (GST నియమాలు discount తర్వాత tax అని చెప్తాయి). అందుకే ఈ design lo **rule ordering explicit గా, కనిపించేలా** ఉండాలి – code lo దాగి ఉండకూడదు.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఏ ఏ discount రకాలు?	Rule types	%, flat, BOGO, category, coupon
Discounts stack అవుతాయా?	Stacking నియమాలు సంక్లిష్టం	అవును, కానీ ఒక క్రమంలో
Coupon ఒక్కటే నా, పలువా?	Validation	ఒక్కటే
Tax ఎప్పుడు?	Discount తర్వాత – చట్టం	Discount తర్వాత
Shipping ఎప్పుడు?	Threshold ని ఏ మొత్తం మీద చూడాలి	Discount తర్వాతి మొత్తం మీద
"ఎందుకు ఈ ధర?" చూపించాలా?	Breakdown కావాలి – ఇది design ని మారుస్తుంది	అవును

3. Deep Dive – Decorator నా Rule Pipeline నా?

రెండు మార్గాలు • ఎందుకు PIPELINE గెలుస్తుంది

A • DECORATOR (nested)

```
new Gst(new Shipping(new Coupon(
new Bogo(new BasePrice(cart)))))
```

✓ ఒక్కో పొర స్వతంత్రం

X క్రమం nesting lo దాగి ఉంది – లోపలి నుంచి బయటికి చదవాలి. Config నుంచి build చేయడం కష్టం

B • RULE PIPELINE ✓

```
rules = [ Bogo, CategoryOff,
Coupon, Shipping, Gst ]
```

✓ క్రమం ఒక list – కనిపిస్తుంది, config అవుతుంది

✓ ప్రతి rule తన contribution ని record చేయొచ్చు

✓ Rules ని A/B test చేయొచ్చు, reorder చేయొచ్చు

నిర్ణయాత్మక కారణం: EXPLAINABILITY

Customer "నా ధర ఇలా ఎందుకు వచ్చింది?" అని అడుగుతాడు. Support team కూడా అడుగుతుంది.

Pipeline lo ప్రతి rule ఒక్కొక్క రాస్తుంది – "BOGO: -₹599", "GST 18%: +₹256". Decorator lo ఆ breakdown ని విడుదల అయ్యారంటే ప్రతి పొర దాస్తూ దాన్ని మోసుకెళ్ళాలి – అది nesting ని ఇంకా జటిలం చేస్తుంది.

4. Code

```
const rupees = (paise) => `₹${(paise / 100).toFixed(2)}`;

class LineItem {
  constructor(sku, name, unitPaise, qty, category) {
    Object.assign(this, { sku, name, unitPaise, qty, category });
  }
  get total() { return this.unitPaise * this.qty; }
}

class Cart {
  constructor(items = []) { this.items = items; }
  get subtotal() { return this.items.reduce((s, i) => s + i.total, 0); }
  itemsIn(category) { return this.items.filter((i) => i.category === category); }
}

// ప్రతి rule తన ప్రభావాన్ని ఇక్కడ నమోదు చేస్తుంది – ఇదే "ఎందుకు ఈ ధర" కి జవాబు.
class PriceBreakdown {
  constructor(subtotal) { this.subtotal = subtotal; this.total = subtotal; this.lines = []; }
  apply(label, deltaPaise) {
    if (deltaPaise === 0) return this;
    this.lines.push({ label, delta: deltaPaise });
    this.total += deltaPaise;
    return this;
  }
  explain() {
    return [`Subtotal: ${rupees(this.subtotal)}`,
      ...this.lines.map((l) => `${l.label}: ${l.delta < 0 ? '-' : '+'}${rupees(Math.abs(l.delta))}`),
      `Total: ${rupees(this.total)}`];
  }
}
```

```
class PricingRule {
  get name() { throw new Error('abstract'); }
```

```

    applies(cart, ctx) { return true; }
    apply(cart, breakdown, ctx) { throw new Error('abstract'); }
}

// "2 కొంటే 1 ఉచితం" - అత్యంత చౌక item ఉచితం (customer కి న్యాయమైనది)
class BuyXGetYFree extends PricingRule {
    constructor(category, buy, free) { super(); Object.assign(this, { category, buy, free }); }
    get name() { return `Buy ${this.buy} get ${this.free} free (${this.category})`; }
    apply(cart, breakdown) {
        const items = cart.itemsIn(this.category);
        const qty = items.reduce((s, i) => s + i.qty, 0);
        const freeCount = Math.floor(qty / (this.buy + this.free)) * this.free;
        if (freeCount === 0) return;
        const cheapest = Math.min(...items.map((i) => i.unitPaise));
        breakdown.apply(this.name, -cheapest * freeCount);
    }
}

class PercentOffCategory extends PricingRule {
    constructor(category, percent) { super(); Object.assign(this, { category, percent }); }
    get name() { return `${this.percent}% off ${this.category}`; }
    apply(cart, breakdown) {
        const amount = cart.itemsIn(this.category).reduce((s, i) => s + i.total, 0);
        breakdown.apply(this.name, -Math.round(amount * this.percent / 100));
    }
}

class CouponRule extends PricingRule {
    constructor(code, offPaise, minSubtotal) { super(); Object.assign(this, { code, offPaise, minSubtotal }); }
    get name() { return `Coupon ${this.code}`; }
    applies(cart, ctx) { return ctx.coupon === this.code && cart.subtotal >= this.minSubtotal; }
    apply(cart, breakdown) { breakdown.apply(this.name, -this.offPaise); }
}

class ShippingRule extends PricingRule {
    constructor(feePaise, freeAbove) { super(); Object.assign(this, { feePaise, freeAbove }); }
    get name() { return 'Shipping'; }
    apply(cart, breakdown) {
        // Threshold ని DISCOUNT తర్వాతి మొత్తం మీద చూస్తున్నాం - ఇది ఒక వ్యాపార నిర్ణయం,
        // మరియు ఇది explicit గా కనిపిస్తోంది, ఎక్కడో దాగి లేదు.
        breakdown.apply(this.name, breakdown.total >= this.freeAbove ? 0 : this.feePaise);
    }
}

class GstRule extends PricingRule {
    constructor(percent) { super(); this.percent = percent; }
    get name() { return `GST ${this.percent}%`; }
    apply(cart, breakdown) { breakdown.apply(this.name, Math.round(breakdown.total * this.percent / 100)); }
}

```

```

class PricingEngine {
  // rules యొక్క క్రమమే వ్యాపార నియమం. అది ఇక్కడ, ఒకేచోట, కనిపించేలా ఉంది.
  constructor(rules) { this.rules = rules; }
  price(cart, ctx = {}) {
    const breakdown = new PriceBreakdown(cart.subtotal);
    for (const rule of this.rules) {
      if (rule.applies(cart, ctx)) rule.apply(cart, breakdown, ctx);
    }
    return breakdown;
  }
}

```

```

// ---- వాడుక ----
const cart = new Cart([
  new LineItem('TS-1', 'T-shirt', 59900, 3, 'apparel'), // ₹599 × 3
  new LineItem('MG-1', 'Mug', 24900, 1, 'home'), // ₹249 × 1
]);

const engine = new PricingEngine([
  new BuyXGetYFree('apparel', 2, 1), // 3 తీసుకుంటే 1 ఉచితం
  new PercentOffCategory('home', 10),
  new CouponRule('SAVE100', 10000, 150000), // subtotal ₹1500+ అయితే ₹100 తగ్గింపు
  new ShippingRule(9900, 200000), // ₹2000 దాటితే ఉచితం
  new GstRule(18), // ← ఎప్పుడూ చివరన
]);

console.log(engine.price(cart, { coupon: 'SAVE100' }).explain().join('\n'));
// Subtotal: ₹2046.00
// Buy 2 get 1 free (apparel): -₹599.00 ← అతి చౌక apparel item ఉచితం
// 10% off home: -₹24.90
// Coupon SAVE100: -₹100.00
// Shipping: +₹99.00 ← discount తర్వాత ₹1322 → ₹2000 threshold దాటలేదు
// GST 18%: +₹255.80 ← discount తర్వాతి మొత్తం మీద
// Total: ₹1676.90

```

ఈ OUTPUT LO ఒక వ్యాపార పాఠం దాగి ఉంది

Subtotal ₹2046 – customer "₹2000 దాటింది, shipping ఉచితం" అని అనుకుంటాడు. కానీ discounts తర్వాత ₹1322 అయింది, కాబట్టి **shipping ₹99 పడింది**. Customer కోపగించుకుంటాడు.

ఇది **bug కాదు** – ఇది ఒక వ్యాపార నిర్ణయం. కానీ ఈ design యొక్క గొప్పతనం ఏమిటంటే – ఆ నిర్ణయం **ShippingRule** lo **ఒకే ఒక్క line గా కనిపిస్తోంది**. Product team "కాదు, subtotal మీద చూడాలి" అంటే – ఒక line మార్పు. Nested decorators lo ఇది ఎక్కడ ఉందో వెతకాలి.

`EXPLAIN()` ఎందుకు ఈ DESIGN LO ముఖ్యమైన భాగం

చాలా మంది pricing engine ని "ఒక number ఇచ్చే function" గా design చేస్తారు. కానీ నిజ ప్రపంచంలో మొదటి production bug ఇదే: "నా bill ₹1,847 ఎందుకు వచ్చింది?" – support team జవాబు చెప్పలేరు, engineer debugger పెట్టాల్సి వస్తుంది.

Breakdown ని ఒక first-class output గా చేయడం – అంటే ప్రతి rule తన ప్రభావాన్ని label తో నమోదు చేయడం – ఈ మొత్తం సమస్యని తొలగిస్తుంది. Interview lo ఇలా చెప్పండి: "నేను total ని కాదు, **breakdown** ని return చేస్తాను. Total అనేది దాని ఒక field మాత్రమే."

5. Extensibility Test

Interviewer: "Add a rule: loyalty members get 5% off, but it can't stack with coupons."

```
class LoyaltyDiscount extends PricingRule {
  constructor(percent) { super(); this.percent = percent; }
  get name() { return `Loyalty ${this.percent}%`; }
  // Stacking నియమం ఇక్కడ, ఒకేటే, స్పష్టంగా.
  applies(cart, ctx) { return Boolean(ctx.isLoyaltyMember) && !ctx.coupon; }
  apply(cart, breakdown) { breakdown.apply(this.name, -Math.round(breakdown.total *
    this.percent / 100)); }
}
// engine rules list lo coupon తర్వాత చేర్చడం – అంతే. ఏ rule నీ ముట్టుకోలేదు.
```

STACKING నియమాలు పెరిగితే – ఒక హెచ్చరిక

"A మరియు B కలవవు, కానీ B మరియు C కలుస్తాయి, D అన్నిటికీ రద్దు చేస్తుంది" – ఇలా పెరిగితే `applies()` lo ఉన్న conditions చిక్కుముడి అవుతాయి.

అప్పుడు తర్వాతి అడుగు: rules కి group మరియు priority ఇచ్చి, "ఒక group నుంచి ఒక్కటే apply అవుతుంది, అత్యధిక priority ఉన్నది" అనే meta-rule పెట్టడం. అంటే **pipeline ఒక చిన్న rule engine** గా పరిణామం చెందుతుంది. ఈ పరిణామాన్ని ముందే చూడగలగడం – అదే senior ఆలోచన.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Strategy	ప్రతి <code>PricingRule</code>	కొత్త offer = కొత్త class, సున్నా edits
Pipeline / Chain	<code>PricingEngine.rules</code>	క్రమం ఒక first-class concept – config, A/B, audit
Builder (సూచించినది)	Rules ని config నుండి build చేయడం	Campaign ని deploy లేకుండా మార్చడం
⚠️ Decorator వద్దు	–	Nesting క్రమాన్ని దాచేస్తుంది మరియు breakdown ని కష్టతరం చేస్తుంది

INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"Let me pin down the rules: what discount types, do they stack, is tax applied before or after discounts, and does the customer need to see **why** their total is what it is? That last question changes the design more than the others."

"The thing I'd emphasise first is that pricing isn't a formula, it's an **ordered pipeline**. Applying tax before discounts versus after gives different totals, and under GST that ordering is a legal requirement, not a preference. So the order must be explicit and visible rather than buried inside nested code."

"That's why I'd use a rule pipeline rather than decorators. Decorators would work, but the execution order ends up encoded in how deeply things are nested, which you have to read inside-out, and it's awkward to build from configuration. A list of rules makes the order a piece of data – I can log it, A/B test it, or load it from config."

"The other reason is explainability. I return a **breakdown**, not a number. Each rule appends a labelled line – 'buy two get one: minus five ninety-nine', 'GST eighteen percent: plus two fifty-six' – and the total is just one field on that object. The first production question anyone asks a pricing system is 'why is my bill this amount', and if the answer requires a debugger, the design has failed."

"Each rule has two methods: **applies**, which decides eligibility, and **apply**, which mutates the breakdown. Splitting those matters, because eligibility is where stacking rules live – 'loyalty discount doesn't combine with a coupon' is one line in one place rather than a condition smeared across the engine."

"A detail worth calling out: the shipping rule checks its free-delivery threshold against the **post-discount** total. That's a business decision with real revenue consequences, and in this design it's a visible line of code rather than an accident of ordering."

"If stacking rules get more complex – group A excludes group B, D overrides everything – I'd evolve this into a small rule engine by giving rules a group and a priority, with a meta-rule that only the highest-priority rule in each group applies. I wouldn't build that on day one, but I'd shape the interface so it's a natural next step rather than a rewrite."

"And all money is integer paise, for the same reason as the Splitwise problem – percentage discounts produce fractions, and floating point loses them."

7. Follow-ups

అడిగేది	జవాబు
"Rules ని DB నుంచి load చేయాలంటే?"	Rule ని { type, params } JSON గా store చేసి, ఒక factory దాన్ని object గా మార్చడం. Marketing team campaigns ని deploy లేకుండా మార్చేచ్చు
"ఒకే item మీద రెండు discounts పడితే?"	ఇదే stacking సమస్య. applies() lo అపడం, లేదా item స్థాయిలో "ఇప్పటికే discount అయింది" అని mark చేయడం (అప్పుడు breakdown item-wise కావాలి)

అడిగేది	జవాబు
"Rounding ఎక్కడ?"	ప్రతి rule తర్వాత round చేయడం (paise integer ఉంచడానికి). కానీ దీనివల్ల rules క్రమం మారితే total 1 paisa మారొచ్చు – దీన్ని అంగీకరించి, tests lo lock చేయాలి
"Cart మారితే ప్రతిసారీ మళ్ళీ లెక్కించాలా?"	అవును – pricing pure function గా ఉంచడం. Cache చేస్తే invalidation bugs. ఇది చౌక operation
"Price ని ఎప్పుడు lock చేయాలి?"	Checkout మొదలైనప్పుడు breakdown ని order తో store చేయడం. లేకపోతే payment మధ్యలో offer ముగిస్తే ధర మారుతుంది
"Tax ఒక్కో item కి వేరుగా ఉంటే?"	Breakdown ని item స్థాయికి తీసుకెళ్ళాలి – PriceBreakdown lo perItem map. Structure అదే, granularity మాత్రం పెరుగుతుంది

Design a Meeting Scheduler

Difficulty **Medium** · Frequency ఎక్కువ · నేర్పే concepts: interval overlap, half-open ranges, free-slot search

1. The Ask

"Design a meeting room booking system. Book rooms, prevent double-booking, suggest free slots, support recurring meetings."

ఈ PROBLEM LO ఒక క్లాసిక్ OFF-BY-ONE BUG

10:00-11:00 meeting ఉంది. ఇప్పుడు 11:00-12:00 book చేయొచ్చా? **ఖచ్చితంగా చేయొచ్చు** – మొదటిది 11:00 కి ముగుస్తుంది.

కానీ overlap check ని $a.start \leq b.end$ & $b.start \leq a.end$ అని రాస్తే – 11:00 $\leq$ 11:00 నిజం అవుతుంది → **తప్పుగా conflict చూపిస్తుంది.**

సరైనది: intervals ని **half-open** గా చూడటం – $[start, end)$. అంటే end క్షణం interval lo భాగం కాదు. అప్పుడు నియమం: $a.start < b.end$ & $b.start < a.end$. ఈ ఒక్క $<$ vs $\leq$ తేడాయే ఈ problem lo అత్యంత సాధారణమైన bug.

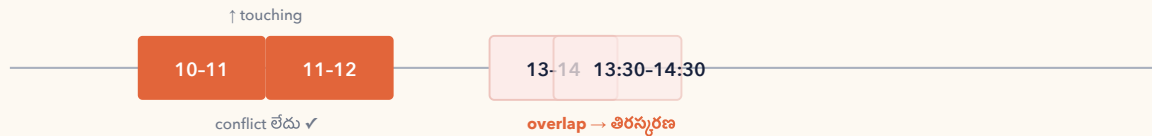
2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
Rooms ఎన్ని, capacity ఉందా?	Room selection logic	అవును, capacity తో
Recurring meetings?	Expansion + conflict అన్ని occurrences మీద	అవును
Timezones?	ఇది ఒక పెద్ద trap	ప్రస్తావిస్తాం
Attendee availability కూడా చూడాలా?	Room + అందరి calendars ని కలపడం	అవును (follow-up)
Buffer time (meetings మధ్య 5 నిమిషాలు)?	Interval ని పెంచడం	Extension గా
Cancel / reschedule?	Booking ని తీసేయడం	అవును

3. Deep Dive – Interval Overlap మరియు Free Slots

HALF-OPEN INTERVALS · TOUCHING ≠ OVERLAPPING

TIMELINE · 09:00 → 18:00



నియమం (half-open [start, end))

$\text{overlaps}(a, b) = a.\text{start} < b.\text{end} \ \&\& \ b.\text{start} < a.\text{end}$

← ఇక్కడ $\leq$ వాడితే touching meetings కూడా conflict అవుతాయి

FREE SLOTS · bookings మధ్య ఖాళీలు

09-10 ✓

booked 10-12

12-18 ✓ ← sorted bookings మీద ఒక్క pass, $O(n)$

4. Code

```
const t = (hhmm) => { const [h, m] = hhmm.split(':').map(Number); return h * 60 + m; };
const hhmm = (mins) => `${String(Math.floor(mins / 60)).padStart(2, '0')}:${String(mins % 60).padStart(2, '0')}`;

// Half-open [start, end) – ఇదే ఈ problem యొక్క మౌలిక నిర్ణయం.
class Interval {
  constructor(start, end) {
    if (end <= start) throw new Error('INVALID_INTERVAL');
    this.start = start; this.end = end;
  }
  get duration() { return this.end - this.start; }
  overlaps(other) { return this.start < other.end && other.start < this.end; }
  contains(other) { return this.start <= other.start && other.end <= this.end; }
  shift(minutes) { return new Interval(this.start + minutes, this.end + minutes); }
  toString() { return `${hhmm(this.start)}-${hhmm(this.end)}`; }
}

class Room { constructor(id, capacity) { this.id = id; this.capacity = capacity; } }
class Booking { constructor(id, roomId, interval, title) { Object.assign(this, { id, roomId, interval, title }); } }
```

```
// ఒక్క room కి ఒక schedule. Bookings ఎప్పుడూ start ప్రకారం sorted, మరియు ఎప్పుడూ overlap కావు.
class RoomSchedule {
  constructor() { this.bookings = []; }

  // start ≥ given ఉన్న మొదటి booking index – binary search,  $O(\log n)$ 
  #lowerBound(start) {
    let lo = 0, hi = this.bookings.length;
    while (lo < hi) {
      const mid = (lo + hi) >> 1;
      if (this.bookings[mid].interval.start < start) lo = mid + 1; else hi = mid;
    }
    return lo;
  }
}
```

```

    }

    isFree(interval) {
        const i = this.#lowerBound(interval.start);
        // Bookings ఒకదానితో ఒకటి overlap కావు కాబట్టి - పొరుగు రెండింటిని చూస్తే చాలు.
        if (i < this.bookings.length && this.bookings[i].interval.overlaps(interval))
            return false;
        if (i > 0 && this.bookings[i - 1].interval.overlaps(interval)) return false;
        return true;
    }

    add(booking) {
        if (!this.isFree(booking.interval)) throw new Error('ROOM_BUSY');
        this.bookings.splice(this.#lowerBound(booking.interval.start), 0, booking);
        return booking;
    }

    remove(id) { this.bookings = this.bookings.filter((b) => b.id !== id); }

    // window లోపల minDuration కంటే పెద్ద ఖాళీలు - sorted bookings మీద ఒక్క pass
    freeSlots(window, minDuration) {
        const slots = [];
        let cursor = window.start;
        for (const b of this.bookings) {
            if (b.interval.end <= window.start) continue;
            if (b.interval.start >= window.end) break;
            if (b.interval.start - cursor >= minDuration) slots.push(new Interval(cursor,
            b.interval.start));
            cursor = Math.max(cursor, b.interval.end);
        }
        if (window.end - cursor >= minDuration) slots.push(new Interval(cursor,
        window.end));
        return slots;
    }
}

```

```

class MeetingScheduler {
    constructor(rooms) {
        this.rooms = new Map(rooms.map((r) => [r.id, r]));
        this.schedules = new Map(rooms.map((r) => [r.id, new RoomSchedule()]));
        this.seq = 0;
    }

    book({ roomId, interval, title }) {
        const schedule = this.schedules.get(roomId);
        if (!schedule) throw new Error('NO_SUCH_ROOM');
        return schedule.add(new Booking(`B${++this.seq}`, roomId, interval, title));
    }

    // అతి చిన్న సరిపోయే room - పెద్ద rooms ని పెద్ద meetings కోసం ఖాళీగా ఉంచడం
    findRoom(interval, minCapacity) {
        return [...this.rooms.values()]
            .filter((r) => r.capacity >= minCapacity &&
            this.schedules.get(r.id).isFree(interval))
            .sort((a, b) => a.capacity - b.capacity)[0]?.id ?? null;
    }

    freeSlots(roomId, window, minDuration) { return

```

```

this.schedules.get(roomId).freeSlots(window, minDuration); }

// Recurring – అన్ని occurrences ఖాళీగా ఉంటేనే book (ALL OR NOTHING)
bookRecurring({ roomId, interval, title, everyDays, count }) {
  const dayMinutes = 24 * 60;
  const occurrences = Array.from({ length: count }, (_, i) => interval.shift(i *
everyDays * dayMinutes));
  const schedule = this.schedules.get(roomId);
  const clash = occurrences.find(o => !schedule.isFree(o));
  if (clash) throw new Error(`SERIES_CONFLICT_AT:${clash}`);
  return occurrences.map(o => schedule.add(new Booking(`B${++this.seq}`, roomId,
o, title)));
}
}

```

```

// ---- వాడుక ----
const scheduler = new MeetingScheduler([new Room('R1', 6), new Room('R2', 12)]);

scheduler.book({ roomId: 'R1', interval: new Interval(t('10:00'), t('11:00')), title:
'Standup' });

try { scheduler.book({ roomId: 'R1', interval: new Interval(t('10:30'), t('11:30')),
title: 'Clash' }); }
catch (e) { console.log(e.message); } // ROOM_BUSY

// 11:00 కి మొదలయ్యేది – touching, overlap కాదు → చెల్లుతుంది
scheduler.book({ roomId: 'R1', interval: new Interval(t('11:00'), t('12:00')), title:
'Review' });

console.log(scheduler.freeSlots('R1', new Interval(t('09:00'), t('18:00')),
60).map(String));
// [ '09:00-10:00', '12:00-18:00' ]

console.log(scheduler.findRoom(new Interval(t('10:30'), t('11:30')), 6)); // R2
console.log(scheduler.findRoom(new Interval(t('10:30'), t('11:30')), 20)); // null
← అంత పెద్ద room లేదు

```

‘FINDROOM’ LO ఒక చిన్న కానీ ముఖ్యమైన నిర్ణయం

సరిపోయే rooms lo నేను **అతి చిన్నదాన్ని** ఎంచుకుంటున్నాను (**sort by capacity**). ఎందుకంటే – 3 మంది meeting కి 20-సీట్ల board room ఇస్తే, తర్వాత వచ్చే 15 మంది meeting కి చోటు ఉండదు.

ఇది **best-fit allocation** – memory allocators, parking lot (Problem 01) lo కూడా ఇదే ఆలోచన.
 "సరిపోయేది ఏదైనా చాలు" కాకుండా "అతి తక్కువ వృథా చేసేది" ఎంచుకోవడం. Interview lo ఈ ఒక్క వాక్యం చెప్తే – మీరు resource allocation గురించి ఆలోచించారని అర్థమవుతుంది.

5. Scale – sorted array ఎప్పుడు సరిపోదు

Data structure	Insert	Conflict check	ఎప్పుడు
Sorted array ✓	$O(n)$ (splice)	$O(\log n)$	ఒక room కి రోజుకి ~20 meetings – ఇదే ఉత్తమం
Balanced BST / skip list	$O(\log n)$	$O(\log n)$	ఒక resource కి వేల bookings
Interval tree	$O(\log n)$	$O(\log n + k)$	"ఈ range lo ఉన్న అన్ని bookings" అనే query తరచుగా అవసరమైతే
Segment tree	$O(\log n)$	$O(\log n)$	"ఈ range lo గరిష్ట concurrent bookings" లాంటి aggregate queries

OVER-ENGINEERING ని ఇలా తప్పించండి

"Interval tree సరైన సమాధానం అనిపిస్తుంది, కానీ ఒక meeting room కి రోజుకి 20 bookings. $n = 20$ దగ్గర binary search మరియు linear scan మధ్య తేడా కొలవలేనిది, కానీ sorted array యొక్క code **పది రెట్లు సులభం**. నేను sorted array తో మొదలుపెట్టి, RoomSchedule ని ఒక interface వెనక ఉంచుతాను – ఎప్పుడైనా అవసరమైతే implementation మార్చొచ్చు." **సరైన సమయంలో సరైన సంక్లిష్టత** – ఇది maturity signal.

6. Timezones & Recurrence – దాచిన రాక్షసులు

ఈ మూడు TRAPS ని ప్రస్తావిస్తే మీరు ఇది నిజంగా రాశారని అర్థమవుతుంది

- 1. DST.** "ప్రతి రోజు ఉదయం 9కి" అనే recurring meeting – DST మారినప్పుడు UTC lo అది 8 లేదా 10 అవుతుంది. అందుకే recurring meetings ని **UTC timestamps గా కాకుండా, (local time + timezone + rule) గా** store చేసి, ప్రతిసారీ expand చేయాలి.
- 2. అనంత recurrence.** "ప్రతి వారం, ఎప్పటికీ" – అన్ని occurrences ని ముందే create చేయలేం. **Lazy expansion** – అడిగిన window కి మాత్రమే generate చేయడం. Google Calendar ఇలానే చేస్తుంది.
- 3. Series lo ఒక్క meeting ని మార్చడం.** "ఈ వారం మాత్రం 10కి" – అప్పుడు ఆ ఒక్క occurrence ఒక **exception** గా store అవుతుంది (rule + exceptions list). ఇది iCalendar RFC lo EXDATE / RECURRENCE-ID .

🔧 INTERVIEW SCRIPT · ఇలా చెప్పుండి (ENGLISH)

"Let me scope: rooms with capacities, recurring meetings, and do we also need to check attendee availability or just rooms? I'll do rooms first and then explain how attendees fold in. I'd also like to flag timezones early, because they're where this problem gets genuinely hard."

"The foundation is how I represent an interval, and I'd make it **half-open** – start inclusive, end exclusive. That sounds pedantic until you notice the bug it prevents: a meeting from ten to eleven and another from eleven to twelve don't conflict, but the naive overlap check with less-than-or-equals says they do. With half-open intervals the rule is simply that A starts before B ends and B starts before A ends, and touching meetings are correctly allowed."

"Each room gets its own schedule holding bookings sorted by start time, with the invariant that they never overlap. Given that invariant, checking a new booking only requires looking at the two neighbours found by binary search – if neither overlaps, nothing does."

"Free-slot search is a single pass over the sorted bookings, tracking a cursor and emitting the gaps that are long enough."

"When choosing a room I pick the **smallest** one that fits. Giving a three-person meeting the twenty-seat boardroom means the next fifteen-person meeting has nowhere to go. That's best-fit allocation, the same reasoning as choosing a parking spot."

"Recurring bookings are all-or-nothing: I expand the occurrences, check them all, and only then commit. Partially booking a series is worse than rejecting it, because the organiser thinks it's scheduled."

"On data structures – an interval tree is the textbook answer, but a meeting room has maybe twenty bookings a day. At that size, binary search versus linear scan is unmeasurable while the sorted array is far simpler code. I'd start with the array behind an interface, so swapping in a tree later is contained. Choosing the right complexity for the actual size is part of the design."

"The traps I'd want to name: daylight saving means 'every day at 9am local' isn't a fixed UTC offset, so recurring meetings must be stored as a local time plus a timezone plus a rule and expanded on read. Infinite recurrence can't be materialised, so expansion is lazy for the requested window. And editing one occurrence of a series needs an exceptions list – which is exactly what the iCalendar spec does."

7. Follow-ups

అడిగేది	జవాబు
"Attendee availability కూడా చూడాలంటే?"	ప్రతి user కి కూడా ఒక <code>Schedule</code> . Common free slots = అన్ని schedules యొక్క free slots ని intersect చేయడం. Interval intersection – అదే logic
"Concurrency – ఇద్దరు ఒకేసారి?"	<code>isFree</code> + <code>add</code> మధ్య race. Room కి ఒక lock, లేదా DB lo <code>UNIQUE</code> constraint + conditional insert (Problem 03 lo చేసినట్టు)

అడిగేది	జవాబు
"Buffer time (meetings మధ్య 5 నిమిషాలు)?"	Booking interval ని రెండువైపులా విస్తరించి conflict check చేయడం. Display lo మాత్రం అసలు సమయం చూపడం
"Room ని cancel చేస్తే waitlist?"	Observer – room ఖాళీ అయితే waitlist lo ఉన్నవాళ్ళకి notify (Problem 10 యొక్క event bus)
"గత meetings ని ఎంతకాలం ఉంచాలి?"	Schedule lo ఇటీవలివి మాత్రం memory lo, పాతవి DB lo. freeSlots ఎప్పుడూ భవిష్యత్తు గురించే
"1000 rooms, 100K bookings?"	Room ప్రకారం sharding సహజం – rooms స్వతంత్రం. అదే ఈ domain యొక్క అదృష్టం (Problem 16 HLD lo shows లాగే)

Design a Library Management System

Difficulty **Medium** · Frequency ఎక్కువ (fresher + lateral) · నేర్పే concepts: identity vs instance, reservation queue, fine policy

1. The Ask

"Design a library system. Members borrow books, return them, pay fines if late, and can reserve books that are currently out."

ఈ **PROBLEM 16** మొదటి ప్రశ్న – మరియు ఇక్కడ చాలా మంది ఓడతారు

"Library lo Clean Code ఉందా?" – ఇది **catalogue** ప్రశ్న.

"ఈ నిర్దిష్ట పుస్తకాన్ని ఎవరు తీసుకున్నారు?" – ఇది **భౌతిక వస్తువు** ప్రశ్న.

Library lo Clean Code యొక్క **5 copies** ఉండొచ్చు. అన్నిటికీ ఒకే ISBN, కానీ ఒక్కో దానికి ఒక్కో barcode, ఒక్కో స్థితి.

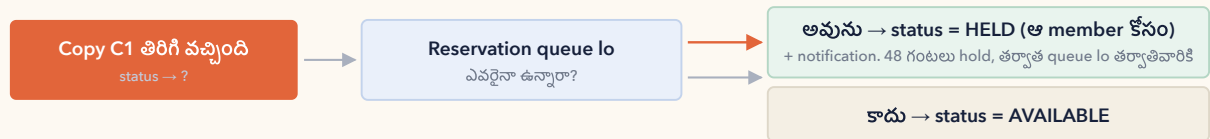
అందుకే **రెండు classes కావాలి**: **Book** (ISBN, title, author – catalogue) మరియు **BookCopy** (barcode, status – భౌతిక వస్తువు). ఇది సరిగ్గా Problem 03 lo Seat vs Show లాంటిదే – **identity vs instance**. ఒకే పాఠం, వేరే domain.

2. Clarifying Questions

ప్రశ్న	ఎందుకు ముఖ్యం	జవాబు
ఒక పుస్తకానికి పలు copies ఉంటాయా?	ఇదే పైన చెప్పిన మౌలిక నిర్ణయం	అవును
Member కి ఎన్ని పుస్తకాలు?	Borrow limit check	3
Loan period, fine ఎంత?	Policy – strategy గా	14 రోజులు, రోజుకి ₹5
Reservation ఉందా?	FIFO queue + hold logic	అవును
Fine కట్టకపోతే borrow చేయొచ్చా?	Business rule	లేదు
Search ఎలా?	Title, author, ISBN మీద index	సాధారణ index

3. Deep Dive – Reservation యొక్క దాచిన నియమం

పుస్తకం తిరిగి వచ్చినప్పుడు ఎవరికి ఇవ్వాలి?



ఇక్కడ చాలా మంది చేసే తప్పు

తిరిగి వచ్చిన copy ని వెంటనే AVAILABLE చేయడం. అప్పుడు - 3 నెలలు queue lo వేచి ఉన్న member కంటే, ఆ క్షణంలో counter దగ్గర నిలబడిన వ్యక్తి గెలుస్తాడు. **Reservation యొక్క అర్థం**

HELD కి TTL ఎందుకు

Member reserve చేసి ఎప్పటికీ రాకపోతే - ఆ copy శాశ్వతంగా ఇరుక్కుంటుంది. అందుకే 48 గంటల hold, తర్వాత queue lo తర్వాతివారికి. ఇది Problem 03 (seat hold), Problem 07 (uber)

TTL ఉన్న claim - మళ్ళీ అదే నమూనా.

4. Code

```
const DAY = 24 * 60 * 60 * 1000;
const CopyStatus = Object.freeze({ AVAILABLE: 'AVAILABLE', LOANED: 'LOANED', HELD: 'HELD', LOST: 'LOST' });

class Book {
  constructor(isbn, title, author) {
    Object.assign(this, { isbn, title, author });
  }
}

class BookCopy {
  constructor(barcode, isbn) {
    Object.assign(this, { barcode, isbn });
    this.status = CopyStatus.AVAILABLE;
    this.heldFor = null;
    this.holdUntil = 0;
  }
}

class Member {
  constructor(id, name) {
    Object.assign(this, { id, name });
    this.finesDue = 0;
  }
}

class Loan {
  constructor(id, barcode, memberId, issuedAt, dueAt) {
    Object.assign(this, { id, barcode, memberId, issuedAt, dueAt });
    this.returnedAt = null;
  }
  get isOpen() {
    return this.returnedAt === null;
  }
}

// Fine ని strategy గా - నియమాలు ఖచ్చితంగా మారతాయి (student vs staff, పండగ రాయితీలు)
class FinePolicy {
  fineFor(loan, now) {
    throw new Error('abstract');
  }
}

class PerDayFine extends FinePolicy {
  constructor(perDayPaise, graceDays = 0, capPaise = Infinity) {
    super();
    Object.assign(this, { perDayPaise, graceDays, capPaise });
  }
  fineFor(loan, now) {
    const lateDays = Math.ceil((now - loan.dueAt) / DAY) - this.graceDays;
    if (lateDays <= 0) return 0;
    return Math.min(lateDays * this.perDayPaise, this.capPaise); // cap - fine
  }
}

పుస్తకం ధర దాటకూడదు
```

```
class Library {
  constructor({ finePolicy, loanDays = 14, maxLoans = 3, holdHours = 48, clock }) {
    Object.assign(this, { finePolicy, loanDays, maxLoans, holdHours, clock });
    this.books = new Map(); // isbn -> Book
  }
}
```

```

    this.copies = new Map();          // barcode → BookCopy
    this.members = new Map();         // id → Member
    this.loans = new Map();           // loanId → Loan
    this.reservations = new Map();    // isbn → [memberId] (FIFO)
    this.seq = 0;
}

addBook(book) { this.books.set(book.isbn, book); return book; }
addCopy(barcode, isbn) { const c = new BookCopy(barcode, isbn);
this.copies.set(barcode, c); return c; }
addMember(m) { this.members.set(m.id, m); return m; }

#openLoansOf(memberId) { return [...this.loans.values()].filter((l) => l.isOpen &&
l.memberId === memberId); }
#copiesOf(isbn) { return [...this.copies.values()].filter((c) => c.isbn === isbn);
}

// ఈ member ఈ copy ని తీసుకోవచ్చా - hold నియమాలతో సహా
#claimable(copy, memberId, now) {
    if (copy.status === CopyStatus.AVAILABLE) return true;
    if (copy.status !== CopyStatus.HELD) return false;
    if (copy.heldFor === memberId) return true;
    return copy.holdUntil <= now;           // hold గడువు ముగిసింది → ఎవరైనా
    తీసుకోవచ్చు
}

borrow(memberId, isbn) {
    const now = this.clock.now();
    const member = this.members.get(memberId);
    if (!member) throw new Error('NO_SUCH_MEMBER');
    if (member.finesDue > 0) throw new Error(`FINES_DUE:${member.finesDue}`);
    if (this.#openLoansOf(memberId).length >= this.maxLoans) throw new
Error('LOAN_LIMIT_REACHED');

    const copy = this.#copiesOf(isbn).find((c) => this.#claimable(c, memberId, now));
    if (!copy) {
        const held = this.#copiesOf(isbn).some((c) => c.status === CopyStatus.HELD);
        throw new Error(held ? 'RESERVED_FOR_ANOTHER_MEMBER' : 'NO_COPY_AVAILABLE');
    }

    copy.status = CopyStatus.LOANED; copy.heldFor = null; copy.holdUntil = 0;
    const queue = this.reservations.get(isbn);
    if (queue?.[0] === memberId) queue.shift();           // తన reservation ని వాడుకున్నాడు

    const loan = new Loan(`L${++this.seq}`, copy.barcode, memberId, now, now +
this.loanDays * DAY);
    this.loans.set(loan.id, loan);
    return loan;
}

returnCopy(barcode) {
    const now = this.clock.now();
    const loan = [...this.loans.values()].find((l) => l.isOpen && l.barcode ===
barcode);
    if (!loan) throw new Error('NOT_ON_LOAN');
    loan.returnedAt = now;

    const fine = this.finePolicy.fineFor(loan, now);
    if (fine > 0) this.members.get(loan.memberId).finesDue += fine;
}

```

```

// ← ఇదే కీలకమైన నియమం: reservation queue ముందు, walk-ins తర్వాత
const copy = this.copies.get(barcode);
const queue = this.reservations.get(copy.isbn) ?? [];
if (queue.length > 0) {
    copy.status = CopyStatus.HELD;
    copy.heldFor = queue[0];
    copy.holdUntil = now + this.holdHours * 60 * 60 * 1000;
} else {
    copy.status = CopyStatus.AVAILABLE; copy.heldFor = null; copy.holdUntil = 0;
}
return { loanId: loan.id, fine, heldFor: copy.heldFor };
}

reserve(memberId, isbn) {
    if (this.#copiesOf(isbn).some((c) => c.status === CopyStatus.AVAILABLE)) throw
new Error('COPY_AVAILABLE_BORROW_IT');
    const queue = this.reservations.get(isbn) ?? [];
    if (queue.includes(memberId)) throw new Error('ALREADY_RESERVED');
    queue.push(memberId);
    this.reservations.set(isbn, queue);
    return { position: queue.length };
}

payFine(memberId, amount) { const m = this.members.get(memberId); m.finesDue =
Math.max(0, m.finesDue - amount); return m.finesDue; }
}

```

```

// ---- వాడుక ----
const clock = { t: 0, now() { return this.t; }, advanceDays(d) { this.t += d * DAY; }
};

const lib = new Library({ finePolicy: new PerDayFine(500, 0, 20000), loanDays: 14,
maxLoans: 3, clock });
lib.addBook(new Book('978-0132350884', 'Clean Code', 'Robert Martin'));
lib.addCopy('C1', '978-0132350884');
lib.addCopy('C2', '978-0132350884');
['m1', 'm2', 'm3', 'm4'].forEach((id, i) => lib.addMember(new Member(id, `Member ${i
+ 1}`)));

console.log(lib.borrow('m1', '978-0132350884').barcode); // C1
console.log(lib.borrow('m2', '978-0132350884').barcode); // C2

try { lib.borrow('m3', '978-0132350884'); } catch (e) { console.log(e.message); }
// NO_COPY_AVAILABLE
console.log(lib.reserve('m3', '978-0132350884'));
// { position: 1 }

clock.advanceDays(20); // 14 రోజుల గడువు చాటి 6 రోజులు
console.log(lib.returnCopy('C1'));
// { loanId: 'L1', fine: 3000, heldFor: 'm3' } ← ₹30 fine, మరియు copy m3 కేసో hold

try { lib.borrow('m4', '978-0132350884'); } catch (e) { console.log(e.message); }
// RESERVED_FOR_ANOTHER_MEMBER
console.log(lib.borrow('m3', '978-0132350884').barcode); // C1 ← queue lo
ముందున్నవాడికే

```

```
try { lib.borrow('m1', '978-0132350884'); } catch (e) { console.log(e.message); }  
// FINES_DUE:3000
```

ఈ DEMO LO నాలుగు నియమాలు ఏకకాలంలో నిరూపితమయ్యాయి

1. Copies అయిపోతే NO_COPY_AVAILABLE , మరియు reserve చేయమని సూచన.
2. Late return కి fine లెక్కించబడింది (6 రోజులు × ₹5 = ₹30).
3. తిరిగి వచ్చిన copy queue lo ఉన్నవాడికే hold అయింది – m4 ఎంత ముందు అడిగినా దొరకలేదు.
4. Fine కట్టకుండా m1 మళ్ళీ borrow చేయలేకపోయాడు.

5. Extensibility Test

Interviewer: "Students get 30 days and no fine for the first 3 days. Staff get unlimited books."

```
// Fine policy - కొత్త class, ఉన్నదాన్ని ముట్టుకోలేదు  
class TieredFinePolicy extends FinePolicy {  
  constructor(policyByTier, fallback) { super(); this.byTier = policyByTier;  
  this.fallback = fallback; }  
  fineFor(loan, now, member) { return (this.byTier.get(member?.tier) ??  
  this.fallback).fineFor(loan, now); }  
}  
  
// Loan నియమాలు కూడా tier ఆధారంగా - ఇవి కూడా ఒక policy కావాలి  
class LoanPolicy {  
  maxLoans(member) { return 3; }  
  loanDays(member) { return 14; }  
}  
class TieredLoanPolicy extends LoanPolicy {  
  maxLoans(m) { return m.tier === 'STAFF' ? Infinity : m.tier === 'STUDENT' ? 5 : 3;  
  }  
  loanDays(m) { return m.tier === 'STUDENT' ? 30 : 14; }  
}
```

ఇక్కడ నా DESIGN LO ఒక లోపం - దీన్ని మరే చెప్పండి

నేను loanDays మరియు maxLoans ని Library constructor lo సాధారణ numbers గా పెట్టాను. Tier-based నియమాలు వచ్చినప్పుడు ఇవి సరిపోవు - వీటిని ఒక LoanPolicy object గా చేయాల్సి వచ్చింది.

Interview lo ఇలా చెప్పండి: "Fine ని నేను policy గా చేశాను ఎందుకంటే అది మారుతుందని ఊహించాను. కానీ loan period ని plain number గా వదిలేశాను - అది తప్పు. **ఏవి మారతాయో ఊహించడంలో నేను ఒకచోట తప్పును,** మరియు అది ఇప్పుడు కనిపిస్తోంది." స్వంత design లోపాన్ని గుర్తించడం - ఇది interviewer కి చాలా బలమైన signal, ఎందుకంటే production lo మీరు ఇలానే ఆలోచిస్తారని అర్థమవుతుంది.

6. Patterns వాడినవి

Pattern	ఎక్కడ	ఎందుకు
Strategy	<code>FinePolicy</code> , <code>LoanPolicy</code>	నియమాలు tier ప్రకారం, కాలం ప్రకారం మారతాయి
State (సరళమైనది)	<code>CopyStatus</code> + <code>claimable()</code>	AVAILABLE/LOANED/HELD మధ్య చెల్లుబాటు transitions
Observer (extension)	Copy ఖాళీ అయితే notify	Problem 10 యొక్క event bus ఇక్కడ నేరుగా వాడొచ్చు
Repository (సూచించినది)	<code>books</code> , <code>copies</code> , <code>loans</code> maps	ఇవి DB queries గా మారతాయి – interface వెనక ఉంచడం మేలు

🔧 INTERVIEW SCRIPT · ఇలా చెప్పండి (ENGLISH)

"The first question I'd settle is whether a title can have multiple physical copies, because that decides the whole entity model. Assuming yes, I need **two** classes: a Book, which is the catalogue entry keyed by ISBN, and a BookCopy, which is a physical object with a barcode and a status. 'Do we have Clean Code' and 'who has this particular copy' are different questions, and collapsing them into one class is the classic mistake here."

"That's the same distinction as seats versus a show in the ticket booking problem – identity versus instance. Different domain, same modelling lesson."

"Borrowing then has a small set of guards: does the member exist, do they owe fines, are they at their loan limit, and is a claimable copy available. I'd keep those as explicit early returns with named errors, because 'you can't borrow' has several different causes and the counter staff needs to know which."

"The part worth dwelling on is reservations. When a copy comes back, the naive implementation marks it available – and then whoever happens to be standing at the desk gets it, ahead of someone who has been queuing for three months. That makes reservations meaningless. So on return, I check the reservation queue first and put the copy into a **held** state for the person at the head."

"But a hold needs a deadline. If the member never turns up, that copy is stranded forever. So the hold expires after forty-eight hours and passes to the next person. That's the same TTL-claim pattern as a seat hold or a driver lock – a claim without an expiry is a leak."

"Fines go behind a policy interface, because fine rules change constantly – different tiers, grace periods, amnesty periods. I'd also cap the fine, since a fine exceeding the replacement cost of the book is nonsense."

"If you ask me to add student and staff tiers, I'd note something about my own design: I made fines a policy because I anticipated change, but I left loan period and loan limit as plain constructor numbers – and those turn out to vary by tier too. So I got the prediction right in one place and wrong in another, and the fix is to promote them into a loan policy object. Being able to see where you mis-predicted the axis of change is most of what improves a design over time."

7. Follow-ups

అడిగేది	జవాబు
"Search ఎలా?"	Title/author మీద inverted index (Map<token, Set<isbn>>). నిజ system lo Elasticsearch. ఇది ఒక వేరే component, core model ని మార్చదు
"పుస్తకం పోగొట్టుకుంటే?"	CopyStatus.LOST + replacement fee. Copy ని catalogue నుంచి తీసేయకూడదు – చరిత్ర ఉండాలి. LOST copies #copiesOf filter lo రావు
"Renewal (గడువు పొడిగింపు)?"	Reservation queue ఖాళీగా ఉంటేనే allow. లేకపోతే queue lo ఉన్నవాళ్ళు అన్యాయానికి గురవుతారు – ఇది product నియమం

అడిగేది	జవాబు
"Notification ఎలా?"	Problem 10 యొక్క event bus – <code>COPY_HELD</code> , <code>DUE_SOON</code> , <code>OVERDUE</code> events. Library కి email గురించి తెలియనవసరం లేదు
"Concurrency – ఇద్దరు ఒకేసారి చివరి copy ని?"	<code>#claimable</code> check మరియు status update మధ్య race. Copy row మీద conditional update (Problem 03 లాగే)
"పలు branches?"	<code>BookCopy</code> కి <code>branchId</code> . Inter-branch transfer అంటే ఒక కొత్త state (<code>IN_TRANSIT</code>) – state machine సహజంగా విస్తరిస్తుంది



16 problems తర్వాత – మీరు ఏం నేర్చుకున్నారు

ఇది revision page · interview ముందు రోజు ఇది ఒక్కటే చదివితే చాలు

తిరిగి తిరిగి వచ్చిన 8 ఆలోచనలు

ఆలోచన	ఏ ఏ problems lo	సారాంశం
Identity vs Instance	03 seat/show · 16 book/copy	"ఇది ఏమిటి" (catalogue) మరియు "ఇది ఏ స్థితిలో ఉంది" (భౌతిక వస్తువు) – రెండూ వేరే classes
TTL ఉన్న claim (lock కాదు)	03 seat hold · 16 book hold · 08 token	గడువు లేని claim = శాశ్వతంగా ఇరుక్కున్న resource. ఎప్పుడూ expiry పెట్టండి
Plan, తర్వాత Commit	06 ATM · 08 composite limiter	మార్చే ముందు "సాధ్యమా" అని ఏమీ మార్చకుండా చూడటం
State ని object గా చేయడం	02 elevator · 05 vending · 06 ATM	Behaviour state ని బట్టి మారితే – if-else కాదు, class
Polymorphism, switch కాదు	01 spot · 11 chess piece · 13 fs node	కొత్త రకం = కొత్త class, ఉన్న code lo సున్నా edits
మారేదాన్ని బయట పెట్టడం	01, 02, 08, 14, 16 (అన్నీ Strategy)	"ఇది మారుతుంది" అని ఊహించిన చోట pattern. ఊహించని చోట వద్దు
Command = undo + simulate + audit	11 chess · 12 editor · 06 ATM	ఒక action ని object చేస్తే మూడు ప్రయోజనాలు ఒకేసారి
Composite: leaf ని container ని ఒకేలా	13 file system · 12 macro · 08 composite limiter	Recursion ఉచితం, caller lo type checks సున్నా

Interview lo ఎప్పుడూ చెప్పాల్సిన 6 వాక్యాలు

ఈ ఆరు వాక్యాలు మీ LEVEL ని మార్చేస్తాయి

1. "ఇది scope బయట పెడతాను" – మీరే scope ని కుదించడం maturity.
2. "ఈ pattern ని ఇక్కడ పెడుతున్నాను ఎందుకంటే ఇది మారుతుందని ఊహిస్తున్నాను" – pattern కి కారణం.
3. "ఇక్కడ ఒక race condition ఉంది" – concurrency ని మీరే లేవనెత్తడం.
4. "నా design lo ఇది ఒక లోపం" – స్వంత లోపాన్ని గుర్తించడం.
5. "ఇది optimal కాదు కానీ n చిన్నది కాబట్టి సరిపోతుంది" – సరైన సంక్లిష్టత ఎంచుకోవడం.
6. "ఒక్క existing class కూడా మార్చకుండా ఇది add చేయగలను" – Open/Closed ని నిరూపించడం.

చివరి 60 సెకన్ల CHECKLIST

1. Clarifying questions అడిగానా? Scope కుదించానా?
2. Nouns → classes చేసేటప్పుడు అనవసరమైన nouns ని వదిలేశానా?
3. ప్రతి class కి ఒకే బాధ్యత ఉందా? (God class లేదు కదా)
4. Enum + if-else chain ఎక్కడైనా ఉందా? దాన్ని polymorphism గా మార్చానా?
5. ప్రతి pattern కి "ఏం మారుతుందని" అనే కారణం చెప్పానా?
6. Concurrency / race conditions ప్రస్తావించానా?
7. Code నిజంగా run అవుతుందా? (edge cases, validation)
8. "కొత్త requirement వస్తే ఏమవుతుంది" అని చూపించానా?

చివరి మాట: LLD interview lo perfect code రాయడం లక్ష్యం కాదు – 45 నిమిషాల్లో అది ఎవరూ చేయలేరు. లక్ష్యం ఒక్కటే: **"ఈ మనిషితో కలిసి పని చేయగలను"** అని interviewer అనుకోవడం. అందుకే ఆలోచనని బయటికి చెప్పండి, trade-offs ని అంగీకరించండి, మీ లోపాలని మీరే చూపించండి. **Code కంటే ఆలోచనే ముఖ్యం.**